

Study Guide: Introduction to Algorithms (CLRS 4th Edition)

Generated 2026-06-04. Subject: Computer Science / Algorithms.
Exam format: Mixed (problem-solving, short answer, MCQ).
Coverage: comprehensive. Target length: ~5000 lines.

Chapter-by-Chapter Breakdown

Ch. 1 — The Role of Algorithms in Computing

Named Entities (Terms & Definitions) - Algorithm: well-defined computational procedure that takes input and produces output in finite time - **Correct algorithm:** halts with correct output for every problem instance - **Problem instance:** input needed to compute a solution - **Sorting problem formal definition:** Input: sequence $\langle a_1, a_2, \dots, a_n \rangle$. Output: permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ with $a'_1 \leq a'_2 \leq \dots \leq a'_n$ - **Data structure:** way to store and organize data to facilitate access and modifications - **NP-complete:** problems with no known efficient algorithm; if efficient algorithm exists for one, then for all; use approximation algorithms instead - **Online algorithms:** input arrives over time, must decide without knowing future - **Task-parallel algorithms:** designed for multicore processors (Ch 26)

Key Comparisons - Efficient algorithms matter more than fast hardware: insertion sort $\Theta(n^2)$ vs merge sort $\Theta(n \lg n)$. At $n=10M$, a slow computer running merge sort can beat a fast computer running insertion sort (17x faster) - Key comparison: insertion sort ($c_1 \cdot n^2$) vs merge sort ($c_2 \cdot n \cdot \lg n$). Crossover point exists where merge sort becomes faster despite larger constant

Applications - Human Genome Project (DP for DNA similarity), internet routing (shortest paths), e-commerce (RSA cryptography), resource allocation (linear programming), compression (Huffman coding) - Sorting is fundamental operation; algorithm choice depends on n , pre-sortedness, value restrictions, architecture, storage type - Traveling-salesperson problem (NP-complete): find shortest route visiting all cities and returning

Ch. 2 — Getting Started

Named Entities (Terms & Definitions) - Sorting problem: Input: sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$. Output: permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$. - **Key:** The value to be sorted. **Satellite data:** Associated data that travels with the key. **Record:** A key plus satellite data. - **Pseudocode:** Uses indentation for block structure, `//` for comments, `=` for assignment, `:` for subarray notation. - **Insertion sort:** Builds sorted array one element at a time by inserting next element into correct position. - **Loop**

invariant: Property that holds before each iteration; used to prove correctness (Initialization, Maintenance, Termination). - **RAM model:** Generic one-processor computational model; instructions execute one after another, each taking constant time. - **Input size:** For sorting, number n of items; for integer multiplication, number of bits. - **Running time:** Number of instructions and data accesses executed. - **Order of growth:** Rate at which running time increases as n grows; focus on leading term, ignoring constants and lower-order terms. - **Θ -notation (informal):** "Roughly proportional when n is large." - **Incremental method:** Build solution by adding one element at a time (insertion sort). - **Divide-and-conquer:** Break into subproblems, solve recursively, combine solutions. - **Merge sort:** Divide-and-conquer sort: divide in half, recursively sort each, merge sorted halves. - **Inversion:** Pair (i, j) with $i < j$ and $A[i] > A[j]$. - **Theorem 2.1:** Any comparison sort can be augmented to count inversions in $O(n \lg n)$ time. - **Bubble sort** sorts by swapping adjacent inversions; $\Theta(n^2)$ compares/swaps worst-case. - **Number of inversions:** Any sequence has between 0 and $C(n, 2)$ inversions. Average: $n(n-1)/4$.

HORNER'S RULE (Problem 2-3): Evaluate polynomial $P(x) = \sum a_i x^i$ in $\Theta(n)$.

```

y = 0
for i = n down to 0
    y = a_i + x·y

```

- Loop invariant: $y = \sum_{k=i}^n a_k x^{k-i}$ after iteration i .
- **Naive evaluation:** $\Theta(n^2)$. Horner: $\Theta(n)$. Key difference: no repeated exponentiation.

INSERTION-SORT(A, n) - Input: Array $A[1:n]$, integer n . Output: $A[1:n]$ sorted. - Steps: 1. for $i = 2$ to n 2. $key = A[i]$ 3. $j = i - 1$ 4. while $j > 0$ and $A[j] > key$ 5. $A[j+1] = A[j]$ 6. $j = j - 1$ 7. $A[j+1] = key$ - **Loop invariant:** At start of each iteration, $A[1:i-1]$ is sorted and contains original elements. - **Complexity:** Best $\Theta(n)$, worst $\Theta(n^2)$, average $\Theta(n^2)$.

MERGE(A, p, q, r) - Merge two sorted subarrays $A[p:q]$ and $A[q+1:r]$ into sorted $A[p:r]$. - Steps: Copy to L and R arrays, then merge by comparing smallest remaining. - Complexity: $\Theta(n)$ where $n = r-p+1$. - **Invariant:** $A[p:k-1]$ contains $k-p$ smallest elements of L and R in sorted order.

MERGE-SORT(A, p, r)

```

if p ≥ r
    return
q = ⌊(p+r)/2⌋
MERGE-SORT(A, p, q)
MERGE-SORT(A, q+1, r)
MERGE(A, p, q, r)

```

- Recurrence: $T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \lg n)$.
- **Proof by recursion tree:** $n + n + \dots + n$ ($\lg n$ times) = $n \lg n$.
- **Space:** $\Theta(n)$ auxiliary for merge.

Selection Sort

```
for i = 1 to n-1
  min = i
  for j = i+1 to n
    if A[j] < A[min]
      min = j
  swap A[i] with A[min]
```

- Loop invariant: $A[1:i-1]$ sorted with $i-1$ smallest elements.
- Complexity: $\Theta(n^2)$ worst, best, and average.
- In-place, not stable (long-distance swaps).

Binary Search

```
left = 1, right = n
while left ≤ right
  mid = ⌊(left+right)/2⌋
  if A[mid] == key return mid
  if A[mid] < key left = mid+1
  else right = mid-1
return NIL
```

- Precondition: A is sorted. $\Theta(\lg n)$ worst-case.
- Invariant: $\text{key} \in A[\text{left}:\text{right}]$ if $\text{key} \in A$ at all. Correctness via bisection.
- Variant: interpolation search $O(\lg \lg n)$ for uniform distribution.

Loop Invariant Proof Pattern 1. Initialization: Invariant true before first iteration 2. **Maintenance:** If true before iteration, remains true before next 3. **Termination:** Loop terminates; invariant + termination gives correctness

Comparisons	Dimension	Insertion Sort	Merge Sort	
Worst-case	$\Theta(n^2)$	$\Theta(n \lg n)$	Best-case	$\Theta(n)$
				$\Theta(n \lg n)$
In-place	Yes	No	Method	Incremental
				Divide-and-conquer

Formulas: Merge sort recurrence $T(n) = 2T(n/2) + \Theta(n)$. General D&C: $T(n) = aT(n/b) + D(n) + C(n)$.

End-of-Chapter: Exercises 2.1-1 through 2.3-6; Problems 2-1 (coarsening), 2-2 (bubblesort), 2-3 (Horner's rule), 2-4 (inversions).

Ch. 3 — Characterizing Running Times

Asymptotic Notations - O-notation (big-oh): $O(g(n)) = \{f(n): \exists c > 0, n_0 > 0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$. Asymptotic upper bound. - **Ω -notation** (big-omega): $\Omega(g(n)) = \{f(n): \exists c > 0, n_0 > 0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$. Asymptotic lower bound. - **Θ -notation** (theta): $\Theta(g(n)) = \{f(n): \exists c_1, c_2 > 0, n_0 > 0 \text{ such that } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n)\}$. Tight bound. - **o -notation** (little-oh): $f(n) \in o(g(n))$ if $\forall c > 0 \exists n_0: 0 \leq f(n) < cg(n)$. Equivalently $\lim f(n)/g(n) = 0$. Not asymptotically tight. - **ω -notation** (little-omega): $f(n) \in \omega(g(n))$ if $\forall c > 0 \exists n_0: 0 \leq cg(n) < f(n)$. Equivalently $\lim f(n)/g(n) = \infty$.

Theorem 3.1: $f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$.

Asymptotic Analogy to Real Numbers: $O = \leq$, $\Omega = \geq$, $\Theta = =$, $o = <$, $\omega = >$. **Asymptotic trichotomy does NOT hold** (e.g., n and $n^{(1+\sin n)}$ are incomparable).

Properties: Transitivity, reflexivity, symmetry (Θ only), transpose symmetry.

- **Transpose symmetry:** $f = O(g) \Leftrightarrow g = \Omega(f)$. $f = o(g) \Leftrightarrow g = \omega(f)$. - **Use in equations:** $2n^2 + 3n + 1 = \Theta(n^2)$ means $\exists c_1, c_2, n_0$ with $c_1 \cdot n^2 \leq 2n^2 + 3n + 1 \leq c_2 \cdot n^2$. - **Multiple variables:** $f(m, n) = O(g(m, n))$ if $\exists c, m_0, n_0$ with $f(m, n) \leq c \cdot g(m, n)$ for all $m \geq m_0, n \geq n_0$.

Limits and Asymptotics: - If $\lim_{n \rightarrow \infty} f(n)/g(n) = 0 \rightarrow f = o(g)$ - If $\lim_{n \rightarrow \infty} f(n)/g(n) = c > 0 \rightarrow f = \Theta(g)$ - If $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty \rightarrow f = \omega(g)$ - L'Hôpital's rule: $\lim f/g = \lim f'/g'$ when limits exist.

Standard Function Growth: - **Constant functions:** $f(n) = c \rightarrow \Theta(1) < \Theta(\lg n)$ - **Logarithmic:** $\lg^i n$, $i \in \mathbb{N}$. All logarithms are Θ of each other: $\log_a n = \Theta(\log_b n)$ for $a, b > 1$. - **Polylog vs polynomial:** $\lg^b n = o(n^a)$ for any $a > 0, b > 0$. - **Polynomial vs exponential:** $n^b = o(a^n)$ for any $a > 1, b > 0$. - **Factorial:** $n! = \omega(2^n)$ and $n! = o(n^n)$. - **Growth order:** $1 < \lg n < \sqrt{n} < n < n \lg n < n\sqrt{n} < n^2 < n^3 < 2^n < n!$ - Polynomial: $p(n) = \Theta(n^d)$ (leading term) - Exponentials ($a > 1$) grow faster than any polynomial: $\lim n^b / a^n = 0$ - Polynomials grow faster than polylogs: $\lim \lg^b n / n^a = 0$ - Stirling's approximation: $n! = \sqrt{(2\pi n)}(n/e)^n (1 + \Theta(1/n))$ - Iterated logarithm $\lg^* n$: $\min\{i \geq 0: \lg^{(i)} n \leq 1\}$ - Fibonacci: $F_i = \lfloor \phi^i / \sqrt{5} + 1/2 \rfloor$, $\phi = (1 + \sqrt{5})/2 \approx 1.618$

Pitfalls: $O(n^2) \neq \Theta(n^2)$; saying "insertion sort running time is $\Theta(n^2)$ " is wrong (best case $\Theta(n)$); asymptotic notation in inductive hypothesis without explicit constants leads to false proofs.

End-of-Chapter: Exercises 3.1-1 through 3.3-9; Problems 3-1 through 3-7.

Ch. 4 — Divide-and-Conquer

Recurrence Solving Methods

1. Substitution Method: Guess solution, prove by induction. Key: use explicit constants, subtract lower-order term if proof fails.

2. Recursion-Tree Method: Draw tree with costs at nodes, sum per level, sum across levels. Best for generating guesses.

3. Master Method: For $T(n) = aT(n/b) + f(n)$, $a > 0, b > 1$:
| Case |
Condition	Solution	
1	$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a} \lg^k n)$	$T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ AND $af(n/b) \leq cf(n)$ for $c < 1$	$T(n) = \Theta(f(n))$

Watershed function: $n^{\log_b a}$. Case 1: leaves dominate. Case 2: tied. Case 3: root dominates.

Examples: - $T(n) = 9T(n/3) + n \rightarrow \text{Case 1} \rightarrow \Theta(n^2)$ - $T(n) = T(2n/3) + 1 \rightarrow \text{Case 2} \rightarrow \Theta(\lg n)$ - $T(n) = 3T(n/4) + n \lg n \rightarrow \text{Case 3} \rightarrow \Theta(n \lg n)$ - $T(n) = 2T(n/2) + n \lg n \rightarrow \text{Case 2, } k=1 \rightarrow \Theta(n \lg^2 n)$ - $T(n) = 7T(n/2) + \Theta(n^2) \rightarrow \text{Case 1} \rightarrow \Theta(n^{\lg 7}) = O(n^{2.81})$

Gaps: Between cases 1-2 ($f(n) = \Theta(n^{\log_b a} / \lg n)$) and cases 2-3 ($f(n) = \Theta(n^{\log_b a} \cdot \lg n)$) but no polynomial separation).

4. Akra-Bazzi Method: For $T(n) = \sum a_i T(n/b_i) + f(n)$. Step 1: find p with $\sum a_i b_i^{-p} = 1$. Step 2: $T(n) = \Theta(n^p + n^p \int_1^n f(x)/x^{p+1} dx)$.

Strassen's Algorithm: Multiply $n \times n$ matrices in $O(n^{\lg 7}) \approx O(n^{2.81})$. - **Key idea:** 7 recursive multiplications instead of 8, using 10 sum/difference matrices. - Steps: Create $S_1 \dots S_{10}$, compute $P_1 \dots P_7$ recursively, combine into $C_{11} \dots C_{22}$. - Recurrence: $T(n) = 7T(n/2) + \Theta(n^2) \rightarrow \Theta(n^{\lg 7})$.

MATRIX-MULTIPLY-RECURSIVE: 8 recursive calls $\rightarrow T(n) = 8T(n/2) + \Theta(1) \rightarrow \Theta(n^3)$ — no faster than basic.

Polynomial-Growth Condition: f satisfies if $f(\Theta(n)) = \Theta(f(n))$. Required for Akra-Bazzi. Exponential 2^n does NOT satisfy.

Edge Cases: Floors/ceilings can usually be ignored. Base cases $T(n) = \Theta(1)$ for $n < n_0$. Master theorem regularity condition must be checked for Case 3. - **Master Theorem pitfalls:** $T(n) = 2T(n/2) + n \lg n$ falls between Case 2 ($k=0: \Theta(n \lg n)$) and Case 3 ($n \lg n \neq \Omega(n^{\lg 2 + \epsilon})$). Correct answer: $\Theta(n \lg^2 n)$ via Case 2 with $k=1$. - **Akra-Bazzi examples:** $T(n) = T(n/2) + T(n/4) + n$. Find p : $(1/2)^p + (1/4)^p = 1 \rightarrow p = 1$. Then $T(n) = \Theta(n + n \int_1^n (x/x^2) dx) = \Theta(n)$. - **Polynomial vs exponential recurrence:** $T(n) = 2T(n-1) + O(2^n) \rightarrow T(n) = \Theta(n \cdot 2^n)$. Decay transform: set $T(n) = 2^n \cdot S(n)$.

Maximum-Subarray Problem (Section 4.1): - **Problem:** Find contiguous subarray with largest sum in $A[1:n]$. - **Brute force:** $\Theta(n^2)$. **Crossing midpoint:** $\Theta(n)$. **Divide-and-conquer:** $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$. - **Kadane's algorithm** (linear):
 $\text{max_ending_here} = \text{max_so_far} = A[1]$
for $i = 2$ to n $\text{max_ending_here} = \max(A[i], \text{max_ending_here} + A[i])$
 $\text{max_so_far} = \max(\text{max_so_far}, \text{max_ending_here})$
return max_so_far $\Theta(n)$ — optimal for this problem.

End-of-Chapter: Exercises 4.1-1 through 4.7-6; Problems 4-1 through 4-7.

Ch. 5 — Probabilistic Analysis and Randomized Algorithms

Key Concepts - Probabilistic analysis: Average over input distribution. - **Randomized algorithm:** Behavior determined by both input and random-number generator. - **Indicator random variable:** $I\{A\} = 1$ if event A occurs, 0 otherwise. $E[I\{A\}] = \Pr\{A\}$. - **Linearity of expectation:** $E[\sum X_i] = \sum E[X_i]$ (no independence required). - **Uniform random permutation:** Each of $n!$ permutations equally likely.

HIRE-ASSISTANT(n): Interview n candidates, hire when better than current best. - Expected number of hires: $\sum 1/i = \ln n + O(1)$. Expected cost:

$O(c_h \cdot \ln n)$. - **RANDOMIZED-HIRE-ASSISTANT**: Randomly permute first $\rightarrow$ guaranteed $O(c_h \cdot \ln n)$ for any input.

RANDOMLY-PERMUTE(A, n): For $i = 1$ to n , swap $A[i]$ with $A[\text{RANDOM}(i, n)]$. $\Theta(n)$. Produces uniform random permutation (Lemma 5.4).

Birthday Paradox: With 23 people, $\Pr[\text{shared birthday}] \geq 1/2$. Expected matching pairs: $C(k, 2) \cdot (1/n)$. Threshold for expectation ≥ 1 : $k \geq (1 + \sqrt{1 + 8n})/2$.

Balls and Bins: Expected balls in bin: n/b . Expected tosses to fill all b bins: $b \cdot H_b \approx b \ln b$.

Streaks: Expected longest streak in n coin flips: $\Theta(\lg n)$. $\Pr\{\text{streak} \geq 2 \lceil \lg n \rceil\} \leq 1/n$.

Online Hiring (Secretary Problem): Interview first k , reject all; then hire first better. Optimal $k = n/e$, success probability $\geq 1/e$.

Edge Cases: Worst-case hiring: strictly increasing quality $\rightarrow$ hire every candidate. Proving each element equally likely in each position ($1/n$) is NOT sufficient for uniform permutation (need all permutations equally likely, not just each position).

Key Results: 23 people $\rightarrow$ 50% birthday match; $b \cdot \ln b$ tosses to fill bins; longest streak in 1000 flips ≥ 20 has probability $\leq 1/1000$; optimal hiring 37% rejection $\rightarrow$ 37% success.

Randomized Algorithm Techniques: - **Monte Carlo**: May produce wrong result with bounded probability (e.g., Miller-Rabin). - **Las Vegas**: Always correct, running time is random variable (e.g., RANDOMIZED-QUICKSORT). - **Derandomization**: Convert randomized algorithm to deterministic using method of conditional expectations (e.g., MAX-3-CNF-SAT). - **Probabilistic method**: Show existence by constructing random object with positive probability.

Probability Bounds for Algorithm Analysis: - **Boole's inequality** (union bound): $\Pr[\cup A_i] \leq \sum \Pr[A_i]$. - **Markov's inequality**: $\Pr[X \geq t] \leq E[X]/t$ for nonnegative r.v. X . - **Chernoff bound**: For $X = \sum \text{Bernoulli}(p_i)$ with $\mu = E[X]$: $\Pr[X \leq (1-\delta)\mu] \leq e^{-\mu\delta^2/2}$ for $\delta \in [0, 1]$; $\Pr[X \geq (1+\delta)\mu] \leq e^{-\mu\delta^2/3}$ for $\delta \in [0, 1]$. - **Conditional expectation**: $E[Y] = E[E[Y|X]]$. Law of total probability. - **Variance**: $\text{Var}[X] = E[X^2] - (E[X])^2$. $\text{Var}[\sum X_i] = \sum \text{Var}[X_i]$ for independent X_i .

Ch. 6 — Heapsort

Binary Heap: Array viewed as nearly complete binary tree. $A[1] = \text{root}$. $\text{PARENT}(i) = \lfloor i/2 \rfloor$, $\text{LEFT}(i) = 2i$, $\text{RIGHT}(i) = 2i+1$.

Max-heap property: $A[\text{PARENT}(i)] \geq A[i]$. Min-heap property: $A[\text{PARENT}(i)] \leq A[i]$.

Height: $\Theta(\lg n)$ for n -element heap. Leaves: indices $\lfloor n/2 \rfloor + 1 \dots n$.

MAX-HEAPIFY(A, i)

```
l = LEFT(i) = 2i
r = RIGHT(i) = 2i+1
if l ≤ A.heap-size and A[l] > A[i]    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]    largest = r
if largest ≠ i
    swap A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

- Complexity: $O(\lg n)$. Worst-case $2n/3$ occurs at root of a tree where bottom half-full.
- Recurrence $T(n) \leq T(2n/3) + O(1) \rightarrow O(\lg n)$ by Master Theorem Case 2.
- Loop invariant version: at start of each iteration, $A[i]$ may be smaller than children; after swap, property holds for subtree rooted at largest.

BUILD-MAX-HEAP(A, n)

```
A.heap-size = n
for i =  $\lfloor n/2 \rfloor$  down to 1
    MAX-HEAPIFY(A, i)
```

- Complexity: $O(n)$. Proof: $\sum_{h=0}^{\lfloor \lg n \rfloor} \lfloor n/2^{h+1} \rfloor \cdot O(h) = O(n \cdot \sum_{h=0}^{\lfloor \lg n \rfloor} h/2^h) = O(n)$.
- Key insight: MAX-HEAPIFY on node of height h takes $O(h)$, and there are $\leq \lfloor n/2^{h+1} \rfloor$ such nodes.

HEAPSORT(A, n)

```
BUILD-MAX-HEAP(A, n)
for i = n down to 2
    swap A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```

- Complexity: $O(n \lg n)$ — BUILD-MAX-HEAP $O(n)$ + $(n-1) \cdot$ MAX-HEAPIFY calls each $O(\lg n)$.
- Loop invariant: $A[1:i]$ is max-heap containing i smallest elements; $A[i+1:n]$ contains $n-i$ largest in sorted order.
- Not stable. In-place.

Priority Queues: - **HEAP-MAXIMUM(A):** return $A[1]$. $\Theta(1)$. - **HEAP-EXTRACT-MAX(A):** $\text{max} = A[1]$; $A[1] = A[A.\text{heap-size}]$; heap-size-- ; MAX-HEAPIFY(A, 1). $O(\lg n)$. - **HEAP-INCREASE-KEY(A, i, key):** $A[i] = \text{key}$; while $i > 1$ and $A[\text{PARENT}(i)] < A[i]$, swap up. $O(\lg n)$. - **MAX-HEAP-INSERT(A, key):** heap-size++ ; $A[\text{heap-size}] = -\infty$; HEAP-INCREASE-KEY(A, heap-size , key). $O(\lg n)$. - Min-priority queue: symmetric operations (HEAP-MINIMUM, HEAP-EXTRACT-MIN, HEAP-DECREASE-KEY, MIN-HEAP-INSERT).

d-ary heap: Each node has d children. Height $\Theta(\log_d n)$. $\text{PARENT}(i) = \lfloor (i-1)/d \rfloor$. $\text{CHILD}(i,j) = d(i-1)+j+1$. **EXTRACT-MAX:** $\Theta(d \log_d n)$. **INCREASE-KEY:** $\Theta(\log_d n)$. Good for dense graphs (Prim's algorithm).

Comparisons: | Dimension | Heapsort | Merge sort | Quicksort | |-----|
 -----|-----|-----| | Worst-case | $O(n \lg n)$ | $O(n \lg n)$ | $\Theta(n^2)$ | | In-place
 | Yes | No | Yes | | Stable | No | Yes | No | | Practical | Moderate | Slower |
 Fastest |

Ch. 7 — Quicksort

Divide-and-conquer: Partition around pivot, recursively sort both sides. Partition does all the work.

PARTITION (Lomuto)

```
x = A[r]    // pivot
i = p - 1
for j = p to r-1
    if A[j] ≤ x
        i = i + 1
        swap A[i] with A[j]
swap A[i+1] with A[r]
return i+1
```

- Loop invariant: $A[p:i] \leq x$, $A[i+1:j-1] > x$, $A[r] = x$.
- $\Theta(n)$ time, where $n = r-p+1$.
- **Pitfall:** Returns r when all elements are $\leq$ pivot $\rightarrow$ worst-case behavior.

QUICKSORT(A, p, r)

```
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q-1)
    QUICKSORT(A, q+1, r)
```

- Worst-case: Already sorted $\rightarrow \Theta(n^2)$. Recurrence $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$.
- Best-case: Perfectly balanced $\rightarrow T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$.
- **Any constant split** (e.g., 99:1): $T(n) = T(\alpha n) + T((1-\alpha)n) + \Theta(n) = O(n \lg n)$. Depth $\leq \log_{1/(1-\alpha)} n$.
- Average-case expected $\Theta(n \lg n)$ for random input (equal element probability = $1/n$ per position).

RANDOMIZED-PARTITION: Swap $A[r]$ with $A[\text{RANDOM}(p,r)]$ before partitioning.

```
int pivot = RANDOM(p, r)
swap A[r] with A[pivot]
return PARTITION(A, p, r)
```


RANDOMIZED-QUICKSORT: As QUICKSORT but using RANDOMIZED-PARTITION. Expected $\Theta(n \lg n)$.

Expected analysis: - Let $z_1 < z_2 < \dots < z_n$ be sorted elements. Define indicator $X_{ij} = 1$ if z_i, z_j compared. - $\Pr[z_i \text{ compares } z_j] = 2/(j-i+1)$ — only if first pivot chosen from $Z_{ij} = \{z_i, \dots, z_j\}$. - $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2/(j-i+1) = O(n \lg n)$. - Lemma 7.1: Partition takes $O(n)$ time $\rightarrow$ total expected time = $O(n \lg n)$.

HOARE-PARTITION

```
x = A[p]    // pivot
i = p - 1
j = r + 1
while true
    repeat j = j - 1 until A[j] ≤ x
    repeat i = i + 1 until A[i] ≥ x
    if i < j
        swap A[i] with A[j]
    else
        return j
```

- Returns j where $p \leq j < r$. Both $A[p:j] \leq x$ and $A[j+1:r] \geq x$.
- More efficient for equal elements (returns near middle). Fewer swaps than Lomuto.

Tail-recursion elimination: Replace second recursive call with iteration. Recursively sort smaller partition, iteratively process larger. Stack depth $\leq \lg n$.

```
QUICKSORT'(A, p, r)
    while p < r
        q = PARTITION(A, p, r)
        if q-p < r-q    // left smaller
            QUICKSORT'(A, p, q-1)
            p = q+1
        else
            QUICKSORT'(A, q+1, r)
            r = q-1
```

Edge Cases: All equal elements $\rightarrow$ Lomuto returns $q = r \rightarrow \Theta(n^2)$. Hoare partition handles better (returns near middle). RANDOMIZED-PARTITION reduces but doesn't eliminate worst case.

Ch. 8 — Sorting in Linear Time

Comparison Sort Lower Bound (Theorem 8.1): Any comparison sort requires $\Omega(n \lg n)$ comparisons in worst case. - **Proof:** Decision tree model. Each leaf = possible permutation. $n!$ permutations $\rightarrow$ at least $n!$ leaves. Height $h \rightarrow \leq 2^h$ leaves. So $n! \leq 2^h \rightarrow h \geq \lg(n!) = \Omega(n \lg n)$ by Stirling. -

Corollary 8.2: Heapsort and merge sort are asymptotically optimal comparison sorts (achieve $\Theta(n \lg n)$ worst-case).

COUNTING-SORT(A, n, k)

```

let C[0:k] be new array of zeros
for i = 1 to n          // count frequencies
    C[A[i]] = C[A[i]] + 1
for i = 1 to k          // running sums → positions
    C[i] = C[i] + C[i-1]
let B[1:n] be new array
for i = n down to 1    // stable placement
    B[C[A[i]]] = A[i]
    C[A[i]] = C[A[i]] - 1
return B

```

- Input: Integers in $[0, k]$. Output: $B[1:n]$ sorted.
- Complexity: $\Theta(n+k)$. When $k = O(n)$, runs in $\Theta(n)$.
- Stable because we process input in reverse order.
- Not a comparison sort — uses array indices, wins when $k = O(n)$.

RADIX-SORT(A, n, d)

```

for i = 1 to d          // least significant digit first
    use stable sort (e.g., counting sort) to sort A on digit i

```

- Input: d-digit numbers, each digit range $[0, k-1]$.
- Complexity: $\Theta(d(n+k))$. If $k = O(n)$, then $\Theta(dn)$.
- **Optimal digit size:** r bits per digit. $d = \lceil b/r \rceil$ digits total. Radix: $\Theta((b/r)(n+2^r))$. Minimize: set $r = \lceil \lg n \rceil \rightarrow \Theta(bn/\lg n)$.
- With n integers in $[0, 2^b-1]$ and $r = \lceil \lg n \rceil$, total $\Theta(bn/\lg n)$. For $b = O(\lg n)$, $\Theta(n)$.
- **LSD-first vs MSD-first:** LSD requires stable sort; MSD can be used for lexicographic order.

BUCKET-SORT(A, n)

```

let B[0:n-1] be new array of empty lists
for i = 1 to n
    insert A[i] into B[ $\lfloor n \cdot A[i] \rfloor$ ]
for i = 0 to n-1
    sort list B[i] with insertion sort
concatenate B[0], B[1], ..., B[n-1]

```

- Assumes input uniform over $[0, 1)$.
- Expected $\Theta(n)$, worst-case $\Theta(n^2)$ (all elements in one bucket).
- $E[\text{sum of squared bucket sizes}] = 2 - 1/n \rightarrow O(n)$ total sort time.

Algorithm	Assumption	Time	Stable
Counting	Integers $[0, k]$	$\Theta(n+k)$	Yes
Radix	d-digit numbers	$\Theta(d(n+k))$	Depends on inner sort
Bucket	Uniform $[0, 1)$	$\Theta(n)$ expected	Depends on inner sort

0-1 Sorting Lemma: If oblivious compare-exchange algorithm sorts all 0-1 inputs, it sorts all inputs. Proof by contradiction: a misordered pair of distinct values maps to a 0-1 violation.

Stability: Insertion sort ✓, Merge sort ✓, Heapsort ✗, Quicksort (Lomuto) ✗, Counting sort ✓, Radix sort (with stable digit sort) ✓.

Ch. 9 — Medians and Order Statistics

Selection problem: Find i -th smallest element from $A[1:n]$. Can be solved in $O(n)$ expected and $O(n)$ worst-case time.

MINIMUM(A, n)

```
min = A[1]
for i = 2 to n
    if A[i] < min    min = A[i]
return min
```

- $n-1$ comparisons (optimal by tournament argument: each non-winner must lose ≥ 1 match).

Simultaneous min and max: Process in pairs: compare pair elements, compare larger to max, smaller to min. - At most $3\lfloor n/2 \rfloor$ comparisons (vs $2n-2$ naive).

RANDOMIZED-SELECT(A, p, r, i)

```
if p == r
    return A[p]
q = RANDOMIZED-PARTITION(A, p, r)
k = q - p + 1      // rank of pivot
if i == k
    return A[q]    // pivot is the answer
else if i < k
    return RANDOMIZED-SELECT(A, p, q-1, i)
else
    return RANDOMIZED-SELECT(A, q+1, r, i-k)
```

- Expected $\Theta(n)$, worst-case $\Theta(n^2)$.
- Intuition: pivot in “middle half” (positions 25%-75%) removes $\geq n/4$ elements with probability $\geq 1/2$. Expected size reduction $\rightarrow T(n) = T(3n/4) + O(n) \rightarrow \Theta(n)$.

SELECT (worst-case linear, Blum-Floyd-Pratt-Rivest-Tarjan)

1. Divide n elements into $\lfloor n/5 \rfloor$ groups of 5
2. Find median of each group (sort each 5-element group, take 3rd)
3. Recursively find median x of the $\lfloor n/5 \rfloor$ medians
4. Partition around x (modified PARTITION)
5. if $i == k$ return x

else if $i < k$ recurse on left side
 else recurse on right side with $i-k$

- **Why groups of 5?** At least $\lceil \frac{n}{5} \rceil / 2$ groups have median $\leq x$ (and $\geq x$). Each such group contributes 3 elements $\leq x$ (or $\geq x$). So at least $3 \cdot \lceil \frac{n}{5} \rceil / 2 \geq 3n/10$ elements are $\leq x$ (and $\geq x$). Worst-case recursion: $n - 3n/10 = 7n/10$.
- **Recurrence:** $T(n) \leq T(\lceil n/5 \rceil) + T(7n/10) + \Theta(n) \rightarrow T(n) = \Theta(n)$ (substitution proof with $T(n) \leq cn$).
- **Groups of 3:** $T(n) \leq T(n/3) + T(2n/3) + \Theta(n) \rightarrow O(n \lg n)$ — NOT linear.
- **Groups of 7:** Works but constant factor worse than 5.
- **Practical note:** RANDOMIZED-SELECT is faster in practice; SELECT is mostly theoretical interest.

Comparisons:	Dimension	RANDOMIZED-SELECT	SELECT	—————
—————	—————	Expected $\Theta(n)$	$\Theta(n)$	Worst-case $\Theta(n^2)$
Practical	Yes, fast	Mostly theoretical	Constant factor	Small (1-2x PARTITION)
		Large (grouping, sorting 5s)		

Tournament argument: Each non-winner must lose at least once $\rightarrow n-1$ comparisons for minimum (optimal). Lower bound proof by adversary.

Ch. 10 — Elementary Data Structures

Stacks: LIFO. Array $S[1:n]$ with top pointer.

```
STACK-EMPTY(S): return S.top == 0
PUSH(S, x): S.top++; S[S.top] = x
POP(S): if STACK-EMPTY then error; else S.top--; return S[S.top+1]
```

- Overflow: PUSH when $S.top == n$. Underflow: POP when empty.
- All operations $O(1)$. Used for DFS, function call stack, expression evaluation.

Queues: FIFO. Circular array $Q[1:n]$ with head $Q.head$ and tail $Q.tail$.

```
ENQUEUE(Q, x): Q[Q.tail] = x; if Q.tail == n then Q.tail = 1 else Q.tail++
DEQUEUE(Q): x = Q[Q.head]; if Q.head == n then Q.head = 1 else Q.head++; return x
```

- Full: $Q.tail == Q.head$ after wrap. Empty: $Q.head == Q.tail$ after DEQUEUE.
- Sentinel: maintain size or sacrifice one slot to distinguish full/empty.
- All operations $O(1)$. Used for BFS, buffering.

Linked Lists: Doubly or singly linked. LIST-SEARCH $\Theta(n)$, LIST-PREPEND $O(1)$, LIST-INSERT $O(1)$, LIST-DELETE $O(1)$ with pointer.

Sentinel (L.nil): Dummy node eliminating boundary checks. Simplifies code; asymptotic same, slightly better constants.

Array address calculation: 1-origin: $a + b(i-1)$. Row-major: $M[i,j]$ at $n(i-1)+j$. Column-major: $i + m(j-1)$.

Left-child, right-sibling representation: For arbitrary rooted trees. Each node stores: parent, left-child, right-sibling. Space $O(n)$. - **Tree traversals:** Preorder (root→children), Postorder (children→root), Level-order (queue-based BFS).

Rooted tree representations: - **Parent array:** $p[i]$ = parent of i . Space $O(n)$. Easy for finding parent, hard for children. - **Adjacency list:** For general trees and graphs. Common in graph algorithms. - **Left-child right-sibling:** Binary representation of arbitrary tree. Used for space efficiency.

Comparisons: | Dimension | Array | Linked List | Stack | Queue | Deque | |
 -----|-----|-----|-----|-----|-----| | Access | $O(1)$ | $O(k)$ | $O(1)$ top | $O(1)$
 front | $O(1)$ ends | | Insert front | $\Theta(n)$ | $O(1)$ | — | — | $O(1)$ | | Insert back |
 $O(1)^*$ | $O(1)^\dagger$ | $O(1)$ | $O(1)$ | $O(1)$ | | Delete front | $\Theta(n)$ | $O(1)$ | — | $O(1)$ | $O(1)$
 | | Delete back | $O(1)^*$ | $O(n)^\dagger$ | $O(1)$ | — | $O(1)$ |

*With dynamic table (amortized). †With tail pointer. Singly linked: back operations $O(n)$.

Multi-array representation: Stack of arrays representation for dynamic arrays (implemented via doubling/realloc). Amortized $O(1)$ per insertion.

Ch. 11 — Hash Tables

Direct-address table: $T[k]$ points to element with key k . $O(1)$ operations. Requires universe U to be small. INSERT(DIRECT-ADDRESS-INSERT), SEARCH, DELETE all $\Theta(1)$.

Hash table: $h(k)$ maps key to slot. Collisions resolved by chaining or open addressing. - **TABULATE-HASH:** Create random table of the form $\{0,1\}^\ell$. Compute $h(k) = \sum_{i=1}^c T_i[k_i] \bmod 2^\ell$.

CHAINED-HASH-INSERT(T, x): insert at head of $T[h(x.key)]$, $O(1)$.

CHAINED-HASH-SEARCH(T, k): search list $T[h(k)]$ linearly, $\Theta(1+\alpha)$

expected. **CHAINED-HASH-DELETE(T, x):** doubly linked list delete, $O(1)$.

Load factor: $\alpha = n/m$. Chaining avg search: $\Theta(1+\alpha)$.

Chaining: Each slot points to linked list. INSERT $O(1)$, SEARCH $\Theta(1+\alpha)$ avg, DELETE $O(1)$ with doubly linked.

Hash function methods: - **Division method:** $h(k) = k \bmod m$. Avoid $m = 2^p$ (poor distribution based on lower bits). Good choice: $m \approx$ prime not near 2^p . - **Multiplication method:** $h(k) = \lfloor m(kA \bmod 1) \rfloor$. $A \approx (\sqrt{5}-1)/2 \approx 0.618$ (Knuth's suggestion). Works for any m . - **Multiply-shift:** $h_a(k) = (k \cdot a \bmod 2^w) \ggg (w-\ell)$, where a odd, w = word size, $\ell = \lfloor \lg m \rfloor$. Fast in hardware.

Universal hashing: Family H such that $\Pr[h(k_1)=h(k_2)] \leq 1/m$ for distinct k_1, k_2 . - **H_{pm} family:** $H = \{h_{ab}(k) = ((ak+b) \bmod p) \bmod m\}$, p prime $> |U|$, $a \in [1, p-1]$, $b \in [0, p-1]$. Universal (Theorem 11.4). Proof: for $k_1 \neq k_2$, the pair (ak_1+b, ak_2+b) uniformly maps to distinct pairs mod p . - **Multiply-shift:** $2/m$ -universal (Theorem 11.5). $\Pr[h_a(k_1)=h_a(k_2)] \leq 2/m$ for distinct k_1, k_2 . - **Tabulation hashing:** View key as c characters, each character indexes a random table $T_i[\cdot]$, XOR results. 3-wise independent. Very fast in practice.

Perfect hashing (Section 11.5): When keys are static (no inserts/deletes). - Two-level scheme: outer hash table maps to inner perfect hash table. - First level: universal hash with $m = n$, expected collisions $O(n)$. - Second level: for each bucket of size n_j , use $m_j = n_j^2$ to guarantee no collisions (expected total space $O(n)$). - **Theorem 11.9:** Expected total space $O(n)$ for static dictionary with $O(1)$ worst-case search.

Choosing hash functions in practice: - **Simple random:** Not repeatable. Use seeded hash (siphash, cityhash, xxhash). - **Cryptographic:** Overkill for hash tables (SHA, MD5). - **Key types:** Integer (multiply-shift), String (polynomial rolling hash), Compound (combine fields).

Open addressing: All elements in table. Probe sequences. - **Linear probing:** $h(k,i) = (h_1(k)+i) \bmod m$. Primary clustering. $O(1)$ with 5-independent hash and $\alpha \leq 2/3$. - **Double hashing:** $h(k,i) = (h_1(k)+i \cdot h_2(k)) \bmod m$. $\Theta(m^2)$ probe sequences. - Expected probes: unsuccessful $\leq 1/(1-\alpha)$, successful $\leq (1/\alpha) \ln(1/(1-\alpha))$.

Deletion in open addressing: Hard — need DELETED marker or special procedure.

Dimension Chaining Open Addressing

α can be > 1	Yes	No ($\alpha < 1$)
Deletion	Easy	Hard
Cache	Poor	Better

Ch. 12 — Binary Search Trees

BST property: For any node x , left subtree keys $\leq x.key \leq$ right subtree keys.

Inorder tree walk: LEFT(x), print $x.key$, RIGHT(x) $\rightarrow$ prints keys sorted in $\Theta(n)$.

Query operations (all $O(h)$ where $h = \text{height}$):

```

TREE-SEARCH( $x, k$ ): if  $x == \text{NIL}$  or  $k == x.key$  return  $x$ ; if  $k < x.key$ 
return TREE-SEARCH( $x.left, k$ ) else TREE-SEARCH( $x.right, k$ )
ITERATIVE-TREE-SEARCH( $x, k$ ): while  $x \neq \text{NIL}$  and  $k \neq x.key$ : if  $k < x.key$ 
then  $x = x.left$  else  $x = x.right$ ; return  $x$ 
TREE-MINIMUM( $x$ ): while  $x.left \neq \text{NIL}$ :  $x = x.left$ ; return  $x$ 
TREE-MAXIMUM( $x$ ): while  $x.right \neq \text{NIL}$ :  $x = x.right$ ; return  $x$ 
TREE-SUCCESSOR( $x$ ): if  $x.right \neq \text{NIL}$  return TREE-MINIMUM( $x.right$ );

```

$y = x.parent$; while $y \neq \text{NIL}$ and $x == y.right$: $x = y$; $y = y.parent$; return y
TREE-PREDECESSOR(x): symmetric to successor

TREE-INSERT(T, z)

```

y = NIL
x = T.root
while x  $\neq$  NIL
    y = x
    if z.key < x.key    x = x.left
    else                x = x.right
z.parent = y
if y == NIL
    T.root = z
else if z.key < y.key  y.left = z
else                  y.right = z

```

- $O(h)$. Inserted node always becomes a leaf (before fixup for RB trees).

TREE-DELETE(T, z) - **Transplant**(u, v): Replace subtree rooted at u with subtree rooted at v . - **Cases**: 1. z has no left child $\rightarrow$ **TRANSPLANT**($z, z.right$)
2. z has no right child $\rightarrow$ **TRANSPLANT**($z, z.left$)
3. z has both children, successor y is right child $\rightarrow$ **TRANSPLANT**(z, y); $y.left = z.left$
4. z has both children, successor y NOT right child $\rightarrow$ **TRANSPLANT**($y, y.right$); $y.right = z.right$; **TRANSPLANT**(z, y); $y.left = z.left$
- $O(h)$. Cases 3-4: successor is always the node with smallest key $> z.key$.

Randomly built BST: Expected height $O(\lg n)$. Expected depth of node: $O(\lg n)$. - **Theorem 12.4**: Expected height of randomly built BST = $O(\lg n)$. Proof via indicator variables and Jensen's inequality.

Catalan number: Number of distinct BSTs with n nodes = $(1/(n+1)) \cdot C(2n, n)$.

Comparisons: Balanced BST ($O(\lg n)$ per op) vs sorted array ($O(\lg n)$ search, $\Theta(n)$ insert/delete) vs linked list ($\Theta(n)$ search, $O(1)$ insert/delete).

Ch. 13 — Red-Black Trees

Red-black properties: 1. Every node red or black 2. Root is black 3. Every leaf ($T.nil$) is black 4. Red node $\rightarrow$ both children black (no consecutive reds) 5. For each node, all paths to descendant leaves have same number of black nodes

Black-height $bh(x)$: Number of black nodes on path from x (excluding x) to leaf.

Lemma 13.1 (Height bound): RB tree with n internal nodes has height $h \leq 2 \lg(n+1)$. - **Proof**: Inductive claim: subtree at x has $\geq 2^{bh(x)} - 1$ internal nodes. Base: $bh(x)=0 \rightarrow 0$ nodes. Inductive: if children have $bh(x)$ or $bh(x)-1$, node count = $2 \cdot (2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$. By property 4

(no consecutive reds), $bh(\text{root}) \geq h/2$. So $n \geq 2^{\lceil h/2 \rceil} - 1 \rightarrow h \leq 2 \lg(n+1)$. - **Corollary:** SEARCH, MIN, MAX, SUCCESSOR, PREDECESSOR all $O(\lg n)$.

Rotations: LEFT-ROTATE(T, x), RIGHT-ROTATE(T, y). $O(1)$. Preserve BST property and inorder ordering.

```
LEFT-ROTATE( $T, x$ )
     $y = x.\text{right}$                 // set  $y$ 
     $x.\text{right} = y.\text{left}$             // turn  $y$ 's left subtree into
     $x$ 's right subtree
    if  $y.\text{left} \neq T.\text{nil}$      $y.\text{left}.\text{parent} = x$ 
     $y.\text{parent} = x.\text{parent}$         // link  $x$ 's parent to  $y$ 
    if  $x.\text{parent} == T.\text{nil}$      $T.\text{root} = y$ 
    else if  $x == x.\text{parent}.\text{left}$      $x.\text{parent}.\text{left} = y$ 
    else     $x.\text{parent}.\text{right} = y$ 
     $y.\text{left} = x$                 // put  $x$  on  $y$ 's left
     $x.\text{parent} = y$ 
```

RB-INSERT(T, z): BST insert + color RED + RB-INSERT-FIXUP(T, z). $O(\lg n)$. At most 2 rotations. - **RB-INSERT-FIXUP cases** (assuming $z.\text{parent}$ is left child — symmetric for right): 1. **$z.\text{uncle}$ RED:** recolor parent, uncle, grandparent; move z up two levels 2. **$z.\text{uncle}$ BLACK, z is right child:** left-rotate parent $\rightarrow$ becomes case 3 3. **$z.\text{uncle}$ BLACK, z is left child:** right-rotate grandparent, recolor parent BLACK, grandparent RED - Loop invariant: z is RED; only violation is property 2 (root black) or property 4 (red-black-red). - Termination: if z is root $\rightarrow$ color BLACK (fix property 2); if uncle is RED $\rightarrow$ recolor; otherwise rotation + termination.

RB-DELETE(T, z): More complex. Tracks y (node spliced out or moved) and x (node replacing y). - If y was BLACK $\rightarrow$ RB-DELETE-FIXUP(T, x). $O(\lg n)$. At most 3 rotations. - **RB-DELETE-FIXUP cases** (assuming x is left child): 1. **w (sibling) RED:** recolor w BLACK, $x.\text{parent}$ RED, LEFT-ROTATE($x.\text{parent}$), new $w = x.\text{parent}.\text{right}$ 2. **w BLACK, both w 's children BLACK:** set w RED, move x up to $x.\text{parent}$ 3. **w BLACK, $w.\text{left}$ RED, $w.\text{right}$ BLACK:** recolor $w.\text{left}$ BLACK, w RED, RIGHT-ROTATE(w), new $w = x.\text{parent}.\text{right}$ 4. **w BLACK, $w.\text{right}$ RED:** recolor $w = x.\text{parent}.\text{color}$, $x.\text{parent}$ BLACK, $w.\text{right}$ BLACK, LEFT-ROTATE($x.\text{parent}$), set $x = \text{root}$ - Loop invariant: x is "doubly black" or RED-AND-BLACK; fixup pushes blackness up or removes it. - **Difference from insert:** Delete fixup can propagate up to root; insert fixup terminates after $O(1)$ rotations.

Comparisons: RB tree ($h \leq 2 \lg(n+1)$, ≤ 2 -3 rotations) vs AVL ($h \leq 1.44 \lg n$, $O(\lg n)$ rotations) vs BST ($\Theta(n)$ worst-case height).

Persistent trees: Without parent pointers, insertion copies $O(h)$ nodes.

Ch. 14 — Dynamic Programming

Key properties: Optimal substructure + overlapping subproblems.

Four-step method: (1) Characterize optimal structure; (2) Recursively define optimal value; (3) Compute value (bottom-up or top-down); (4) Construct optimal solution.

Rod Cutting: $r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$, $r_0 = 0$. Naive recursive: 2^n . DP (top-down memoized or bottom-up): $\Theta(n^2)$. - **Bottom-up:** for $j=1..n$: $r_j = \max(p_i + r_{j-i})$ for $1 \leq i \leq j$. $O(n^2)$. - **Reconstruction:** $s[j]$ = optimal first cut length for rod of length j .

Matrix-Chain Multiplication: $m[i,j] = \min_{i \leq k < j} \{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j\}$. $\Theta(n^3)$ time, $\Theta(n^2)$ space. - Number of parenthesizations = Catalan $C_{n-1} = \Omega(4^{n/n} \{3/2\})$ — factorial growth vs polynomial DP. - **Bottom-up:** fill $m[i,j]$ by chain length $\ell=2..n$; for each $[i,j]$ try all k in $[i,j-1]$. $O(n^3)$. - **Example:** $A_1(30 \times 35)$, $A_2(35 \times 15)$, $A_3(15 \times 5)$, $A_4(5 \times 10)$, $A_5(10 \times 20)$, $A_6(20 \times 25)$. Optimal: $(A_1(A_2A_3))((A_4A_5)A_6) = 15,125$ scalar multiplications (vs 71,250 naive).

LCS (Longest Common Subsequence): $c[i,j] = 0$ if $i=0$ or $j=0$; $= c[i-1,j-1]+1$ if $x_i=y_j$; $= \max(c[i-1,j], c[i,j-1])$ if $x_i \neq y_j$. $\Theta(mn)$ time, $\Theta(mn)$ space (can be $O(\min(m,n))$ space). - **Reconstruction:** follow $b[i,j]$ arrows ($\nwarrow$, $\uparrow$, $\leftarrow$) from $c[m,n]$ to $c[0,0]$. $O(m+n)$. - **Space-efficient:** keep only two rows; reconstruct via Hirschberg algorithm ($O(mn)$ time, $O(\min(m,n))$ space).

OBST (Optimal Binary Search Tree): $e[i,j] = \min_{i \leq r \leq j} \{e[i,r-1] + e[r+1,j] + w[i,j]\}$, where $w[i,j] = \sum_{k=i}^j p_k + \sum_{k=i-1}^j q_k$ (success + failure probabilities). - $\Theta(n^3)$ naive, $\Theta(n^2)$ with Knuth optimization (monotonicity of optimal root choice: $\text{root}[i,j-1] \leq \text{root}[i,j] \leq \text{root}[i+1,j]$). - **Key insight:** Optimal BST minimizes expected search cost. Similar to matrix-chain but with probability weights.

Longest Increasing Subsequence (LIS): - DP: $dp[i] = 1 + \max\{dp[j] : j < i \text{ and } A[j] < A[i]\}$. $O(n^2)$. - Patience sorting: maintain piles (smallest top card of each pile). $O(n \lg n)$. - **Reconstruction:** track predecessor pointers.

Edit Distance (Levenshtein distance): $dp[i,j] = \min(dp[i-1,j]+1 \text{ (delete)}, dp[i,j-1]+1 \text{ (insert)}, dp[i-1,j-1]+\text{cost} \text{ (substitute)})$ where $\text{cost} = 0$ if $x_i=y_j$, 1 otherwise. - $O(mn)$. Applications: spell checking, DNA sequence alignment (Needleman-Wunsch).

0-1 Knapsack: $dp[i,w] = \max(dp[i-1,w], dp[i-1,w-w_i] + v_i)$ for $w_i \leq w$, else $dp[i-1,w]$. - $O(nW)$ — pseudopolynomial (W is numeric value, not input length). - **FPTAS:** round profits to reduce $W \rightarrow O(n^3/\epsilon)$.

Knuth Optimization Condition: DP of form $dp[i,j] = \min_{i \leq k < j} (dp[i,k] + dp[k+1,j]) + C[i,j]$. If C satisfies quadrangle inequality ($C[i,j] + C[i+1,j+1] \leq C[i,j+1] + C[i+1,j]$) and monotonicity ($C[i,j] \leq C[i+1,j+1]$), then optimal k is monotonic $\rightarrow O(n^2)$ instead of $O(n^3)$.

DP Problem Table: | Problem | Table Size | Time | State Definition | |
 -----|-----|-----|-----|-----|
 | Rod cutting | $O(n)$ | $O(n^2)$ | $r_j = \max$ revenue for length j |
 | Matrix-chain | $O(n^2)$ | $O(n^3)$ | $m[i,j] = \min$ ops for $A_i \dots A_j$ |
 | LCS | $O(mn)$ | $O(mn)$ | $c[i,j] = \text{LCS length for } X_i, Y_j$ |
 | OBST | $O(n^2)$ | $O(n^2)^\dagger$ | $e[i,j] = \text{expected cost for keys } i..j$ |
 | LIS | $O(n)$ | $O(n \lg n)^\dagger^\dagger$ | $dp[i] = \text{length of LIS}$ |

ending at i | | Edit distance | $O(mn)$ | $O(mn)$ | $dp[i,j]$ = edit distance for X_i, Y_j | | 0-1 Knapsack | $O(nW)$ | $O(nW)$ | $dp[i,w]$ = max value with first i items, capacity w | | Floyd-Warshall | $O(V^2)$ | $O(V^3)$ | $d^{\{k\}}[i,j]$ = shortest path via $\{1..k\}$ |

† With Knuth optimization. †† With patience sorting (binary search on piles).

Cut-and-Paste Argument: If subproblem solution not optimal, cut it out and paste in better one $\rightarrow$ contradiction with original optimality. - Example: In rod cutting, if $r_{\{n-i\}}$ is not optimal for subproblem of size $n-i$, then r_n wouldn't be optimal either.

tD/eD classification: Table size $O(n^t)$, each entry depends on $O(n^e)$ others. - 1D/1D: LIS, rod cutting. 2D/1D: LCS. 3D/2D: matrix-chain (n^2 entries, n choices each).

Reconstruction: Store choices to recover solution (parent pointers, split points, etc.): - Rod cutting: $s[j]$ = optimal cut. Matrix-chain: $s[i,j]$ = optimal split k . LCS: $b[i,j]$ = direction ($\uparrow, \leftarrow, \searrow$).

Common DP Patterns: | Pattern | Recurrence | Example | |-----|-----|
 -----| | Linear 1D | $dp[i] = f(dp[j])$ for $j < i$ | LIS, rod cutting | | Interval |
 $dp[i,j] = f(dp[i,k], dp[k+1,j])$ | Matrix-chain, OBST | | Edit distance | $dp[i,j] =$
 $f(dp[i-1,j], dp[i,j-1], dp[i-1,j-1])$ | LCS, string edit | | Knapsack | $dp[i,w] =$
 $\max(dp[i-1,w], dp[i-1,w-w_i] + v_i)$ | 0-1 Knapsack | | Tree DP | $dp[u] =$
 $f(dp[\text{children}[u]])$ | Largest independent set | | Knuth optimization | $dp[i,j] =$
 $\min_{k \in [p_i, p_{i+1}]}$... | OBST $\rightarrow O(n^2)$ |

Comparisons: DP vs Greedy (DP explores all subproblems, greedy commits locally; DP for 0-1 knapsack, greedy for fractional), DP vs D&C (DP subproblems overlap, D&C subproblems disjoint).

DP on Trees: For rooted tree, compute from leaves up. Example: Largest independent set in tree. $dp[u,0] = \sum \max(dp[v,0], dp[v,1])$ (u excluded); $dp[u,1] = 1 + \sum dp[v,0]$ (u included). $O(n)$.

Ch. 15 — Greedy Algorithms

Greedy-choice property: Locally optimal choice leads to global optimum.

Optimal substructure: Solution contains optimal solutions to subproblems.

Activity Selection: Choose activity with earliest finish time. Greedy: $\Theta(n)$ (after sorting by finish time). Exchange argument proves optimality (Theorem 15.1).

Huffman Codes: Merge two lowest-frequency characters. $O(n \lg n)$ with min-heap. Produces optimal prefix-free code (Theorem 15.4). - Lemma 15.2: lowest-frequency chars can be deepest siblings in some optimal code. - Lemma 15.3: optimal substructure via merging. - Full binary tree required for optimality. - Compression example: 224K bits vs 300K fixed-length (25% savings).

Offline Caching (Furthest-in-Future): Evict block whose next access is furthest. Optimal (Theorem 15.5). Exchange argument.

Fractional Knapsack: Sort by v_i/w_i (value/weight). Take items in order until knapsack full. $\Theta(n \lg n)$. Greedy works because you can take fractions. -

0-1 Knapsack: Requires DP. Recurrence: $dp[i,w] = \max(dp[i-1,w], dp[i-1,w-w_i] + v_i)$. $O(nW)$ — pseudopolynomial. No greedy (counterexample: $W=50$, item1: $v=60, w=10 \rightarrow$ ratio 6, item2: $v=100, w=20 \rightarrow$ ratio 5, item3: $v=120, w=30 \rightarrow$ ratio 4; greedy picks item1+item2=160, but item2+item3=220 better).

Making Change (canonical coin systems): Greedy (largest coin first) optimal for US coins (1,5,10,25) but not all systems (e.g., 1,3,4: greedy for 6 gives $4+1+1=3$ coins, optimal is $3+3=2$ coins).

Scheduling (Minimizing lateness): Schedule jobs by deadline (earliest deadline first). $O(n \lg n)$. Exchange argument: any optimal schedule can be transformed to EDF without increasing max lateness.

Coin changing DP: $C[i] = \min_{\{c_j \leq i\}} C[i-c_j] + 1$. $\Theta(n \cdot k)$ for n amount and k coin types.

Proof patterns: - **Exchange argument** ("greedy stays ahead"): Take any optimal solution, transform step by step to include greedy choice without degrading quality. Example: activity selection — order activities by finish time, show that if OPT starts with different first activity, we can swap it for greedy's first choice. - **Matroid theory:** Many greedy algorithms (including MST, scheduling) are special cases of matroid optimization. A matroid $M = (S, I)$ satisfies: (1) $\emptyset \in I$; (2) $A \subseteq B \in I \rightarrow A \in I$; (3) $A, B \in I, |A| < |B| \rightarrow \exists x \in B$ with $A \cup \{x\} \in I$. Greedy on matroid weighted matroid finds maximum-weight independent set.

Edge Cases: Greedy fails when greedy-choice property absent (0-1 knapsack, making change, longest simple path).

Greedy Algorithm Design Checklist: 1. Does the problem have optimal substructure? (Optimal solution contains optimal solutions to subproblems) 2. Does the problem have the greedy-choice property? (A globally optimal solution can be reached by making locally optimal choices) 3. Can you prove that making the greedy choice reduces to a smaller instance of the same problem? 4. If both properties hold, greedy works (activity selection, Huffman, MST, Dijkstra, fractional knapsack) 5. If only optimal substructure holds, use DP (0-1 knapsack, LCS, matrix-chain)

Ch. 16 — Amortized Analysis

Three methods: 1. **Aggregate:** Total cost $T(n)$ for n ops $\rightarrow$ amortized $= T(n)/n$. 2. **Accounting:** Overcharge cheap ops, store credit; credit pays for expensive ops later. Credit must stay nonnegative. 3. **Potential:** $\Phi(D_i)$ maps data structure to real. Amortized $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$. Require $\Phi(D_i) \geq \Phi(D_0)$ for all i .

Stack with MULTIPOP: - **Aggregate:** Each element pushed once, popped at most once. $n \text{ PUSH} + n \text{ POP (including MULTIPOP)} \leq 2n \text{ ops}$. Amortized: $O(1)$. - **Accounting:** PUSH costs \$2 (\$1 for push, \$1 stored). POP/MULTIPOP use stored credit. Credit never negative.

Binary counter: - **Aggregate:** Bit $A[i]$ flips $\lceil n/2^i \rceil$ times. Total flips $= \sum \lceil n/2^i \rceil < 2n$. Amortized $O(1)$ per INCREMENT. - **Potential:** Φ = number of 1-bits. $\Phi_0 = 0$. Flip from $0 \rightarrow 1$ increases Φ by 1; $1 \rightarrow 0$ decreases by 1. $\hat{c}_i = c_i + \Delta\Phi = (t_i + 1) + (1 - t_i) = 2$, where t_i = trailing 1s. - **DECREMENT breaks $O(1)$:** alternating $0 \rightarrow 1$ and $1 \rightarrow 0$ can make each op cost $\Theta(\lg n)$.

Dynamic table (doubling only): - **Aggregate:** TABLE-INSERT cost = i when $i-1$ is power of 2, 1 otherwise. Total cost $< 2n$ (sum of 1s + powers of 2 up to n). Amortized < 3 per insert = $O(1)$. - **Accounting:** Charge \$3 per insertion (\$1 for immediate insert, \$1 for future resize of this element, \$1 for future resize of an older element). Credit suffices. - **Potential:** $\Phi(T) = 2 \cdot \text{num} - \text{size}$. $\Phi \geq 0$, $\Phi_0 = 0$. When $\alpha = 1$ (full), $\Phi = \text{size}$. After doubling: $\text{num}' = \text{size}$, $\text{size}' = 2 \cdot \text{size} \rightarrow \Phi' = 2 \cdot \text{size} - 2 \cdot \text{size} = 0$. $\Delta\Phi = -\text{size}$. Insert causing resize: $c = \text{num} + 1 = \text{size} + 1$ (1 for insert + size to copy). $\hat{c} = (\text{size}+1) + (-\text{size}) = 1 = O(1)$.

Dynamic table with insert + delete: - **Key issue:** If we halve at $\alpha = 1/2$, alternating insert/delete can cause repeated resize (thrashing). Solution: halve at $\alpha = 1/4$. - **Potential:** $\Phi = 2 \cdot \text{num} - \text{size}$ if $\alpha \geq 1/2$; $= \text{size}/2 - \text{num}$ if $1/4 \leq \alpha < 1/2$. $\Phi \geq 0$ always, $\Phi_0 = 0$. Each operation amortized $O(1)$.

Comparisons: Aggregate (same amortized cost for all ops), Accounting (different costs per op type), Potential (most flexible, global view).

Pitfalls: Credit must never go negative; halving at $\alpha = 1/2$ causes $\Theta(n)$ amortized for alternating insert/delete; DECREMENT on binary counter breaks $O(1)$ amortized.

Ch. 17 — Augmenting Data Structures

Generic 4-step method: (1) Choose underlying DS; (2) Determine additional info; (3) Verify maintainability; (4) Develop new operations.

Theorem 17.1: If $x.f$ depends only on $O(1)$ -computable info from x , $x.\text{left}$, $x.\text{right}$, then RB tree can maintain f in $O(\lg n)$. Constant rotations (≤ 2 insert, ≤ 3 delete) critical.

Order-Statistic Tree: Red-black tree + size attribute ($\text{size} = x.\text{left}.\text{size} + x.\text{right}.\text{size} + 1$).

```
OS-SELECT(x, i)           // i-th smallest in subtree rooted at x
    r = x.left.size + 1    // rank of x
    if i == r    return x
    else if i < r    return OS-SELECT(x.left, i)
    else    return OS-SELECT(x.right, i-r)
```

```
OS-RANK(T, x)             // rank of x in inorder traversal
```

```

r = x.left.size + 1
y = x
while y ≠ T.root
    if y == y.parent.right
        r = r + y.parent.left.size + 1
    y = y.parent
return r

```

- Both $O(\lg n)$. Size maintained during insert/delete by path traversal + fixing rotation ($O(1)$ per rotation).

Interval Tree: Red-black tree keyed by low endpoint + max attribute (max endpoint in subtree).

```

INTERVAL-SEARCH(T, i)
    x = T.root
    while x ≠ T.nil and i does not overlap x.int
        if x.left ≠ T.nil and x.left.max ≥ i.low
            x = x.left
        else
            x = x.right
    return x

```

- Finds any one overlapping interval in $O(\lg n)$. Correctness (Theorem 17.2): if no overlap in current subtree, searching either direction is safe — an overlapping interval exists iff it can be found this way.
- **Interval trichotomy:** For any two intervals i, j : i overlaps j ; i is left of j ($i.\text{high} < j.\text{low}$); i is right of j ($i.\text{low} > j.\text{high}$).

Interval trichotomy: For any two intervals: they overlap, or one is left of the other.

Applications: Josephus permutation ($O(n \lg n)$), counting inversions, k -th order statistics, interval scheduling.

Augmenting Red-Black Trees: Theorem 17.1 shows that if $x.f$ can be computed from x , $\text{left}.f$, $\text{right}.f$ in $O(1)$ time, then RB tree can maintain f in $O(\lg n)$ per operation. Examples: - **Size** (for order-statistic trees): $x.\text{size} = x.\text{left}.\text{size} + x.\text{right}.\text{size} + 1$ - **Max** (for interval trees): $x.\text{max} = \max(x.\text{int}.\text{high}, x.\text{left}.\text{max}, x.\text{right}.\text{max})$ - **Sum** (for range-sum queries): $x.\text{sum} = x.\text{left}.\text{sum} + x.\text{key} + x.\text{right}.\text{sum}$ - **Min gap** (for min difference queries): more complex since depends on in-order neighbors - During rotations: update f for the two nodes involved in $O(1)$ extra time.

Ch. 18 — B-Trees

Properties: Minimum degree $t \geq 2$. Every node (except root) $\geq t-1$ keys, $\leq 2t-1$ keys. All leaves at same depth.

Height bound (Theorem 18.1): $h \leq \log_t((n+1)/2)$. With $t=1001$, $n=10^9 \rightarrow h \leq 3$.

B-TREE-SEARCH(x, k): Search within node x linearly ($O(t)$), recurse to appropriate child. $O(t \log_t n)$ CPU, $O(\log_t n)$ disk accesses.

```

B-TREE-SEARCH(x, k)
    i = 1
    while i ≤ x.n and k > x.key[i]    i++
    if i ≤ x.n and k == x.key[i]    return (x, i)
    if x.leaf    return NIL
    DISK-READ(x.c[i])
    return B-TREE-SEARCH(x.c[i], k)

```

B-TREE-INSERT(T, k): Single pass — split full nodes on way down.

```

B-TREE-INSERT(T, k)
    r = T.root
    if r.n == 2t-1    // root is full
        s = ALLOCATE-NODE()
        T.root = s
        s.leaf = false; s.n = 0; s.c[1] = r
        B-TREE-SPLIT-CHILD(s, 1)
        B-TREE-INSERT-NONFULL(s, k)
    else
        B-TREE-INSERT-NONFULL(r, k)

```

- **B-TREE-SPLIT-CHILD(x, i):** Split full child y (with $2t-1$ keys) into y ($t-1$ keys) and z ($t-1$ keys). Median key moves to x. $O(t)$ CPU.
- **B-TREE-INSERT-NONFULL(x, k):** Recurse down, splitting full children along the way. Only splits at root increase height. $O(h)$ disk accesses.

B-TREE-DELETE(T, k): Single downward pass maintaining invariant: every visited node has $\geq t$ keys. - **Cases:** 1. k in leaf x: delete directly 2. k in internal node x: - If child preceding k has $\geq t$ keys: find predecessor, swap, recurse delete - Else if child following k has $\geq t$ keys: find successor, swap, recurse delete - Else: merge k and children; recurse delete in merged node 3. k not in internal node x: if child would have $< t$ keys, borrow from sibling (rotation) or merge with sibling; then recurse - Root may become empty after merge $\rightarrow$ height decreases by 1.

Comparisons: B-tree (disk-optimized, high branching factor) vs RB tree (memory, binary).

Ch. 19 — Disjoint Sets

Linked-list representation: Weighted-union heuristic (always append shorter list to longer). $O(m + n \lg n)$ for m ops (n MAKE-SET). Each element's pointer updated $\leq \lceil \lg n \rceil$ times.

Disjoint-set forest algorithms:

```

MAKE-SET(x)
    x.p = x    // parent pointer

```

```
x.rank = 0
```

```
FIND-SET(x)
```

```
    if x ≠ x.p  
        x.p = FIND-SET(x.p)    // path compression  
    return x.p
```

```
UNION(x, y)
```

```
    LINK(FIND-SET(x), FIND-SET(y))
```

```
LINK(x, y)
```

```
    if x.rank > y.rank  
        y.p = x  
    else  
        x.p = y  
        if x.rank == y.rank  
            y.rank = y.rank + 1
```

Union by rank: Root with smaller rank points to larger. Ranks only increase; only roots have their rank incremented. Rank = upper bound on height ($\leq \lg n$ for n elements).

Path compression: FIND-SET makes every node on path point directly to root. Two-pass: first pass traverses to root, second pass updates pointers.

Amortized analysis: $O(m \alpha(n))$ where $\alpha(n)$ is inverse Ackermann (≤ 4 for all practical $n \leq 10^{80}$). - **Potential function:** complex — uses levels and iteration counts. At most $\alpha(n)+2$ nodes per find path can require non-constant work.

Ackermann-like function $A_k(j)$: - $A_0(j) = j+1$ - $A_1(j) = 2j+1$ - $A_2(j) = 2^{\{j+1\}(j+1)-1}$ - $\alpha(n) = \min\{k : A_k(1) \geq n\}$. $A_3(1)=2047$, $A_4(1) > 10^{80}$. - For any practical n , $\alpha(n) \leq 4$. So effectively $O(m)$.

Ackermann-like function $A_k(j)$: $A_0(j)=j+1$, $A_1(j)=2j+1$, $A_2(j)=2^{\{j+1\}(j+1)-1}$. $\alpha(n) = \min\{k : A_k(1) \geq n\}$. $A_3(1)=2047$, $A_4(1) > 10^{80}$.

Amortized analysis: Complex potential function with level(x) and iter(x). At most $\alpha(n)+2$ nodes per find path without potential drop $\rightarrow O(\alpha(n))$ per FIND-SET.

Applications: Connected components (dynamic), offline minimum, Tarjan's offline LCA.

Implementation	m ops, n MAKE-SET
-----------------------	--------------------------

Linked-list simple	$\Theta(n^2)$ worst
--------------------	---------------------

Linked-list weighted	$O(m + n \lg n)$
----------------------	------------------

Forest (union by rank)	$O(m \lg n)$
------------------------	--------------

Forest (both heuristics)	$O(m \alpha(n))$
--------------------------	------------------

Ch. 20 — Elementary Graph Algorithms

Graph representations: Adjacency list $\Theta(V+E)$ memory, $\Theta(\text{degree}(u))$ edge lookup. Adjacency matrix $\Theta(V^2)$, $O(1)$ edge lookup.

BFS (Breadth-First Search)

```
for each  $u \in G.V - \{s\}$ :  $u.\text{color} = \text{WHITE}$ ,  $u.d = \infty$ ,  $u.\pi = \text{NIL}$ 
 $s.\text{color} = \text{GRAY}$ ,  $s.d = 0$ ,  $s.\pi = \text{NIL}$ 
 $Q = \emptyset$ ; ENQUEUE( $Q$ ,  $s$ )
while  $Q \neq \emptyset$ 
     $u = \text{DEQUEUE}(Q)$ 
    for each  $v \in G.\text{Adj}[u]$ 
        if  $v.\text{color} == \text{WHITE}$ 
             $v.\text{color} = \text{GRAY}$ ;  $v.d = u.d + 1$ ;  $v.\pi = u$ ; ENQUEUE( $Q$ ,  $v$ )
     $u.\text{color} = \text{BLACK}$ 
```

- Queue-based. Computes shortest-path distances (unweighted) and BFS tree.
- $O(V+E)$. Correctness (Theorem 20.5): $v.d = \delta(s,v)$ for all v reachable from s .
- **BFS tree:** predecessor subgraph $G_\pi = (V_\pi, E_\pi)$ where $V_\pi = \{v: v.\pi \neq \text{NIL}\} \cup \{s\}$.

DFS (Depth-First Search)

```
DFS( $G$ )
    for each  $u \in G.V$ :  $u.\text{color} = \text{WHITE}$ ,  $u.\pi = \text{NIL}$ 
     $\text{time} = 0$ 
    for each  $u \in G.V$ 
        if  $u.\text{color} == \text{WHITE}$     DFS-VISIT( $G$ ,  $u$ )
```

```
DFS-VISIT( $G$ ,  $u$ )
     $\text{time}++$ ;  $u.d = \text{time}$ ;  $u.\text{color} = \text{GRAY}$ 
    for each  $v \in G.\text{Adj}[u]$ 
        if  $v.\text{color} == \text{WHITE}$ 
             $v.\pi = u$ ; DFS-VISIT( $G$ ,  $v$ )
     $u.\text{color} = \text{BLACK}$ ;  $\text{time}++$ ;  $u.f = \text{time}$ 
```

- Discovery time $u.d$, finish time $u.f$. $O(V+E)$.
- **Edge classification:** tree edge ($u \rightarrow v$, v WHITE); back edge ($u \rightarrow v$, v GRAY); forward edge ($u \rightarrow v$, v BLACK, $u.d < v.d$); cross edge ($u \rightarrow v$, v BLACK, $u.d > v.d$).
- **Undirected graphs:** only tree edges and back edges (no forward/cross).

Parenthesis Theorem (Theorem 20.6): For any u, v , intervals $[u.d, u.f]$ and $[v.d, v.f]$ are either disjoint or one contains the other (nested). u is ancestor of $v \Leftrightarrow [v.d, v.f] \subseteq [u.d, u.f]$.

White-Path Theorem (Theorem 20.7): v is descendant of u in DFS forest $\Leftrightarrow$ at time $u.d$, there is a white path from u to v .

Topological Sort

TOPOLOGICAL-SORT(G)

```
    call DFS( $G$ ) to compute finish times  $v.f$  for all  $v$ 
    as each vertex is finished, prepend to linked list
    return linked list
```

- DFS + decreasing finish times $\rightarrow$ linear ordering of DAG. $\Theta(V+E)$.
- **Correctness** (Lemma 20.10): In DAG, for any edge (u,v) , $u.f > v.f$ (no back edges in DAG).
- **Kahn's algorithm**: Remove vertices with in-degree 0 iteratively. Alternative to DFS-based. $O(V+E)$.

Strongly Connected Components (Kosaraju)

STRONGLY-CONNECTED-COMPONENTS(G)

```
    call DFS( $G$ ) to compute finish times
    compute  $G^T$  (transpose: reverse all edges)
    call DFS( $G^T$ ), processing vertices in decreasing order of  $G$ 's
    finish times
    each DFS tree in  $G^T$  forest = one SCC
```

- $\Theta(V+E)$. Component graph (each SCC $\rightarrow$ one vertex) is a DAG.
- **Correctness**: G and G^T have same SCCs. Second DFS on G^T with finish order ensures only vertices within same SCC are discovered.
- **Tarjan's SCC algorithm**: single DFS using lowlink values (no transpose needed).
- **Applications**: dependency resolution, program optimization (loop detection), web page ranking.

Edge Cases: Self-loops = back edges; BFS distances independent of adjacency order, tree structure depends on it; DAG required for topological sort (cycle $\rightarrow$ impossible); Kosaraju works for any directed graph.

Ch. 21 — Minimum Spanning Trees

Cut Property (Theorem 21.1): Light edge crossing any cut that respects A is safe for A . Proof: cut-and-paste.

Cycle Property: Heaviest edge on any cycle is in no MST.

Kruskal's algorithm

MST-KRUSKAL(G, w)

```
 $A = \emptyset$ 
for each  $v \in G.V$     MAKE-SET( $v$ )
sort edges of  $G.E$  by nondecreasing weight  $w$ 
for each edge  $(u,v) \in G.E$  in order
    if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
         $A = A \cup \{(u,v)\}$ 
```

```

        UNION(u, v)
    return A

```

- Sort edges by weight; add if no cycle (FIND-SET). $O(E \lg E) = O(E \lg V)$ (sorting dominates).
- Builds forest (multiple trees merge). Uses disjoint-set data structure.
- **Cut Property** (safe edge): Across any cut that respects A, lightest edge is safe.

Prim's algorithm

```

MST-PRIM(G, w, r)
    for each u ∈ G.V: u.key = ∞, u.π = NIL
    r.key = 0
    Q = G.V      (min-priority queue keyed by .key)
    while Q ≠ ∅
        u = EXTRACT-MIN(Q)
        for each v ∈ G.Adj[u]
            if v ∈ Q and w(u,v) < v.key
                v.π = u; v.key = w(u,v)    (DECREASE-KEY)

```

- Grow single tree from root r; add lightest edge connecting tree to non-tree vertex.
- $O(E \lg V)$ with binary heap (E DECREASE-KEY + V EXTRACT-MIN), $O(V^2)$ with array for dense graphs, $O(E + V \lg V)$ with Fibonacci heap.

Uniqueness: If every cut has unique light edge, MST is unique. All MSTs have same sorted edge weight list.

Comparisons: | Dimension | Kruskal | Prim | |-----|-----|-----| | Strategy |
 Edge-based (global min) | Vertex-based (grow tree) | | DS | Disjoint-set +
 sorting | Min-priority queue | | Best for sparse | $O(E \lg V)$ | $O(E \lg V)$ binary |
 | Best for dense | $O(E \lg V)$ | $O(V^2)$ array |

Ch. 22 — Single-Source Shortest Paths

Relaxation: If $v.d > u.d + w(u,v)$, update $v.d = u.d + w(u,v)$, $v.\pi = u$.

Properties (Section 22.5): 1. Triangle inequality: $\delta(s,v) \leq \delta(s,u) + w(u,v)$ 2. Upper-bound: $v.d \geq \delta(s,v)$ always; once $v.d = \delta(s,v)$, never changes 3. No-path: unreachable $\rightarrow v.d = \infty$ always 4. Convergence: if $s \rightsquigarrow u \rightarrow v$ is shortest path and $u.d = \delta(s,u)$ before relaxing (u,v) , then $v.d = \delta(s,v)$ after 5. Path-relaxation: edges $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$ relaxed in order $\rightarrow v_k.d = \delta(s, v_k)$ 6. Predecessor-subgraph: $G_\pi = (V_\pi, E_\pi)$ is a shortest-paths tree rooted at s

Bellman-Ford

```

BELLMAN-FORD(G, w, s)
    INITIALIZE-SINGLE-SOURCE(G, s)    // s.d=0, all others ∞
    for i = 1 to |G.V| - 1
        for each edge (u,v) ∈ G.E

```

```

        RELAX(u, v, w)
    for each edge (u,v) ∈ G.E
        if v.d > u.d + w(u,v)
            return FALSE    // negative-weight cycle
    return TRUE

```

- Relax all edges $|V|-1$ times (any shortest path has $\leq |V|-1$ edges).
- $O(VE)$. Detects negative-weight cycles reachable from s .
- **Lemma 22.2:** After i iterations, Bellman-Ford finds shortest paths with $\leq i$ edges (path-relaxation property).
- **DAG Shortest Paths:** $\Theta(V+E)$. Topological sort + relax each edge once in order. Handles negative edges (no cycles). Fastest when applicable.

Dijkstra

```

DIJKSTRA(G, w, s)
    INITIALIZE-SINGLE-SOURCE(G, s)
    S = ∅
    Q = G.V    (min-priority queue keyed by .d)
    while Q ≠ ∅
        u = EXTRACT-MIN(Q)
        S = S ∪ {u}
        for each v ∈ G.Adj[u]
            RELAX(u, v, w)    // may call DECREASE-KEY

```

- Greedy: always extracts vertex with minimum shortest-path estimate.
- Requires nonnegative weights (Theorem 22.10: once extracted, $u.d = \delta(s,u)$).
- $O(V^2)$ with array, $O(E \lg V)$ with binary heap, $O(V \lg V + E)$ with Fibonacci heap.

Algorithm	Negative edges	Cycles	Time
BFS	No	No cycles	$O(V+E)$
Bellman-Ford	Yes	Detects	$O(VE)$
DAG-SP	Yes	None	$\Theta(V+E)$
Dijkstra	No	Not allowed	$O(E \lg V)$

Difference constraints: $x_j - x_i \leq b_k \rightarrow$ constraint graph $w(v_i, v_j) = b_k$; solve with Bellman-Ford.

Ch. 23 — All-Pairs Shortest Paths

Floyd-Warshall Algorithm:

```

FLOYD-WARSHALL(W)    // W = adjacency matrix (n×n)
    n = W.rows
    D(0) = W
    for k = 1 to n
        D(k) = new n×n matrix
        for i = 1 to n

```

```

    for j = 1 to n
         $d^{(k)}_{ij} = \min(d^{(k-1)}_{ij}, d^{(k-1)}_{ik} + d^{(k-1)}_{kj})$ 
    return  $D^{(n)}$ 

```

- DP on intermediate vertices $\{1, \dots, k\}$. $d^{(k)}_{ij} = \min(d^{(k-1)}_{ij}, d^{(k-1)}_{ik} + d^{(k-1)}_{kj})$.
- $\Theta(V^3)$ time, $\Theta(V^2)$ space when done in-place (single D matrix).
- **In-place version:** single D matrix, overwritten each iteration. $D[i,j] = \min(D[i,j], D[i,k] + D[k,j])$.
- **Path reconstruction:** maintain $\Pi^{(k)}[i,j]$ = predecessor of j on shortest path via $\{1..k\}$.
- **Negative edges:** works if no negative cycles. Detect by checking $D[i,i] < 0$ after completion.

Matrix Multiplication APSP (min-plus algebra): - EXTEND-SHORTEST-PATHS(A, B): $C[i,j] = \min_k (A[i,k] + B[k,j])$ — like matrix mult with (min, +) replacing (+, ×). - **SLOW-APSP:** start with $L^{(1)} = W$, compute $L^{(m)} = \text{EXTEND}(L^{(m-1)}, W)$ for $m=2..n-1$. $\Theta(V^4)$. - **Repeated squaring (FASTER-APSP):** $L^{(m)} = \text{EXTEND}(L^{(m/2)}, L^{(m/2)})$. Compute $L^{(1)}, L^{(2)}, L^{(4)}, \dots, L^{(2^{\lceil \lg(n) \rceil})}$. $\Theta(V^3 \lg V)$. - **Min-plus matrix multiplication** is associative, enabling squaring.

Johnson's Algorithm:

```

JOHNSON(G, w)
    compute  $G' = G + \text{super-source } s \text{ with edges } (s,v) \text{ weight } 0 \text{ for all } v$ 
    if BELLMAN-FORD( $G', w, s$ ) == FALSE
        return "negative-weight cycle"
    for each  $v \in G.V$   $h(v) = \delta(s,v)$  from Bellman-Ford
    for each edge  $(u,v) \in G.E$   $\hat{w}(u,v) = w(u,v) + h(u) - h(v)$ 
    for each  $u \in G.V$ 
        run DIJKSTRA( $G, \hat{w}, u$ ) to compute  $\delta(u,v)$ 
        for each  $v \in G.V$   $\delta(u,v) = \delta(u,v) + h(v) - h(u)$ 
    return  $D = (\delta(u,v))$  matrix

```

- $O(V^2 \lg V + VE)$. Best for sparse graphs with negative edges.
- **Reweighting lemma:** $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$. Preserves shortest paths; nonnegative $\hat{w}$ via triangle inequality.
- **Why reweighting works:** $h(v)$ = shortest path from s to v in G' . By triangle inequality, $h(v) \leq h(u) + w(u,v) \rightarrow \hat{w}(u,v) \geq 0$.
- **Correctness:** Since reweighting preserves path costs up to constant, shortest paths under $\hat{w}$ correspond to shortest paths under w .

Transitive closure: Boolean variant of Floyd-Warshall: - $t^{(k)}_{ij} = t^{(k-1)}_{ij} \vee (t^{(k-1)}_{ik} \wedge t^{(k-1)}_{kj})$. $\Theta(V^3)$. - Uses boolean matrix multiplication (OR for +, AND for ×). Can be optimized to $O(V^{\omega})$ using fast matrix multiplication.

Algorithm	Complexity	Best for
Floyd-Warshall	$\Theta(V^3)$	Dense

Algorithm	Complexity	Best for
FASTER-APSP	$\Theta(V^3 \lg V)$	Dense
Johnson	$O(V^2 \lg V + VE)$	Sparse, negative edges
Dijkstra $\times V$	$O(VE \lg V)$	Sparse, nonnegative

Ch. 24 — Maximum Flow

Flow network: Directed $G=(V,E)$ with capacity $c(u,v) \geq 0$, source s , sink t . Flow f satisfies capacity constraint ($0 \leq f \leq c$) and flow conservation (in = out for non-source/sink).

Ford-Fulkerson method

```

FORD-FULKERSON( $G, s, t$ )
  for each edge  $(u,v) \in G.E$ :  $f(u,v) = 0$ 
  while there exists augmenting path  $p$  in residual network  $G_f$ 
     $c_f(p) = \min\{c_f(u,v) : (u,v) \in p\}$ 
    for each edge  $(u,v) \in p$ 
      if  $(u,v) \in E$      $f(u,v) = f(u,v) + c_f(p)$ 
      else               $f(v,u) = f(v,u) - c_f(p)$ 
  return  $f$ 

```

- While augmenting path p exists in G_f , augment flow by bottleneck capacity.
- Worst-case $O(E |f^*|)$ with integer capacities (pathological: saturates 1 unit each time).

Residual network G_f : Edge capacities: $c_f(u,v) = c(u,v) - f(u,v)$ forward, $f(v,u)$ backward. - Augmenting path p : simple $s \rightarrow t$ path in G_f . Augment by $c_f(p) = \min\{c_f(u,v) \text{ along } p\}$. - **Lemma 24.1:** f' (after augmentation) is a flow in G with $|f'| = |f| + c_f(p) > |f|$.

Max-Flow Min-Cut Theorem (Theorem 24.6): For any flow network, TFAE:
 1. f is a maximum flow
 2. G_f has no augmenting path
 3. $|f| = c(S,T)$ for some cut (S,T) - **Proof:** (1) $\Rightarrow$ (2): path contradicts maximality. (2) $\Rightarrow$ (3): S = vertices reachable from s in G_f ; edges $S \rightarrow T$ saturated $\rightarrow |f| = c(S,T)$. (3) $\Rightarrow$ (1): $|f| \leq c(S,T)$ for any cut (Corollary 24.5).

Edmonds-Karp: Ford-Fulkerson using BFS (shortest augmenting path in terms of #edges). - $O(V E^2)$. Each edge becomes critical (bottleneck) $\leq |V|/2$ times $\rightarrow O(VE)$ augmentations. - **Theorem 24.8:** After each augmentation, shortest-path distance from s to any v is nondecreasing.

Integrality Theorem (Theorem 24.10): With integer capacities, Ford-Fulkerson produces integer-valued max flow. Crucial for bipartite matching reduction.

Maximum bipartite matching: Reduce to max flow: source $\rightarrow$ left(1), left $\rightarrow$ right(1), right $\rightarrow$ sink(1) capacities. Matching $M \leftrightarrow$ integer flow of value $|M|$.

Push-Relabel algorithm: Local operations: push (move flow along edge), relabel (increase height). $O(V^2 E)$ generic, $O(V^3)$ with FIFO. Practical alternative to augmenting paths. - **Preflow:** flow that may exceed conservation (excess at vertices). - **Height function h :** valid if $h(s)=|V|$, $h(t)=0$, $h(u) \leq h(v)+1$ for every residual edge. - **Discharge:** repeatedly push from overflowing vertex with highest label.

Comparisons: | Algorithm | Complexity | Notes | |-----|-----|-----| | Ford-Fulkerson | $O(E|f^*|)$ | Can be exponential | | Edmonds-Karp | $O(VE^2)$ | Polynomial | | Capacity scaling | $O(E^2 \lg C)$ | Reduces iterations | | Push-relabel | $O(V^3)$ | Practical, dominates |

Ch. 25 — Matchings in Bipartite Graphs

Berge's Theorem: M is maximum matching $\Leftrightarrow$ no M -augmenting path exists in G . - **M -augmenting path:** starts and ends at unmatched vertices, alternates non-matching/matching edges. - Used to prove correctness of all augmenting-path matching algorithms.

Hopcroft-Karp (Maximum bipartite matching)

HOPCROFT-KARP(G)

```

    M = ∅
    while there exists an augmenting path
        run BFS from unmatched left vertices to find shortest
        augmenting paths
        find maximal set of vertex-disjoint shortest augmenting
        paths via DFS (layered DAG)
        augment M by all paths simultaneously
    return M

```

- $O(\sqrt{V} \cdot E)$. Each phase increases shortest path length; $\leq 2\sqrt{V}$ phases.
- BFS creates layered graph (alternating levels). DFS finds vertex-disjoint paths ($O(E)$ per phase).

Gale-Shapley (Stable Marriage)

GALE-SHAPLEY(H, W)

```

    for each man m ∈ H: m.free = true
    while ∃ free man m with nonempty preference list
        w = highest-ranked woman on m's list not yet proposed to
        if w.free
            engage m and w
        else if w prefers m over current m'
            free m', engage m and w
        else
            m remains free (rejected)
    return matching

```

- $O(n^2)$ worst-case. Always produces a stable matching.

- **Properties:** Man-optimal (proposing) or woman-optimal depending on who proposes. Woman-optimal variant: women propose, men decide.
- **Theorem 25.8:** All stable matchings have same size; proposing side gets optimal matching, receiving side gets pessimal.

Hungarian Algorithm (Assignment Problem) - Find max-weight perfect matching in complete bipartite graph with $|L| = |R|$. - **Key ideas:** feasible labeling ℓ ($\ell(u) + \ell(v) \geq w(u,v)$), equality subgraph G_ℓ (edges with equality). Theorem: If G_ℓ has perfect matching, it's optimal. - $O(n^3)$ with careful implementation (slack array). $O(n^4)$ naive. - **Steps:** Initialize ℓ , find max matching in G_ℓ . If not perfect, update ℓ (decrease for unmatched left, increase for unmatched right).

Hall's Theorem: Perfect bipartite matching exists iff $|A| \leq |N(A)|$ for all $A \subseteq L$.

Problem	Algorithm	Complexity
Max bipartite matching	Hopcroft-Karp	$O(\sqrt{V} \cdot E)$
Stable marriage	Gale-Shapley	$O(n^2)$
Assignment (max weight)	Hungarian	$O(n^3)$

Ch. 26 — Parallel Algorithms

Fork-Join Model: - **spawn:** May execute in parallel with caller (scheduling is up to runtime). - **sync:** Wait for all spawned children to complete. - **pfor** (parallel for loop): Implemented via recursive spawning (divide range in half, spawn each half). - **Nested parallelism:** spawn within spawn; sync joins at multiple levels.

Work and Span Analysis: - **Work T_1 :** Total number of operations (1 processor). Represents serial time. - **Span T_∞ :** Length of critical path (infinite processors). Longest chain of dependencies. - **Space:** $T_P \geq \max(T_1/P, T_\infty)$ — lower bound for P processors. - **Greedy Scheduling** (Graham-Brent Theorem): $T_P \leq T_1/P + T_\infty$. Achieved by greedy scheduler. - **Parallelism:** $T_1/T_\infty =$ maximum possible speedup (linear speedup if $T_1/T_\infty \geq P$). - **Perfect linear speedup:** $T_P = T_1/P$ (requires $T_\infty \leq T_1/P$).

Parallel Merge Sort (P-MERGE): - MERGE(A,B,C): Use binary search on longer array to split; recursively merge pairs in parallel. - Work: $T_1 = \Theta(n)$. Span: $T_\infty = \Theta(\lg^2 n)$. Parallelism: $\Theta(n/\lg^2 n)$. - P-MERGE-SORT: Work $\Theta(n \lg n)$, Span $\Theta(\lg n \cdot \lg \lg n)$, Parallelism $\Theta(n/\lg \lg n)$.

Parallel Matrix Multiply (P-MATRIX-MULTIPLY): - Divide 2×2 blocks, recursively spawn multiplications in parallel. - Work: $T_1 = \Theta(n^3)$. Span: $T_\infty = \Theta(\lg n)$. Parallelism: $\Theta(n^3/\lg n)$. - Strassen-based: Work $\Theta(n^{\lg 7})$, Span $\Theta(\lg^2 n)$.

Parallel Sample Sort (P-SAMPLE-SORT): - Select $p-1$ splitters via random sampling, partition, recursively sort buckets. - Work: $\Theta(n \lg n)$, Span: $\Theta(\lg^2 n)$ with high probability. - **Linear speedup** with n processors when n is large enough.

Parallel Design Principles: - **Amdahl's Law:** $\text{Speedup} \leq 1/((1-f) + f/P)$ where f = parallel fraction. - **Work-efficient:** $T_1 = O(T_{\text{best serial}})$. Avoid parallel slowdown for small inputs. - **Granularity:** Too fine $\rightarrow$ overhead dominates; too coarse $\rightarrow$ low parallelism.

Ch. 27 — Online Algorithms

Competitive analysis: $C_{\text{ONLINE}}(I) \leq \rho \cdot C_{\text{OPT}}(I) + \alpha$ for all input sequences I . ρ = competitive ratio. If $\alpha = 0$, ρ -competitive.

Ski-Rental Problem: - Rent skis \$1/day or buy \$B. Future unknown. Deterministic: rent $B-1$ days then buy $\rightarrow$ cost = $B-1 + B = 2B-1$. OPT cost = $\min(B, \text{total days})$. Worst-case: total days = $B \rightarrow$ competitive ratio $2 - 1/B$. - **Randomized:** Rent with probability $1/(B-1)$, buy otherwise. Optimal ratio $e/(e-1) \approx 1.582$.

Move-to-Front (List Accessing): - After accessing element, move it to front of (linked) list. - **Theorem 27.1:** MTF is 2-competitive against optimal offline algorithm (which knows all future accesses). - **Cost model:** paying c_i to access element at position i , cost 1 to swap adjacent elements.

Caching (Paging): - **Model:** Cache size k . On miss (fault), fetch page from slow memory, evict one if cache full. Goal: minimize faults. - **Marking algorithm:** On fault, mark pages when requested. If all marked, clear marks. Evict unmarked page. - **Deterministic:** LRU (least recently used), FIFO (first-in-first-out), Marking are all k -competitive. - Proof that no deterministic beats k : adversary accesses pages $1..k$, then $k+1 \rightarrow$ fault. Then accesses $1..k$ again. LRU/fifo: each cycles through all pages, fault every time. OPT knows future, evicts page used furthest in future $\rightarrow$ faults once per full cycle. - **LIFO is not competitive:** adversary accesses page A , then B . LIFO evicts A on B 's fault. Then access A again $\rightarrow$ fault. Repeats $\rightarrow$ faults every other access = $O(n)$ vs $O(1)$ OPT. - **Randomized:** Random Marking = $O(\lg k)$ competitive. Lower bound $\Omega(\lg k)$ — Fiat et al. - **Application:** Virtual memory, web caching, CDN content delivery.

Online Learning (Weighted Majority): - n experts predict binary outcomes daily. Maintain weights w_i (initialized to 1). - **Deterministic:** Follow majority vote (weighted). Wrong experts multiplied by $(1-\gamma)$. - Mistake bound: $M \leq 2(1+\gamma) \cdot m^* + (2 \ln n)/\gamma$ where m^* = best expert's mistakes. - **Randomized:** Sample expert proportional to weights. Expected mistakes $\leq (1+\gamma) \cdot m^* + (\ln n)/\gamma$. - **Reduction to boosting:** AdaBoost uses multiplicative weights with caching of previous hypotheses. - **Regret minimization:** $\text{Regret} = M_{\text{alg}} - m^*$. Optimal γ gives regret $O(\sqrt{T \ln n})$.

Potential function analysis (MTF): - Φ = number of inversions (pairs (x,y) where order differs between MTF list and static optimal list). - Each access at position c : amortized cost = $2c-1 = O(c_{\text{OPT}})$. - MTF 2-competitive because amortized cost $\leq 2 \cdot (\text{cost of OPT list}) + O(1)$.

Ch. 28 — Matrix Operations

LUP Decomposition: $PA = LU$ where L is unit lower-triangular, U upper-triangular, P permutation matrix. - **Fact:** Any nonsingular matrix A has an LUP decomposition. Differences from LU: P handles zeros on diagonal. -

LUP-SOLVE: (1) Solve $Ly = Pb$ (forward substitution, $O(n^2)$); (2) Solve $Ux = y$ (back substitution, $O(n^2)$). - **Total:** $\Theta(n^3)$ decomposition (Gaussian elimination with partial pivoting) + $O(n^2)$ solve per RHS. -

Multiple solves: For many b vectors with same A , decompose once, solve each in $O(n^2)$. -

Partial pivoting: Choose row with largest absolute value in current column as pivot. Ensures numerical stability ($|L[i,j]| \leq 1$). - **CROUT/Doolittle:**

Variants of LU factorization without explicit P (store permutation separately). -

Dense vs sparse: Dense matrices $\rightarrow$ LUP ($\Theta(n^3)$). Sparse matrices $\rightarrow$ iterative methods (conjugate gradient, GMRES) — $O(\text{nnz} \cdot \text{iterations})$.

Matrix inversion: Solve $A \cdot X = I \rightarrow$ compute LUP, then solve $A \cdot x_j = e_j$ for each column j . - $\Theta(n^3)$ total (same as LUP solve $\times n$). Strassen-based inversion: $\Theta(n^{\lg 7})$ using recursive block formulation (Strassen 1969). -

Sherman-Morrison formula: $(A + uv^T)^{-1} = A^{-1} - A^{-1}uv^T A^{-1} / (1 + v^T A^{-1}u)$. Rank-1 update to inverse in $O(n^2)$. - **Woodbury**

identity: Generalizes to low-rank updates: $(A + UC^T V)^{-1} = A^{-1} - A^{-1}U(C + VA^T U)^{-1}VA^T$.

Positive-definite matrices: $x^T A x > 0$ for all nonzero x . All eigenvalues > 0 . - **Cholesky decomposition:** $A = LL^T$, L lower triangular with positive diagonal. Work = $\Theta(n^3/3)$ — half of LU ($\Theta(2n^3/3)$). Only works for symmetric positive-definite matrices. -

Cholesky algorithm: for $j = 1$ to n
 $L[j,j] = \sqrt{A[j,j] - \sum_{k<j} L[j,k]^2}$ for $i = j+1$ to n
 $L[i,j] = (A[i,j] - \sum_{k<j} L[i,k]L[j,k]) / L[j,j]$ -

Stability: Cholesky is numerically stable without pivoting. Used in Kalman filters, optimization (Newton's method), machine learning (GP regression).

Least squares: Minimize $\|Ax - b\|_2$ (overdetermined, $m > n$ equations). -

Normal equations: $A^T A x = A^T b$. Solve via Cholesky ($A^T A$ positive definite if full rank). Condition number $\kappa(A^T A) = \kappa(A)^2$ — squaring loses precision for ill-conditioned problems. - **QR decomposition:** $A = QR$ where Q is $m \times m$ orthogonal, R is $m \times n$ upper triangular. Solve $Rx = Q^T b$. More stable ($\kappa = \kappa(A)$). $\Theta(mn^2)$ using Householder reflectors or Givens rotations. -

Householder reflection: $H = I - 2vv^T/\|v\|^2$. Zeroes out below-diagonal entries. More stable than Gram-Schmidt. - **SVD (Singular Value**

Decomposition): $A = U\Sigma V^T$. Most numerically stable. $x = V\Sigma^{-1}U^T b$. Handles rank-deficient cases (pseudoinverse). $\Theta(mn^2 + n^3)$ cost. -

Comparison: Normal equations (fast but less stable), QR (moderate cost, good stability), SVD (most expensive, best stability).

Determinant: Via LUP decomposition: $\det(A) = \det(P) \cdot \prod_{i=1}^n U[i,i]$. $\det(P) = \pm 1$. $O(n^3)$.

Matrix norm: - **Frobenius:** $\|A\|_F = \sqrt{(\sum \sum A[i,j]^2)}$. - **Spectral (L2):** $\|A\|_2 = \sigma_{\max}(A)$ (largest singular value). - **Condition number:** $\kappa(A) = \|A\| \cdot \|A^{-1}\| = \sigma_{\max}/\sigma_{\min}$. High $\kappa \rightarrow$ ill-conditioned problem.

Ch. 29 — Linear Programming

Linear Programming Problem: Maximize (or minimize) a linear objective function subject to linear equality and inequality constraints.

Standard form: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. - **Converting to standard form:** (min $\rightarrow$ max, $\geq$ constraints $\rightarrow$ multiply by -1, equality $\rightarrow$ two inequalities, unrestricted variables $\rightarrow$ difference of two nonnegative variables). - **Slack form:** Maximize $z = c^T x$, subject to $Ax + s = b$, $x \geq 0$, $s \geq 0$ (s are slack variables). - **Basic feasible solution (BFS):** Set n nonbasic variables to 0, solve for m basic variables. BFS corresponds to vertex of feasible polytope. - **Degenerate BFS:** Some basic variables = 0. Can cause cycling (Bland's rule prevents).

Duality: - **Primal** (max): $\max c^T x$, $Ax \leq b$, $x \geq 0$. - **Dual** (min): $\min b^T y$, $A^T y \geq c$, $y \geq 0$. - **Asymmetric form** (equality constraints): Primal $\max c^T x$, $Ax = b$, $x \geq 0 \rightarrow$ Dual $\min b^T y$, $A^T y \geq c$ (y unrestricted). - **Weak duality:** $c^T x \leq b^T y$ for any feasible (x,y) . Proof: $c^T x \leq (A^T y)^T x = y^T (Ax) \leq y^T b$. - **Strong duality:** If primal (or dual) has an optimal solution, so does the other, and $c^T x^* = b^T y^*$. - **Complementary Slackness:** $x_j > 0 \Rightarrow \sum_i a_{ij} y_i = c_j$; $y_i > 0 \Rightarrow \sum_j a_{ij} x_j = b_i$. - **Duality gap:** $c^T x - b^T y \leq 0$ for feasible (x,y) . Zero at optimality. - **Economic interpretation:** Primal = resource allocation (max profit); Dual = resource valuation (min cost).

Simplex Algorithm: - Move from one BFS to adjacent BFS with better objective. - **Pivot:** Choose entering variable (positive reduced cost), exiting variable (minimum ratio test). - **Cycling:** Can loop forever. **Bland's rule:** choose smallest-index entering/exiting $\rightarrow$ guaranteed termination. - **Worst-case:** Klee-Minty hypercube $\rightarrow$ exponential (2^n iterations). - **Smoothed complexity:** Polynomial after random perturbation (Spielman-Teng, 2004).

Ellipsoid Method (Khachiyan, 1979): First polynomial-time LP algorithm. $O(n^4 \cdot L)$ operations, $O(n^6 \cdot L^2)$ bit complexity. Not practical.

Interior Point Methods (Karmarkar, 1984): - Follow central path through interior of feasible region. $O(n^3 \cdot L)$ total time. Practical for large problems. - **Barrier methods:** Add logarithmic barrier for inequality constraints. Solve sequence of Newton iterations. - **Primal-Dual methods:** Simultaneously update primal and dual variables. Fastest in practice.

LP Applications: - Max flow, min cost flow, bipartite matching, assignment, game theory (Nash equilibrium via zero-sum LP). - **Integer Linear Programming (ILP):** Constraints $x \in \mathbb{Z}$. NP-hard in general. Branch-and-bound. - **0-1 ILP:** Variables $\in \{0,1\}$. NP-complete (reduction from SAT).

Ch. 30 — Polynomials and FFT

Polynomial representations: - **Coefficient:** $a(x) = \sum_{j=0}^{n-1} a_j x^j$. Multiply: $O(n^2)$ naive. - **Point-value:** $(x_0, y_0), \dots, (x_{n-1}, y_{n-1})$ for n distinct x_k . Multiply: $O(n)$. Interpolate: $O(n^2)$ Gaussian, $O(n \lg n)$ FFT.

DFT and Roots of Unity: - $\omega_n = e^{2\pi i/n} = \cos(2\pi/n) + i \cdot \sin(2\pi/n)$. Principal n -th root of unity. - **Properties:** $\omega_n^n = 1$; $\omega_n^k \neq 1$ for $0 < k < n$; $\omega_{2n}^{2k} = \omega_n^k$; $\omega_n^{k+n/2} = -\omega_n^k$. - **DFT:** Evaluate $A(x)$ at $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$.

FFT (Cooley-Tukey, 1965):

RECURSIVE-FFT(a) // a is coefficient vector, $n = \text{length}$ (power of 2)

```
    if n == 1    return a
     $\omega_n = e^{2\pi i/n}$ ,  $\omega = 1$ 
     $a^0 = (a_0, a_2, \dots, a_{n-2})$     // even
     $a^1 = (a_1, a_3, \dots, a_{n-1})$     // odd
     $y^0 = \text{RECURSIVE-FFT}(a^0)$     // DFT of even
     $y^1 = \text{RECURSIVE-FFT}(a^1)$     // DFT of odd
    for k = 0 to  $n/2 - 1$ 
         $y_k = y^0_k + \omega \cdot y^1_k$ 
         $y_{k+n/2} = y^0_k - \omega \cdot y^1_k$ 
         $\omega = \omega \cdot \omega_n$ 
    return y
```

- Recurrence: $T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \lg n)$.
- **Inverse FFT:** Same algorithm with ω_n^{-1} (conjugate), divide by n . $\Theta(n \lg n)$.

Convolution (Polynomial Multiplication): - $a(x) \cdot b(x)$: Coefficient vector $c_k = \sum_{i+j=k} a_i \cdot b_j$. $O(n^2)$ naive. - **FFT-based:** (1) Evaluate A and B at n roots of unity (FFT, $O(n \lg n)$); (2) $C[k] = A[k] \cdot B[k]$ (pointwise, $O(n)$); (3) Inverse FFT ($O(n \lg n)$). Total: $\Theta(n \lg n)$.

Schönhage-Strassen Algorithm (1971): Large integer multiplication via FFT. $O(n \lg n \lg \lg n)$. Used in GMP (GMP). Theoretical record until 2007.

Practical FFT issues: - Bit-reversal permutation for in-place FFT. -

Bluestein's algorithm: FFT for arbitrary n (not just powers of 2). -

Number-theoretic transform (NTT): FFT over finite field $\mathbb{Z}_p$ where $p = k \cdot 2^m + 1$. No floating-point errors.

Ch. 31 — Number-Theoretic Algorithms

Ch. 31 — Number-Theoretic Algorithms

Euclid's algorithm

```

EUCLID(a, b)
  while b ≠ 0
    r = a mod b; a = b; b = r
  return a

```

- $O(\lg b)$ divisions. Worst-case: consecutive Fibonacci numbers (Lamé's theorem: $O(\lg \min(a,b))$).
- Recursive version: $\text{EUCLID}(a,b) = \text{if } b=0 \text{ then } a \text{ else } \text{EUCLID}(b, a \bmod b)$.

Extended Euclid

```

EXTENDED-EUCLID(a, b)
  if b == 0 return (a, 1, 0)
  (d, x', y') = EXTENDED-EUCLID(b, a mod b)
  (d, x, y) = (d, y', x' - [a/b]·y')
  return (d, x, y)

```

- Returns (d, x, y) where $d = \gcd(a,b) = a \cdot x + b \cdot y$. $O(\lg b)$.
- Computes modular inverses: $a^{-1} \bmod n = x \bmod n$ where x is from $\text{EXTENDED-EUCLID}(a,n)$ (requires $\gcd=1$).

Modular arithmetic: $\mathbb{Z}_n = \{0, \dots, n-1\}$. $a^{-1} \bmod n$ exists iff $\gcd(a,n)=1$. $\varphi(n) = n \prod_{p|n} (1-1/p)$ (Euler's totient). - **Fermat's theorem:** $a^{p-1} \equiv 1 \pmod{p}$ for prime $p \nmid a$. - **Euler's theorem:** $a^{\varphi(n)} \equiv 1 \pmod{n}$ for $\gcd(a,n)=1$.

Modular exponentiation: Square-and-multiply

```

MODULAR-EXPONENTIATION(a, b, n)
  result = 1
  while b > 0
    if b is odd result = (result·a) mod n
    a = a² mod n
    b = b >> 1
  return result

```

- $\Theta(\lg b)$ time (b 's bit length). Avoids huge intermediate values.

RSA (Rivest-Shamir-Adleman, 1977) - Key generation: Choose large primes p, q . $n = p \cdot q$. Choose e with $\gcd(e, \varphi(n))=1$. Compute $d = e^{-1} \bmod \varphi(n)$. Public: (n,e) . Private: (d) . - **Encrypt:** $c = m^e \bmod n$. **Decrypt:** $m = c^d \bmod n$. - **Correctness:** $m^{e \cdot d} \equiv m \pmod{n}$ by Euler's theorem + CRT. Security based on hardness of factoring n .

Primality testing: - **Fermat test:** $a^{n-1} \bmod n \neq 1 \rightarrow$ composite. **Carmichael numbers** (e.g., 561) pass all a with $\gcd(a,n)=1$. - **Miller-Rabin:** Write $n-1 = 2^s \cdot t$ with t odd. Check $a^t \bmod n \neq 1$ and no -1 in repeated squaring. Error $\leq 4^{-k}$. Practical for cryptography. - **AKS (Agrawal-Kayal-Saxena, 2002):** First deterministic polynomial-time primality test. $O((\log n)^{12}) = O(n^{12})$ naive $\rightarrow O((\log n)^{7.5})$ improved.

Chinese Remainder Theorem (Theorem 31.27): System $x \equiv a_i \pmod{n_i}$, pairwise coprime $n_i \rightarrow$ unique solution modulo $N = \prod n_i$. - **Solution:** $x = \sum a_i \cdot N_i \cdot y_i \pmod{N}$ where $N_i = N/n_i$, $y_i = N_i^{-1} \pmod{n_i}$.

Discrete logarithms: Given g, h in $\mathbb{Z}_p$, find x with $g^x \equiv h \pmod{p}$. Hard for large p . - **Baby-step giant-step:** $O(\sqrt{n})$ time and space. Write $x = i\lfloor\sqrt{n}\rfloor + j$, tabulate g^j , check $h \cdot g^{-i\lfloor\sqrt{n}\rfloor}$. - Basis for Diffie-Hellman key exchange and ElGamal encryption.

Ch. 32 — String Matching

Algorithm	Preprocessing	Matching	Space
Naive	0	$O(nm)$	0
Rabin-Karp	$O(m)$	$O(n+m)$ avg	$O(1)$
FA (automaton)	$O(m)$	Σ	)
KMP	$O(m)$	$O(n)$	$O(m)$

Rabin-Karp

```

RABIN-KARP(T, P, d, q)
    n = T.length, m = P.length
    h = d^{m-1} mod q
    p = 0; t_0 = 0
    for i = 1 to m
        p = (d · p + P[i]) mod q
        t_0 = (d · t_0 + T[i]) mod q
    for s = 0 to n-m
        if p == t_s
            if P[1:m] == T[s+1:s+m]    // verify (possible
spurious hit)
                print "match at shift s"
            if s < n-m
                t_{s+1} = (d · (t_s - T[s+1] · h) + T[s+m+1]) mod q

```

- Rolling hash: computes next hash in $O(1)$ from previous. Expected $O(n+m)$. Worst-case $\Theta(nm)$ (many spurious hits).
- Choose q large (e.g., $2^{31}-1$) to minimize collisions. d = alphabet size (e.g., 256 for ASCII).

KMP (Knuth-Morris-Pratt)

```

KMP-MATCHER(T, P)
    n = T.length, m = P.length
    π = COMPUTE-PREFIX-FUNCTION(P)
    q = 0    // number of matched characters
    for i = 1 to n
        while q > 0 and P[q+1] ≠ T[i]    q = π[q]
        if P[q+1] == T[i]    q++
        if q == m
            print "match at shift" i-m

```

$q = \pi[q]$

```

COMPUTE-PREFIX-FUNCTION(P)
  m = P.length,  $\pi[1] = 0$ 
  k = 0
  for q = 2 to m
    while k > 0 and P[k+1]  $\neq$  P[q]    k =  $\pi[k]$ 
    if P[k+1] == P[q]    k++
     $\pi[q] = k$ 
  return  $\pi$ 

```

- Prefix function $\pi[q]$ = longest proper prefix of P that is suffix of P[1:q].
- $O(n+m)$ — no backtracking in T. Each char compared at most once.
- **Idea:** On mismatch, use π to shift pattern without missing matches.

Suffix array: SA[i] = starting index of i-th lexicographically smallest suffix of T. - **Construction:** $O(n)$ via SA-IS (Suffix Array Induced Sorting) or $O(n \lg n)$ via doubling. - **LCP array:** Longest Common Prefix between adjacent suffixes in SA. Built in $O(n)$ via Kasai algorithm. - **Pattern matching:** Binary search over SA using LCP for acceleration: $O(m + \lg n)$ with LCP, $O(m \lg n)$ without. - **Applications:** substring search, longest repeated substring, longest common substring, Burrows-Wheeler transform.

Ch. 33 — Machine-Learning Algorithms

k-Means (Lloyd's Procedure):

```

K-MEANS(X[1:n], k)
  randomly select k initial centroids  $c_1, \dots, c_k$  from X
  repeat until convergence
    for i = 1 to n
      assign  $x_i$  to cluster  $j = \operatorname{argmin}_j ||x_i - c_j||^2$ 
    for j = 1 to k
       $c_j$  = mean of points in cluster j
  return centroids and assignments

```

- Converges to local minimum (not global). $O(T \cdot d \cdot k \cdot n)$ per iteration (T iterations, d dimensions, k clusters, n points).
- **Initialization:** k-means++ (spread-out initial centers) gives $O(\lg k)$ approximation guarantee.
- **Choosing k:** Elbow method (within-cluster SSE vs k), silhouette score, gap statistic.
- **Limitations:** Assumes spherical clusters, sensitive to outliers, local minima.

Multiplicative-Weight Algorithm (Weighted Majority):

```

WEIGHTED-MAJORITY(n, experts)
   $w_i = 1$  for  $i = 1..n$     // weights
  for each day t
    predict =  $\operatorname{argmax}_j \sum \{i: \text{expert}_i \text{ predicts } j\} w_i$ 

```

```

observe true outcome
for each expert i that was wrong
    w_i = w_i · (1-γ)    // γ = learning rate (e.g., 1/2)

```

- **Deterministic bound:** Mistakes $\leq 2(1+\gamma) \cdot m^* + (2 \ln n)/\gamma$ where m^* = best expert's mistakes.
- **Randomized variant:** Sample expert proportional to weights. Expected mistakes $\leq (1+\gamma) \cdot m^* + (\ln n)/\gamma$.
- **Regret:** Difference between algorithm's mistakes and best expert's. Randomized: $O(\sqrt{T \ln n})$ for optimal γ .
- **Applications:** Online learning, boosting (AdaBoost as special case), game theory (fictitious play).

Gradient Descent:

```

GRADIENT-DESCENT(f, x_0, T, γ)
    x = x_0
    for t = 1 to T
        x = x - γ · ∇f(x)    // move opposite gradient
    return weighted average Σ (x_t)/T

```

- For convex f with $\|x_0 - x^*\| \leq R$ (bound on distance) and $\|\nabla f(x)\| \leq L$ (Lipschitz gradient):
 - $f(x_{\text{avg}}) - f(x^*) \leq R \cdot L / \sqrt{T}$ after T iterations.
- **Learning rate:** $\gamma = R/(L \cdot \sqrt{T})$ gives optimal convergence.
- **Stochastic GD:** Use random sample ∇f instead of full gradient. Slower per-iteration (convergence $O(1/\sqrt{T})$), faster per-data-point.
- **Momentum:** $v_t = \beta \cdot v_{t-1} + \nabla f(x_t)$; $x_{t+1} = x_t - \gamma \cdot v_t$. Accelerates convergence.

Constrained GD: Projection $\Pi_K(x) = \operatorname{argmin}_{y \in K} \|x - y\|$. $x_{t+1} = \Pi_K(x_t - \gamma \cdot \nabla f(x_t))$. Same asymptotic bound. - **Projected GD:** For convex compact K . Each iteration: gradient step + projection. - **Mirror descent:** Generalized projection using Bregman divergence. For non-Euclidean geometry.

Ch. 34 — NP-Completeness

Complexity classes: - **P** = {L: L can be decided by a polynomial-time algorithm}. Examples: shortest path, MST, sorting. - **NP** = {L: L can be verified in polynomial time given a certificate}. "Solution verifiable in polynomial time." Examples: SAT, TSP, CLIQUE, SUBSET-SUM. - **NP-complete (NPC):** $L \in \text{NP}$ and every $L' \in \text{NP}$ reduces to L in polynomial time ($L' \leq_p L$). Hardest problems in NP. - **NP-hard:** $L' \leq_p L$ for every $L' \in \text{NP}$ (L may or may not be in NP). Includes optimization variants (e.g., TSP-OPT).

P ≠ NP is central open question (Clay Millennium Problem). If any NPC problem $\in \text{P}$, then $\text{P} = \text{NP}$.

Polynomial-time reduction: $L_1 \leq_p L_2$: exists polynomial-time computable function f such that $x \in L_1 \Leftrightarrow f(x) \in L_2$. - Reductions are transitive: $L_1 \leq_p L_2$

and $L_2 \leq_p L_3 \Rightarrow L_1 \leq_p L_3$. - To prove L is NP-complete: (1) Show $L \in \text{NP}$ (polynomial certificate and verifier); (2) Show $L' \leq_p L$ for some known NPC problem L' .

Standard NP-complete problems and reduction chain: 1. **CIRCUIT-SAT** (Theorem 34.7): Given boolean circuit, is there a satisfying assignment? First problem proved NP-complete (Cook-Levin, 1971). Proof: encode TM computation as circuit with AND/OR/NOT gates. 2. **SAT** $\leftarrow$ CIRCUIT-SAT: transform circuit gates into CNF clauses (Tseitin transformation). 3. **3-CNF-SAT** $\leftarrow$ SAT: parse SAT formula as parse tree, add auxiliary variables, convert to 3-CNF. 4. **CLIQUE** $\leftarrow$ 3-CNF-SAT: Create vertex per literal occurrence; edge between consistent literals in different clauses. $k =$ number of clauses. Clique of size $k \Leftrightarrow$ satisfying assignment. 5. **VERTEX-COVER** $\leftarrow$ CLIQUE: Complement graph $\bar{G} = (V, \bar{E})$. CLIQUE of size k in $G \Leftrightarrow$ VERTEX-COVER of size $|V|-k$ in $\bar{G}$. 6. **HAM-CYCLE** $\leftarrow$ VERTEX-COVER: Widgets for edges (selector paths) and vertices (cover verification). Complex gadget construction. 7. **TSP** $\leftarrow$ HAM-CYCLE: Complete graph with edge costs: 0 for edges in G , 1 for non-edges. HAM-CYCLE in $G \Leftrightarrow$ TSP tour of cost ≤ 0 . 8. **SUBSET-SUM** $\leftarrow$ 3-CNF-SAT: Base-10 digit encoding of clauses and variables. Variable numbers (v_i, v'_i) and slack numbers per clause.

Reduction strategies: - **Restriction:** Show problem contains known NPC problem as special case. - **Local replacement:** Replace each element with small gadget (3-CNF $\rightarrow$ CLIQUE). - **Component design:** Build components that interact (HAM-CYCLE gadgets). - **Gadgets:** Subgraphs/structures that enforce constraints (SAT $\rightarrow$ SUBSET-SUM).

Other NP-complete problems: 3-Colorability ($\pm$ gadget from 3-CNF-SAT), Independent Set ($\pm$ from CLIQUE), Hamiltonian Path, Set Partition, 0-1 Integer Programming, Subgraph Isomorphism, Dominating Set, Feedback Vertex Set.

Problems known to be in P: - 2-CNF-SAT (implication graph + SCCs — linear), - GRAPH-ISOMORPHISM status unknown (not known NP-complete, not known in P; Babai's quasipolynomial algorithm), - PRIMALITY (AKS, 2002 — polynomial), - Linear programming (ellipsoid, interior-point — polynomial).

In P: 2-CNF-SAT (implication graph SCCs), GRAPH-ISOMORPHISM status unclear.

Ch. 35 — Approximation Algorithms

Performance ratios: $C \leq \rho \cdot C^*$ (minimization) or $C^* \leq \rho \cdot C$ (maximization), $\rho \geq 1$. - **PTAS** (Polynomial-Time Approximation Scheme): $(1+\varepsilon)$ -approximation, polynomial in n for fixed ε (might be $n^{1/\varepsilon}$). - **FPTAS** (Fully PTAS): polynomial in n and $1/\varepsilon$ (e.g., $O(n^3/\varepsilon)$). Best possible for NP-hard optimization. - **APX:** Problems with constant-factor approximation (no PTAS unless $P=NP$ for many).

Vertex Cover (2-approximation):

APPROX-VERTEX-COVER(G)

```
C = ∅
E' = G.E
while E' ≠ ∅
    pick arbitrary (u,v) ∈ E'
    C = C ∪ {u,v}
    remove all edges incident to u or v from E'
return C
```

- Picks any edge, adds both endpoints, removes incident edges. Lower bound: maximal matching size $|M|$.
- $|C| = 2|M| \leq 2|OPT|$ (since OPT must cover M, each edge needs distinct vertex).
- **Weighted Vertex Cover:** LP relaxation + rounding (set $x(v) \geq 1/2$) → 2-approximation. LP: minimize $\sum w(v) \cdot x(v)$, $x(u) + x(v) \geq 1$ for all edges.

TSP with triangle inequality: - 2-approximation (Theorem 35.2):

Compute MST T, do preorder walk of T (visit each vertex when first encountered). $\text{Cost}(\text{TSP}) \leq 2 \cdot \text{Cost}(\text{optimal TSP})$. MST is lower bound on OPT (removing edge from optimal tour gives spanning tree). - **Christofides algorithm** (3/2-approximation): (1) MST T; (2) minimum-weight perfect matching on odd-degree vertices of T; (3) Eulerian tour on combined multigraph; (4) shortcut to Hamiltonian cycle. - **General TSP** (without triangle inequality): No constant-factor approximation unless $P=NP$ (Theorem 35.3). Reduction from HAM-CYCLE: cost 0 for edges in G, cost 1 otherwise. Distinguishing cost 0 vs cost n is NP-hard.

Set Cover (Greedy):

GREEDY-SET-COVER(X, F)

```
U = X    // uncovered elements
C = ∅
while U ≠ ∅
    select S ∈ F that maximizes |S ∩ U|    // most uncovered
elements covered
    U = U - S
    C = C ∪ {S}
return C
```

- $O(\ln |X|)$ -approximation. **Theorem 35.4:** Greedy achieves $H(d)$ -approximation where $d = \max \text{set size}$.
- **Tight bound:** $H_n = \ln n + O(1)$ — no polynomial algorithm can do better unless $P=NP$.
- **Weighted Set Cover:** Greedy picks set minimizing cost per newly covered element.

MAX-3-CNF-SAT: Random assignment satisfies each clause with probability $7/8 \rightarrow$ expected $8/7$ -approximation. - **Derandomization via method of conditional expectations:** Fix variables greedily, maintain conditional expectation $> 7/8$ of clauses.

Subset-Sum (FPTAS):

APPROX-SUBSET-SUM(S, t, ϵ)

$n = |S|, L_0 = \{\emptyset\}$

for $i = 1$ to n

$L_i = \text{MERGE}(L_{i-1}, L_{i-1} + x_i)$ // union

remove duplicates and trim: if $y \leq (1+\epsilon/2n) \cdot y'$, remove y

return max element $\leq t$ in L_n

- Trims lists with $\delta = \epsilon/2n$. $O(\text{poly}(n, 1/\epsilon))$. Running time: $O(n^3/\epsilon)$.
- **PTAS vs FPTAS**: Subset-Sum has FPTAS. MAX-CLIQUE has no PTAS unless $P=NP$.
- **0-1 Knapsack**: Also has FPTAS via rounding profits to $O(n/\epsilon)$ distinct values.

Key results table: | Problem | Approx Ratio | Algorithm | Complexity | |
 ---|---|---|---|---| | Vertex Cover | 2 | Maximal matching |
 $O(V+E)$ | | Weighted VC | 2 | LP rounding | $O(V^3)$ | | TSP (Δ -ineq) | 2 | MST-
 based | $O(V^2)$ | | TSP (Δ -ineq) | $3/2$ | Christofides | $O(V^3)$ | | General TSP | no
 constant | — | NP-hard | | Set Cover | $O(\ln n)$ | Greedy | $O(\Sigma|S|)$ | | MAX-3-
 CNF-SAT | $8/7$ | Random + derandomize | $O(n+m)$ | | Bin Packing | 2 (First-
 Fit) | $11/9$ approx | $O(n \lg n)$ | | Bin Packing | PTAS | Karmarkar-Karp |
 $O(n^{\{1/\epsilon\}})$ | | Max Cut | $1/2$ | Random assignment | $O(V+E)$ | | Max Cut |
 0.878 | Goemans-Williamson (SDP) | $O(V^3)$ | | 0-1 Knapsack | FPTAS | Profit
 rounding | $O(n^3/\epsilon)$ | | Subset-Sum | FPTAS | Trimming | $O(n^3/\epsilon)$ | | Steiner
 Tree | 2 | MST in metric closure | $O(V^2)$ |

Cross-Cutting Topics

Design Paradigms & Meta-Methods

Paradigm	Description	Examples
Incremental	Build solution element by element	Insertion sort
Divide-and-Conquer	Split, solve recursively, combine	Merge sort, Strassen, FFT
Dynamic Programming	Overlapping subproblems, optimal substructure	Matrix-chain, LCS, OBST, Floyd-Warshall
Greedy	Local optimal choice	Activity selection, Huffman, Prim, Kruskal, Dijkstra
Amortized Analysis	Average worst-case over sequence	Dynamic tables, binary counters
Randomization	Random choices eliminate input dependence	RANDOMIZED-QUICKSORT, RANDOMIZED-SELECT, universal hashing
Augmentation	Extra info in standard DS	Order-statistic trees, interval trees

Paradigm	Description	Examples
Relaxation	Iterative bound-tightening	Bellman-Ford, Dijkstra, Prim
Augmenting Paths	Iterative improvement along paths	Ford-Fulkerson, Hopcroft-Karp, Hungarian
Primal-Dual	Dual feasibility + complementary slackness	Hungarian, max-flow min-cut
Reweighting	Transform weights preserving structure	Johnson's algorithm
Reduction	Solve by transforming to known problem	Matching→Flow, Diff constraints→Shortest paths

Algorithm Selection Guide

When to choose each sorting algorithm: | Scenario | Algorithm | Why | |
 -----|-----|---| | n is small | Insertion sort | Low overhead, adaptive | | n
 is large, need worst-case guarantee | Heapsort or merge sort | $O(n \lg n)$
 worst-case | | n is large, average case fine | Quicksort | Fastest in practice | |
 Need stability | Merge sort | Only stable $O(n \lg n)$ | | Nearly sorted |
 Insertion sort | $O(n)$ best-case, adaptive | | Integers in $[0, k]$, $k=O(n)$ |
 Counting sort | $\Theta(n+k)$ | | Fixed-length integer keys | Radix sort | $\Theta(dn)$ | |
 Uniform reals in $[0, 1]$ | Bucket sort | $\Theta(n)$ expected |

When to choose each graph algorithm: | Scenario | Algorithm | Why | |
 -----|-----|---| | Shortest path, unweighted | BFS | $O(V+E)$ | | Shortest
 path, nonnegative weights | Dijkstra (binary heap) | $O(E \lg V)$ | | Shortest
 path, negative weights | Bellman-Ford | $O(VE)$ | | All-pairs, dense graph |
 Floyd-Warshall | $\Theta(V^3)$ | | All-pairs, sparse, negative edges | Johnson | $O(V^2 \lg V + VE)$ | |
 MST, sparse | Kruskal | $O(E \lg V)$ | | MST, dense | Prim (array) |
 $O(V^2)$ | | Max flow, general | Push-relabel (FIFO) | $O(V^3)$ | | Max flow, unit
 capacities | Dinic | $O(\min(V^{2/3}, \sqrt{E}) \cdot E)$ | | Bipartite matching | Hopcroft-
 Karp | $O(\sqrt{V} \cdot E)$ |

Proof & Argument Patterns

Pattern	Description	Used In
Loop Invariant	Property holds before each iteration	Insertion sort, PARTITION, HEAPSORT, RB-INSERT-FIXUP
Induction	Base case + inductive step	Substitution method, RB height bound, DP correctness
Cut-and-Paste	Swap nonoptimal sub-solution for optimal	DP optimal substructure, MST cut property
Exchange Argument		

Pattern	Description	Used In
	Transform any optimal to include greedy choice	Activity selection, Huffman, offline caching
Decision Tree Model	Lower bound via permutation counting	Comparison sort $\Omega(n \lg n)$
Contradiction	Assume false, derive impossible	BFS correctness, Dijkstra correctness
Parenthesis Nesting	Interval containment (discovery-finish)	DFS properties
White-Path Theorem	DFS descendant characterization	SCC correctness
Duality	Equivalent optimization perspectives	Max-flow min-cut, LP duality, Hungarian
Potential Function	Energy-like measure for amortized analysis	Dynamic tables, disjoint-set forests, binary counter
Path Relaxation	Edge relaxation in path order $\rightarrow$ correct distances	Bellman-Ford, Dijkstra, DAG-SP
Competitive Analysis	Online vs optimal offline	Paging, ski-rental, list accessing
Work-Span Analysis	Parallel algorithm performance	Graham-Brent scheduling, P-MERGE
0-1 Sorting Lemma	Oblivious compare-exchange sorts all iff sorts 0-1	Sorting networks
Regularity Condition	Checking Master Theorem Case 3	Recurrence solving
Adversary Argument	Worst-case input construction	Lower bounds (sorting, selection)
Amortized Analysis	Average cost over operations	Dynamic tables, binary counters, splay trees
Pigeonhole Principle	If n items in m boxes, some box has $\geq \lceil n/m \rceil$	Hashing, birthday paradox
Probabilistic Method	Show object exists by constructing randomly	Random graphs, MAX-SAT approximation

Probability & Statistics Foundation

- **Indicator random variables:** $E[I\{A\}] = \Pr\{A\}$
- **Linearity of expectation:** $E[\sum X_i] = \sum E[X_i]$ (always, even with dependence)
- **Geometric distribution:** Expected trials until success = $1/p$
- **Binomial distribution:** Expected successes in n trials = np ; $\text{Var} = np(1-p)$
- **Poisson distribution:** $\Pr[X=k] = e^{-\lambda} \lambda^k / k!$, $E[X] = \lambda$
- **Harmonic numbers:** $H_n = \ln n + \gamma + O(1/n)$, $\gamma \approx 0.577$
- **Birthday paradox:** $\Pr[\text{no collision}] \approx e^{-k(k-1)/(2n)}$; $k \approx \sqrt{2n \ln 2}$

- **Coupon collector:** Expected $b \cdot H_b \approx b \ln b$ trials
- **Chernoff bound:** $\Pr[X \leq (1-\delta)\mu] \leq e^{-\mu\delta^2/2}$, $\Pr[X \geq (1+\delta)\mu] \leq e^{-\mu\delta^2/3}$
- **Markov's inequality:** $\Pr[X \geq t] \leq E[X]/t$ for $X \geq 0$
- **Chebyshev's inequality:** $\Pr[|X-\mu| \geq k\sigma] \leq 1/k^2$
- **Union bound (Boole):** $\Pr[\cup A_i] \leq \sum \Pr[A_i]$
- **Conditional expectation:** $E[X] = E[E[X|Y]]$
- **Bayes' theorem:** $\Pr[A|B] = \Pr[B|A] \cdot \Pr[A] / \Pr[B]$
- **Jensen's inequality:** $f(E[X]) \leq E[f(X)]$ for convex f
- **Law of total probability:** $\Pr[A] = \sum \Pr[A|B_i] \cdot \Pr[B_i]$
- **Hypergeometric:** Sampling without replacement; $E[X] = n \cdot K/N$
- **Negative binomial:** Expected trials for r successes = r/p ; $\text{Var} = r(1-p)/p^2$
- **Multinomial:** Joint distribution for k categories; $E[X_i] = n \cdot p_i$
- **Exponential distribution:** $f(x) = \lambda e^{-\lambda x}$, $E[X] = 1/\lambda$, $\text{Var} = 1/\lambda^2$
- **Normal distribution:** $f(x) = (1/\sigma\sqrt{2\pi}) \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$; CLT: $\sum X_i \approx N(n\mu, n\sigma^2)$
- **Wald's equation:** $E[\sum_{i=1}^T X_i] = E[T] \cdot E[X]$ if T is stopping time, X_i i.i.d.
- **Median of sums:** $\text{median}(\sum X_i) \approx \mu \cdot n$ for symmetric distributions
- **Moment generating function:** $M_X(t) = E[e^{tX}]$; derivatives give moments
- **Characteristic function:** $\phi_X(t) = E[e^{itX}]$; always exists, uniquely determines distribution
- **Concentration of measure:** For $f(x)$ Lipschitz on n -dimensional sphere, $\Pr[|f - \text{median}| > t] \leq 2e^{-c \cdot n \cdot t^2}$
- **Azuma's inequality:** Martingale with bounded differences: $\Pr[|S_n - S_0| \geq t] \leq 2e^{-t^2/(2 \cdot \sum c_i^2)}$
- **Hoeffding's inequality:** For bounded independent $X_i \in [a_i, b_i]$: $\Pr[|S_n - E[S_n]| \geq t] \leq 2e^{-2t^2/\sum (b_i - a_i)^2}$
- **Method of moments:** $E[X^k] = \sum x^k \cdot \Pr[X=x]$; first two moments determine mean and variance
- **Maximum likelihood estimation:** $\theta = \text{argmax}_{\theta} \prod f(x_i|\theta)$ or equivalently $\text{argmax}_{\theta} \sum \ln f(x_i|\theta)$

People & Dates

Person	Contribution	Year
Al-Khowârizmî	Origin of "algorithm"	9th century
C. A. R. Hoare	Quicksort, RANDOMIZED-SELECT	1962
J. W. J. Williams	Heapsort, priority queue	1964
R. W. Floyd	BUILD-MAX-HEAP, Floyd-Warshall	1964
V. Strassen	$O(n^{\{2.81\}})$ matrix multiplication	1969
Blum, Floyd, Pratt, Rivest, Tarjan	Worst-case linear selection	1973
C. E. Shannon	Information theory, minimax for decision trees	1940s
D. Huffman	Huffman codes	1952
R. Bellman	Dynamic programming	1955

Person	Contribution	Year
J. Kruskal	MST algorithm	1956
R. Prim	MST algorithm (also Jarník 1930)	1957
E. W. Dijkstra	Shortest paths, SCC algorithm	1959
L. Ford, D. Fulkerson	Max-flow min-cut theorem	1956
J. Edmonds, R. Karp	Edmonds-Karp algorithm	1972
R. E. Tarjan	Disjoint-set analysis, SCC, Fibonacci heaps	1970s
R. Bayer, E. McCreight	B-trees	1972
L. Guibas, R. Sedgwick	Red-black trees	1978
J. Hopcroft, R. Karp	Bipartite matching (Hopcroft-Karp)	1973
D. Knuth	The Art of Computer Programming	1968
D. Gale, L. Shapley	Stable marriage problem	1962
H. Kuhn	Hungarian algorithm	1955
R. Rivest, A. Shamir, L. Adleman	RSA cryptography	1977
M. Rabin, G. Miller	Miller-Rabin primality test	1976
J. Cooley, J. Tukey	FFT algorithm	1965
L. Valiant	Theory of NP-completeness, #P	1979
S. Cook, L. Levin	NP-completeness theory	1971
R. Karp	21 NP-complete problems	1972
J. von Neumann	Merge sort for EDVAC	1945
A. C. Li, X. Williams	Matrix multiplication $O(n^{\{2.37287\}})$	2012
M. Agrawal, N. Kayal, N. Saxena	AKS primality test	2002
L. Euler	Eulerian circuits, totient function $\varphi(n)$	1700s
C. F. Gauss	Gaussian elimination	1809
N. Wirth	"Algorithms + Data Structures = Programs"	1976
J. McCarthy	Lisp, garbage collection, recursion	1960
A. Turing	Turing machine, computability, Enigma	1936
L. Lamport	LaTeX, Paxos consensus, temporal logic	1980s
B. Kernighan, D. Ritchie	C programming language	1978
E. F. Codd	Relational databases, relational algebra	1970
A. Aho, J. Hopcroft, J. Ullman	Design & Analysis of Algorithms textbook	1974

Person	Contribution	Year
T. Cormen, C. Leiserson, R. Rivest, C. Stein	Introduction to Algorithms (CLRS)	1990
L. Lovász	Lovász Local Lemma, ellipsoid method	1970s
P. Erdős	Probabilistic method, random graphs	1940s–90s
M. Blum	Blum’s axioms, Blum-Blum-Shub PRNG	1980s
A. Yao	Yao’s principle, pseudorandom generators	1980s
S. Cook	Cook-Levin theorem, NP-completeness	1971
L. Levin	Cook-Levin theorem (independent)	1973
R. Solovay, V. Strassen	Solovay-Strassen primality test	1974
L. Kantorovich	Linear programming (Kantorovich duality)	1939
G. Dantzig	Simplex method for linear programming	1947
N. Karmarkar	Interior-point method for LP	1984
L. Khachiyan	Ellipsoid method for LP	1979
A. Schönhage, V. Strassen	Schönhage-Strassen multiplication	1971
U. Feige, S. Goldwasser, L. Lovász, S. Safra, M. Szegedy	PCP theorem and inapproximability	1990s
D. Coppersmith, S. Winograd	Matrix multiplication $O(n^{\{2.376\}})$	1987
J. Hopcroft, J. Ullman	Automata theory, formal languages	1969
A. Church, S. C. Kleene	Lambda calculus, recursive functions	1930s
A. Schönhage	Fast integer multiplication	1971
R. Impagliazzo, R. Paturi	Exponential time hypothesis (ETH)	2001
P. Erdős, A. Rényi	Random graph model $G(n,p)$	1959
M. O. Rabin	Nondeterministic automata, Rabin-Karp, Miller-Rabin	1959
D. Scott	Nondeterministic finite automata	1959
D. S. Hirschberg	Linear space LCS algorithm	1975
E. M. Luks	Graph isomorphism in polynomial time for bounded degree	1982
L. Babai	Quasipolynomial graph isomorphism	2015
H. Buhrman, R. Cleve, A. Wigderson	Quantum computing, BQP	1990s

Person	Contribution	Year
P. Shor	Shor's factoring algorithm (quantum)	1994
L. Grover	Grover's search algorithm (quantum)	1996
S. Arora, C. Lund, R. Motwani, M. Sudan, M. Szegedy	PCP theorem proof	1992
S. Goldwasser, S. Micali	Zero-knowledge proofs	1985
A. Shamir	IP = PSPACE	1990
M. Santha, U. Vazirani	Quantum vs classical computation	1984
N. Alon	Combinatorial Nullstellensatz, expander graphs	1990s
L. Trevisan	Extractors, pseudorandomness	2000s
J. Håstad	MAX-SNP hardness, circuit lower bounds	1990s
R. Raz	Elusive functions, multilinear formulas	2010s
O. Reingold	Undirected connectivity in log space	2005
D. Zuckerman	Derandomization, extractors	2000s

Exam Questions by Type

MCQ

- Q:** What is the worst-case running time of quicksort on an already-sorted array using Lomuto partition? **A:** $\Theta(n^2)$
- Q:** Which of the following is NOT a stable sorting algorithm? **A:** Heapsort
- Q:** What is the running time of BUILD-MAX-HEAP on an array of n elements? **A:** $O(n)$
- Q:** What is the worst-case running time of any comparison sort for n elements? **A:** $\Omega(n \lg n)$
- Q:** In SELECT (worst-case linear selection), what group size is used? **A:** 5
- Q:** What is the maximum number of rotations during RB-INSERT-FIXUP? **A:** 2
- Q:** What is the load factor α in a hash table with n elements and m slots? **A:** $\alpha = n/m$

8. **Q:** In a red-black tree, if a node is red, what must be true about its children? **A:** Both children must be black
9. **Q:** What is the expected number of probes in unsuccessful open-address search when $\alpha = 0.9$? **A:** ≤ 10
10. **Q:** What is $\alpha(10^{100})$ (inverse Ackermann)? **A:** 4
11. **Q:** In a disjoint-set forest with union by rank and path compression, what is the amortized time per operation? **A:** $O(\alpha(n))$
12. **Q:** How many passes does Bellman-Ford make over all edges? **A:** $|V|-1$
13. **Q:** Which shortest-path algorithm works correctly with negative-weight edges but no negative cycles? **A:** Bellman-Ford
14. **Q:** The max-flow min-cut theorem states that: **A:** Maximum flow value = minimum cut capacity
15. **Q:** In DFS, what edge type is indicated when v is GRAY when (u,v) is first explored? **A:** Back edge
16. **Q:** Which is an asymptotically optimal comparison sort? **A:** Heapsort (or merge sort)
17. **Q:** What is the competitive ratio of LRU caching? **A:** k -competitive
18. **Q:** What is the recurrence for Floyd-Warshall? **A:** $d^{(k)}_{ij} = \min(d^{(k-1)}_{ij}, d^{(k-1)}_{ik} + d^{(k-1)}_{kj})$
19. **Q:** What property must a problem have for dynamic programming? **A:** Optimal substructure + overlapping subproblems
20. **Q:** What is the running time of KMP string matching? **A:** $O(n+m)$
21. **Q:** What is the inverse Ackermann function $\alpha(n)$ value for all practical n ? **A:** ≤ 4
22. **Q:** Which algorithm solves the assignment problem in $O(n^3)$? **A:** Hungarian algorithm
23. **Q:** What is the competitive ratio of optimal deterministic ski-rental? **A:** $2 - 1/B$
24. **Q:** What property must hold for a problem to be solved by a greedy algorithm? **A:** Greedy-choice property + optimal substructure
25. **Q:** What is the height bound for a B-tree with minimum degree t and n nodes? **A:** $h \leq \log_t((n+1)/2)$
26. **Q:** How many rotations can RB-INSERT-FIXUP perform at most? **A:** 2
27. **Q:** How many rotations can RB-DELETE-FIXUP perform at most? **A:** 3

28. **Q:** What is the expected number of probes for successful search in open addressing at load factor α ? **A:** $(1/\alpha) \ln(1/(1-\alpha))$
29. **Q:** Which hash family is universal: $h_{\{ab\}}(k) = ((ak+b) \bmod p) \bmod m$?
A: Yes, for p prime large
30. **Q:** What is the Miller-Rabin primality test's error bound after k tests?
A: $\leq 4^{-k}$
31. **Q:** In the Ford-Fulkerson method, what is the worst-case running time?
A: $O(E \cdot |f|)$ where $|f|$ is max flow value
32. **Q:** What does the Hopcroft-Karp algorithm compute? **A:** Maximum bipartite matching in $O(\sqrt{V} \cdot E)$
33. **Q:** What is the running time of Kosaraju's SCC algorithm? **A:** $\Theta(V+E)$
34. **Q:** What is the optimal substructure property? **A:** Optimal solution contains optimal solutions to subproblems
35. **Q:** What is the space complexity of Floyd-Warshall? **A:** $\Theta(V^2)$
36. **Q:** What is parallelism in the fork-join model? **A:** T_1/T_∞ (work over span)
37. **Q:** What is the first problem ever proven NP-complete? **A:** CIRCUIT-SAT (Cook-Levin, 1971)
38. **Q:** What type of tree must a Huffman encoding tree be? **A:** Full binary tree (every internal node has two children)
39. **Q:** What is the coupon collector's expected number of trials? **A:** $b \cdot H_b \approx b \ln b$
40. **Q:** For the secretary problem, what is the optimal rejection fraction? **A:** $1/e \approx 37\%$
41. **Q:** What is the time complexity of the naive recursive solution to the rod cutting problem? **A:** $O(2^n)$
42. **Q:** What data structure does Dijkstra's algorithm use as a priority queue? **A:** Min-heap (binary heap, or Fibonacci heap)
43. **Q:** In BFS, what color indicates an undiscovered vertex? **A:** WHITE
44. **Q:** What is the space complexity of the in-place version of Floyd-Warshall? **A:** $\Theta(V^2)$
45. **Q:** How many probes does double hashing typically require compared to linear probing? **A:** Fewer (avoid primary clustering)
46. **Q:** What is the running time of the Bellman-Ford algorithm on a graph with V vertices and E edges? **A:** $O(VE)$

47. **Q:** Which scheduling bound gives $T_P \leq T_1/P + T_\infty$? **A:** Graham-Brent (greedy scheduling)
48. **Q:** In the Ford-Fulkerson method, what is a residual network? **A:** Graph G_f showing remaining capacity and backward flow edges
49. **Q:** What is the key property that allows counting sort to be stable? **A:** Processing input in reverse order when placing elements
50. **Q:** What is the time complexity to build a segment tree from an array of n elements? **A:** $\Theta(n)$
51. **Q:** What is the amortized time for TABLE-INSERT in a dynamic array that doubles when full? **A:** $O(1)$
52. **Q:** What property do all paths from root to leaf in a red-black tree satisfy? **A:** Same number of black nodes (black-height)
53. **Q:** In Huffman coding, what type of binary tree must the optimal code tree be? **A:** Full binary tree (every internal node has exactly 2 children)
54. **Q:** What is the time complexity to find the minimum element in a max-heap? **A:** $\Theta(n)$ (must search all leaves)
55. **Q:** In the SELECT algorithm, what group size provides guaranteed linear time? **A:** 5
56. **Q:** What is the running time of Kruskal's algorithm on a graph with E edges? **A:** $O(E \lg V)$ (dominated by sorting edges)
57. **Q:** Does Johnson's algorithm require the graph to have nonnegative edge weights? **A:** No — it reweights to make all edges nonnegative
58. **Q:** What is the competitive ratio of the Move-to-Front list accessing algorithm? **A:** 2-competitive
59. **Q:** In the master theorem, what function determines the watershed between cases? **A:** $n^{\log_b a}$ (the critical exponent)
60. **Q:** What is the minimum number of keys in a B-tree of minimum degree t (non-root)? **A:** $t-1$
61. **Q:** What is the expected number of comparisons in RANDOMIZED-QUICKSORT? **A:** $O(n \lg n)$
62. **Q:** Which NP-complete problem is the first one proven to be NP-complete? **A:** CIRCUIT-SAT (Cook-Levin theorem)
63. **Q:** What is the approximation ratio of the greedy set cover algorithm? **A:** $O(\ln |X|)$
64. **Q:** What condition must hold for a greedy algorithm to work on a problem? **A:** Greedy-choice property + optimal substructure

65. **Q:** In the activity selection problem, which activity should be chosen first by the greedy algorithm? **A:** The one with the earliest finish time
66. **Q:** What is the worst-case time complexity of the Simplex algorithm? **A:** Exponential (Klee-Minty cube)
67. **Q:** What is the space complexity of the LCS DP table for two strings of length m and n ? **A:** $\Theta(mn)$
68. **Q:** In a BST, what is the successor of a node x ? **A:** The smallest key greater than $x.key$
69. **Q:** What technique does Johnson's algorithm use to make all edge weights nonnegative? **A:** Reweighting: $\hat{w}(u,v) = w(u,v) + h(u) - h(v)$
70. **Q:** What is the running time of the Hungarian algorithm for assignment? **A:** $O(n^3)$
71. **Q:** In parallel computing, what is the work T_1 ? **A:** Total number of operations (1 processor)
72. **Q:** What is the competitive ratio of optimal randomized ski-rental? **A:** $e/(e-1) \approx 1.582$
73. **Q:** What is the running time of the modular exponentiation algorithm? **A:** $O(\lg b)$ where b is the exponent
74. **Q:** For the extended Euclid algorithm, what does it return besides the gcd? **A:** Coefficients x, y such that $ax + by = \gcd(a, b)$
75. **Q:** What is the running time of the FFT algorithm? **A:** $\Theta(n \lg n)$
76. **Q:** In open addressing, what is the expected number of probes for an unsuccessful search? **A:** $\leq 1/(1-\alpha)$
77. **Q:** What does the MULTIPOP operation on a stack cost in amortized analysis? **A:** $O(1)$ amortized (each element popped at most once)
78. **Q:** What property does a universal family of hash functions guarantee? **A:** For any distinct keys, collision probability $\leq 1/m$
79. **Q:** What is the running time of Floyd-Warshall all-pairs shortest paths? **A:** $\Theta(V^3)$
80. **Q:** What is the optimal worst-case competitive ratio for deterministic caching? **A:** k (cache size)
81. **Q:** What is the running time of the extended Euclid algorithm? **A:** $O(\lg \min(a, b))$
82. **Q:** In RSA encryption, what is the public key? **A:** (n, e) where $n = p \cdot q$, $\gcd(e, \phi(n)) = 1$

83. **Q:** What is the running time of the naive polynomial multiplication algorithm? **A:** $\Theta(n^2)$
84. **Q:** In activity selection, what is the greedy choice? **A:** Activity with earliest finish time
85. **Q:** What is the competitive ratio of the OPTIMAL deterministic ski-rental algorithm? **A:** $2 - 1/B$
86. **Q:** What is the size of a Fibonacci heap's potential function based on? **A:** Number of trees + $2 \cdot (\text{number of marked nodes})$
87. **Q:** What is the time complexity to DELETE a node from a red-black tree? **A:** $O(\lg n)$
88. **Q:** In interval trees, what additional attribute is stored at each node? **A:** max = maximum endpoint in the subtree
89. **Q:** What does the π function in KMP represent? **A:** Longest proper prefix that is also a suffix
90. **Q:** What is the time complexity of the Schönhage-Strassen integer multiplication algorithm? **A:** $O(n \lg n \lg \lg n)$
91. **Q:** What type of bound does Chernoff's inequality provide for random variables? **A:** Concentration bound (exponentially decreasing tail)
92. **Q:** In the Hopcroft-Karp bipartite matching algorithm, what is the complexity? **A:** $O(\sqrt{V} \cdot E)$
93. **Q:** What is the worst-case space complexity of the naive recursive Fibonacci algorithm? **A:** $\Theta(n)$ (stack depth)
94. **Q:** What is the role of the sentinel in a red-black tree? **A:** T.nil represents all NIL leaves, simplifying boundary checks
95. **Q:** In Euler's totient function $\phi(n)$, what is $\phi(p)$ for a prime p ? **A:** $p-1$
96. **Q:** What is the running time of the LRU caching algorithm on a miss? **A:** $O(1)$ with a doubly linked list + hash map
97. **Q:** What is the worst-case stack depth of QUICKSORT with tail-recursion elimination? **A:** $O(\lg n)$
98. **Q:** What does the aggregate method of amortized analysis compute? **A:** Total cost over n operations divided by n
99. **Q:** In the Chinese Remainder Theorem, what condition must the moduli satisfy? **A:** Pairwise coprime
100. **Q:** What is the critical path in parallel algorithm analysis? **A:** The longest chain of dependent operations (span T_∞)
101. **Q:** What data structure does BFS use? **A:** Queue

102. **Q:** What data structure does DFS use? **A:** Stack (recursion or explicit stack)
103. **Q:** What is the running time of Strassen's matrix multiplication? **A:** $O(n^{\lg 7}) \approx O(n^{2.81})$
104. **Q:** In the decision tree model, what is the minimum height for a binary tree with $n!$ leaves? **A:** $\lceil \lg(n!) \rceil$
105. **Q:** What is the maximum number of edges in a bipartite matching? **A:** $\min(|L|, |R|)$
106. **Q:** What is the space complexity of BFS? **A:** $O(V)$
107. **Q:** What does the CLRS pseudocode operator $a \bmod b$ return? **A:** The remainder of a divided by b
108. **Q:** In the Chinese Remainder Theorem, how is the solution computed?
A: $x = \sum a_i \cdot N_i \cdot y_i \bmod N$ where $N = \prod n_i$, $N_i = N/n_i$, $y_i = N_i^{-1} \bmod n_i$
109. **Q:** What is the energy complexity of a circuit in the TM simulation of Cook-Levin? **A:** Polynomial ($O(T^2)$ gates where T is TM running time)
110. **Q:** In the Greedy Set Cover algorithm, what is the competitive ratio? **A:** $H(d) \leq \ln d + 1$ where $d = \max \text{ set size}$
111. **Q:** What is the approximation ratio of Christofides algorithm for TSP?
A: $3/2$
112. **Q:** What is the running time of the deterministic SELECT algorithm? **A:** $\Theta(n)$
113. **Q:** In the push-relabel algorithm, what does "discharge" mean? **A:** Repeatedly push flow from overflowing vertex until excess = 0
114. **Q:** What is the height of a d -ary heap with n elements? **A:** $\Theta(\log_d n)$
115. **Q:** What is the number of probe sequences in double hashing? **A:** $\Theta(m^2)$
116. **Q:** In the Longest Common Subsequence problem, what is the optimal substructure relation? **A:** If $x_i = y_j$, LCS includes x_i ; else LCS is max of $\text{LCS}(X_{\{i-1\}}, Y_j)$ and $\text{LCS}(X_i, Y_{\{j-1\}})$
117. **Q:** What is the maximum number of keys in a B-tree node of minimum degree t ? **A:** $2t-1$
118. **Q:** What is the purpose of the sentinel in a linked list? **A:** Eliminates boundary checks, simplifies code
119. **Q:** What is the query time of an interval tree for finding one overlapping interval? **A:** $O(\lg n)$

120. **Q:** In van Emde Boas trees, what is the universe size assumption? **A:** $u = 2^k$ for some integer k
121. **Q:** What is the expected height of a randomly built binary search tree? **A:** $O(\lg n)$
122. **Q:** What is the size of a perfect matching in a graph with $2n$ vertices? **A:** n edges
123. **Q:** What is the potential function of a binary counter for amortized analysis? **A:** Number of 1-bits
124. **Q:** In the Gale-Shapley algorithm, which side gets the optimal stable matching? **A:** The proposing side (man-optimal if men propose)
125. **Q:** What is the competitive ratio of the optimal randomized paging algorithm? **A:** $\Theta(\lg k)$ ($O(\lg k)$ by Random Marking, $\Omega(\lg k)$ lower bound)
126. **Q:** What is the running time of the Hungarian algorithm for the assignment problem? **A:** $O(n^3)$
127. **Q:** In the core of the SELECT algorithm, how many elements are guaranteed to be greater than the pivot? **A:** At least $3n/10$
128. **Q:** What is the convergence rate of gradient descent for convex functions? **A:** $O(1/\sqrt{T})$ (or $O(1/T)$ for strongly convex)
129. **Q:** What is the maximum number of edges in a graph with V vertices that avoids a triangle? **A:** $\lfloor V^2/4 \rfloor$ (Mantel's theorem)
130. **Q:** In the boyer-moore string matching algorithm, what are the two shift rules? **A:** Bad character rule and good suffix rule
131. **Q:** What is the definition of a red-black tree's black-height? **A:** Number of black nodes on path from node (excluding node) to any descendant leaf
132. **Q:** What is the time complexity of Dijkstra's algorithm with a Fibonacci heap? **A:** $O(V \lg V + E)$
133. **Q:** What is the expected length of a chain in a hash table with chaining when $\alpha = 2$? **A:** 2
134. **Q:** In the potential method for amortized analysis, what condition must Φ satisfy? **A:** $\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ for all i
135. **Q:** How many colors are needed to color any planar graph? **A:** 4 (Four Color Theorem)
136. **Q:** What is the formal definition of a Hamiltonian cycle? **A:** A cycle that visits each vertex exactly once

137. **Q:** In a flow network, what is the value of a flow f ? **A:** $|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$
138. **Q:** What is the most number of times the edge (u, v) can become critical in Edmonds-Karp? **A:** $|V|/2$ times
139. **Q:** What is the definition of the convolution of two sequences a and b ?
A: $(a * b)_k = \sum_{i+j=k} a_i \cdot b_j$
140. **Q:** In the RSA cryptosystem, what is the relationship between the public and private exponents? **A:** $d \equiv e^{-1} \pmod{\phi(n)}$
141. **Q:** What is the concept of a “safe edge” in building an MST? **A:** An edge that can be added to A while maintaining the invariant that $A \subseteq$ some MST
142. **Q:** In a suffix array, what does $LCP[i]$ represent? **A:** Length of the longest common prefix between suffixes at $SA[i]$ and $SA[i-1]$
143. **Q:** How many comparisons does MINIMUM require? **A:** $n-1$
144. **Q:** In BFS, what invariant holds for the queue? **A:** Vertices have d values differing by at most 1
145. **Q:** What is the time complexity of the fast exponentiation (square-and-multiply)? **A:** $O(\lg b)$ where b is the exponent
146. **Q:** What is the expected number of ball tosses to fill b bins? **A:** $b \cdot H_b \approx b \ln b$
147. **Q:** What is the amortized cost of TABLE-DELETE when halving at $\alpha = 1/4$? **A:** $O(1)$
148. **Q:** In a trie, what is the time to search for a pattern of length m ? **A:** $O(m)$
149. **Q:** What is the running time to build a suffix array using the doubling algorithm? **A:** $O(n \lg n)$
150. **Q:** What is the number of vertices in a complete binary tree of height h ?
A: $2^{h+1} - 1$ ($h = 0$ for root)
151. **Q:** In the unit-cost RAM model, what operations take constant time? **A:** Arithmetic, data movement, control, and memory access (within word size)
152. **Q:** How can you detect a negative-weight cycle in a graph? **A:** Run Bellman-Ford; if any vertex can still be relaxed after $|V|-1$ iterations, there’s a negative cycle
153. **Q:** What is the minimax principle in Yao’s algorithm analysis? **A:** Expected running time of optimal randomized algorithm = expected running time of optimal deterministic on worst-case input distribution

154. **Q:** What is the rank of an element in an order-statistic tree? **A:** Its position in the inorder traversal (1-indexed)
155. **Q:** What does the LCP array enable in suffix-array-based string matching? **A:** $O(m + \lg n)$ pattern matching vs $O(m \lg n)$ without LCP
156. **Q:** What is the maximum depth of a red-black tree with n nodes? **A:** $2 \lg(n+1)$
157. **Q:** In disjoint-set forests, what does the rank of a node represent? **A:** An upper bound on the height of the subtree rooted at that node
158. **Q:** What is the optimal number of bits per digit in radix sort when sorting n b -bit integers? **A:** $r = \lceil \lg n \rceil$, giving $\Theta(bn/\lg n)$ total time
159. **Q:** What is the recurrence for the running time of the naive recursive Fibonacci algorithm? **A:** $T(n) = T(n-1) + T(n-2) + \Theta(1) = \Theta(2^n)$
160. **Q:** What property of the Miller-Rabin test makes it practical for cryptography? **A:** Error probability $\leq 4^{-k}$ after k tests; can be made negligibly small
161. **Q:** In worst-case linear selection (SELECT), what is the role of the group medians? **A:** The median of group medians serves as pivot, ensuring at least $3n/10$ elements on each side
162. **Q:** In the analysis of QUICKSORT, what is the probability that two elements are compared? **A:** $2/(j-i+1)$ where i, j are their indices in sorted order
163. **Q:** What is the concept of a “spurious hit” in Rabin-Karp string matching? **A:** A hash collision that incorrectly suggests a pattern match, requiring verification
164. **Q:** What is the work of P-MERGE on n elements? **A:** $\Theta(n)$ — same as serial merge
165. **Q:** What is the span of P-MERGE on n elements? **A:** $\Theta(\lg^2 n)$ — binary search on longer half, then parallel recursive merges
166. **Q:** In Brent’s scheduling theorem, what does T_P represent? **A:** Running time on P processors under greedy scheduling
167. **Q:** What is the purpose of the DECREASE-KEY operation in Prim’s algorithm? **A:** Updates the minimum edge weight connecting a non-tree vertex to the growing MST
168. **Q:** What is the time to compute powers of a matrix for the FASTER-APSP algorithm? **A:** $\Theta(V^3 \lg V)$ — repeated squaring of min-plus matrix multiplication
169. **Q:** In the Gale-Shapley algorithm, what does it mean for a matching to be “stable”? **A:** No pair of unmatched man and woman both prefer each other over their current partners

170. **Q:** What is the definition of an M-augmenting path in bipartite matching? **A:** A path starting and ending at unmatched vertices, alternating non-matching and matching edges
171. **Q:** In Berge's theorem, what condition characterizes a maximum matching? **A:** No M-augmenting path exists
172. **Q:** What is the time complexity to solve a system $Ax=b$ using LUP decomposition? **A:** $\Theta(n^3)$ decomposing A + $O(n^2)$ solving
173. **Q:** In the Simplex algorithm, what is the minimum ratio test used for? **A:** To determine the exiting variable (which basic variable reaches zero first)
174. **Q:** What is Bland's rule for the Simplex algorithm? **A:** Choose smallest-index entering and exiting variables to prevent cycling
175. **Q:** In the ellipsoid method, what is the volume reduction factor per iteration? **A:** $e^{-1/(2(n+1))}$ for n-dimensional ellipsoid
176. **Q:** What is the principal n-th root of unity in the FFT? **A:** $\omega_n = e^{2\pi i/n}$, satisfying $\omega_n^n = 1$ and $\omega_n^k \neq 1$ for $0 < k < n$
177. **Q:** In the FFT, what property of roots of unity enables the divide-and-conquer? **A:** $\omega_n^{2k} = \omega_{n/2}^k$ (halving lemma) and $\omega_n^{k+n/2} = -\omega_n^k$
178. **Q:** What is the running time of integer multiplication using Schönhage-Strassen? **A:** $O(n \lg n \lg \lg n)$
179. **Q:** In the RSA algorithm, what is the purpose of Euler's totient $\phi(n)$? **A:** Choose e coprime to $\phi(n)$, compute $d = e^{-1} \bmod \phi(n)$
180. **Q:** What is the Fermat test for primality? **A:** Check if $a^{n-1} \equiv 1 \pmod{n}$ for random a . Fails for Carmichael numbers
181. **Q:** In the KMP prefix function, what is the meaning of $\pi[q]$? **A:** Length of longest proper prefix of $P[1:q]$ that is also a suffix of $P[1:q]$
182. **Q:** What is the worst-case number of comparisons in the KMP matching phase? **A:** $O(n)$ — each character of T compared at most once
183. **Q:** What is the basic operation in the push-relabel max-flow algorithm? **A:** Push (move flow along residual edge) and Relabel (increase height)
184. **Q:** In the Hopcroft-Karp algorithm, what property do augmenting paths have in each phase? **A:** They are vertex-disjoint and all have the same minimal length
185. **Q:** What is the time complexity of the Hungarian algorithm using slack variables? **A:** $O(n^3)$
186. **Q:** In the multiplicative-weights algorithm, what is the regret bound? **A:** $\text{Regret} = O(\sqrt{T \ln n})$ after T rounds with n experts

187. **Q:** What is the purpose of gradient descent for convex optimization? **A:** Find x^* minimizing $f(x)$ by iteratively moving opposite the gradient
188. **Q:** In gradient descent, what is the convergence rate for general convex functions? **A:** $O(1/\sqrt{T})$, meaning $f(x_{\text{avg}}) - f(x^*) \leq O(1/\sqrt{T})$
189. **Q:** What is the definition of a PTAS? **A:** Polynomial-Time Approximation Scheme: $(1+\epsilon)$ -approximation in time polynomial in n for any fixed ϵ
190. **Q:** What is the difference between PTAS and FPTAS? **A:** FPTAS is polynomial in n AND $1/\epsilon$; PTAS may be exponential in $1/\epsilon$
191. **Q:** What is the integrality gap of an LP relaxation? **A:** Maximum ratio between IP optimum and LP optimum (bound on achievable approximation)
192. **Q:** In linear programming, what is a basic feasible solution? **A:** A solution where n variables are set to 0 (nonbasic) and the remaining m are solved from equality constraints
193. **Q:** In a flow network, what is the capacity of a cut (S, T) ? **A:** $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$
194. **Q:** In bipartite matching, what is a maximal matching? **A:** A matching that cannot be extended by adding another edge (not necessarily maximum)
195. **Q:** In the residual network G_f , what does a backward edge represent? **A:** The ability to “undo” previously sent flow (send flow in opposite direction)
196. **Q:** In interval trees, what is the max attribute used for? **A:** To determine whether an interval overlapping the query exists in a subtree
197. **Q:** In the order-statistic tree, how is the size attribute updated during a rotation? **A:** Recompute $\text{size} = \text{size}[\text{left}] + \text{size}[\text{right}] + 1$ for the two rotated nodes
198. **Q:** In dynamic programming, what is the Principle of Optimality? **A:** An optimal policy has the property that whatever the initial state and initial decision, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision
199. **Q:** What is a Catalan number and what does it count? **A:** $C_n = \frac{1}{(n+1)} \cdot C(2n, n)$. Counts: BSTs with n nodes, parenthesizations of $n+1$ factors, triangulations of convex $(n+2)$ -gon
200. **Q:** In a red-black tree, what is the maximum number of red nodes on any root-to-leaf path? **A:** At most $h/2$ (no consecutive reds, at least half are black)
201. **Q:** What is the relationship between P and NP if a polynomial-time algorithm for SUBSET-SUM exists? **A:** $P = NP$ (SUBSET-SUM is NP-complete, so any polynomial algorithm for it puts all of NP in P)

202. **Q:** What is the recurrence for the expected time of RANDOMIZED-QUICKSORT? **A:** $E[T(n)] = (1/n) \cdot \sum_{k=1}^n (E[T(k-1)] + E[T(n-k)]) + \Theta(n) = \Theta(n \lg n)$
203. **Q:** What does the notation $f(n) = o(g(n))$ mean? **A:** $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$ (f is asymptotically smaller than g)
204. **Q:** How many probe sequences does double hashing provide? **A:** $\Theta(m^2)$ (depends on two independent hash functions)
205. **Q:** What is the span of P-MERGE for merging two length- n sequences? **A:** $\Theta(\lg^2 n)$
206. **Q:** In parallel algorithms, what is the work of P-MERGE-SORT on n elements? **A:** $\Theta(n \lg n)$
207. **Q:** What is the competitive ratio of the deterministic ski-rental algorithm? **A:** $2 - 1/B$ (rent $B-1$ days then buy)
208. **Q:** What is the expected competitive ratio of randomized ski-rental? **A:** $e/(e-1) \approx 1.582$
209. **Q:** In the secretary problem with n candidates, what is the optimal rejection window size? **A:** n/e candidates (approximately 37% of n)
210. **Q:** What is the probability of hiring the best candidate in the optimal secretary algorithm? **A:** $1/e \approx 37\%$
211. **Q:** What is the condition for applying Case 3 of the Master Theorem? **A:** $f(n) = \Omega(n^{\log_b a + \epsilon})$ AND regularity: $a \cdot f(n/b) \leq c \cdot f(n)$ for $c < 1$
212. **Q:** In RSA, what is the relationship between n , p , and q ? **A:** $n = p \cdot q$ where p and q are distinct large primes
213. **Q:** In RSA, given public key (n, e) , what is the private key d ? **A:** $d \equiv e^{-1} \pmod{\phi(n)}$ where $\phi(n) = (p-1)(q-1)$
214. **Q:** In the FFT, what is the value of the primitive n th root of unity ω_n ? **A:** $\omega_n = e^{2\pi i/n} = \cos(2\pi/n) + i \cdot \sin(2\pi/n)$
215. **Q:** What is the convolution theorem? **A:** $\text{FFT}(a \otimes b) = \text{FFT}(a) \cdot \text{FFT}(b)$ where $\otimes$ is convolution and $\cdot$ is pointwise multiplication
216. **Q:** What does the cancellation lemma state for ω_n ? **A:** $\omega_{dn}^{dk} = \omega_n^k$
217. **Q:** In dynamic programming, what is the difference between memoization and bottom-up? **A:** Memoization: top-down recursion + caching; Bottom-up: iterative table-filling from base cases
218. **Q:** In a flow network, what is a blocking flow? **A:** A flow such that every path from s to t in G_f contains at least one saturated edge

219. **Q:** In Dinic's algorithm, how is the level graph constructed? **A:** BFS from s in G_f , edges only if they go to a higher level ($d[v] = d[u] + 1$)
220. **Q:** What is the time complexity of Dinic's algorithm on a unit-capacity network? **A:** $O(\sqrt{V} \cdot E)$
221. **Q:** What is the difference between LRU and FIFO cache eviction policies? **A:** LRU evicts least recently used (based on access history); FIFO evicts the page that arrived earliest
222. **Q:** In amortized analysis of a dynamic table, what is a typical choice of potential function? **A:** $\Phi = 2 \cdot \text{num} - \text{size}$ (or $\Phi = |2 \cdot \text{num} - \text{size}|$ for contraction)
223. **Q:** What is the expected number of collisions when inserting n keys into m slots? **A:** $C(n, 2)/m \approx n^2/(2m)$ (birthday paradox)
224. **Q:** What does the KMP prefix function $\pi[q]$ represent? **A:** The length of the longest proper prefix of P that is a suffix of $P[1..q]$
225. **Q:** In Dijkstra's algorithm, what data structure is typically used for the priority queue? **A:** Binary min-heap (or Fibonacci heap for theoretical improvement)
226. **Q:** What is the key invariant in Dijkstra's algorithm? **A:** When a vertex u is extracted from the priority queue, $u.d = \delta(s, u)$ (shortest path is final)
227. **Q:** In the cut property for MSTs, what does it mean for a cut to "respect" a set A ? **A:** No edge in A crosses the cut ($A \subseteq$ one side of the cut)
228. **Q:** How many times can each edge be relaxed in a correct implementation of Bellman-Ford? **A:** Exactly $|V|-1$ times (once per pass)
229. **Q:** What is the best achievable competitive ratio for deterministic online caching? **A:** k -competitive (k = cache size); lower bound matches upper bound
230. **Q:** In the Weighted Majority algorithm, what is the regret bound? **A:** $O(\sqrt{T \ln n})$ or $O(\ln n)$ for expert setting; number of mistakes $\leq 2.41 \cdot (M) + O(\ln n)$ for the Halving version
231. **Q:** What is the expected number of iterations of k -means clustering? **A:** Finite (distortion strictly decreases), but exponential worst-case. Typically small in practice.
232. **Q:** In gradient descent for convex functions, how many iterations are needed to reach ϵ error? **A:** $O(1/\epsilon)$ for standard GD; $O(1/\sqrt{\epsilon})$ for Nesterov accelerated GD
233. **Q:** What is the time complexity of Johnson's algorithm for all-pairs shortest paths? **A:** $O(V^2 \lg V + VE)$

234. **Q:** In Strassen's algorithm, how many recursive multiplications are performed? **A:** 7 (instead of 8 for conventional D&C)
235. **Q:** What is the fastest known matrix multiplication exponent? **A:** $\omega < 2.37287$ (Alman-Williams, 2024)
236. **Q:** In 0-1 knapsack, why is the DP algorithm not considered polynomial? **A:** W (capacity) is input in binary, so $O(nW)$ is exponential in input size (pseudopolynomial)
237. **Q:** What is the approximation ratio of the greedy set cover algorithm? **A:** $H_n \approx \ln n$ (H_n -approximation; cannot be improved unless $P=NP$)
238. **Q:** What is the approximation ratio of the greedy vertex cover algorithm? **A:** 2-approximation (repeatedly pick an uncovered edge, add both endpoints)
239. **Q:** In Christofides' algorithm for TSP with triangle inequality, what is the approximation ratio? **A:** $3/2$ -approximation
240. **Q:** What is the time complexity of the naive algorithm for solving TSP (brute force)? **A:** $O(n!)$
241. **Q:** In the AKS primality test, what is the key identity used? **A:** $(x-a)^n \equiv x^n - a \pmod{n}$ iff n is prime
242. **Q:** What is the error probability of the Miller-Rabin primality test after k iterations? **A:** At most 4^{-k}
243. **Q:** In the number field sieve, what is the asymptotic complexity for factoring n ? **A:** $\exp(O((\lg n)^{1/3}(\lg \lg n)^{2/3}))$
244. **Q:** What is the computational complexity of the Fourier transform on n points? **A:** $\Theta(n \lg n)$ via FFT; $\Theta(n^2)$ via direct DFT
245. **Q:** In the Gale-Shapley stable marriage algorithm, is the resulting matching always optimal for the proposers? **A:** Yes — it is proposer-optimal (each proposer gets best possible partner in any stable matching)
246. **Q:** In the Hungarian algorithm for assignment, what is the time complexity? **A:** $O(n^3)$
247. **Q:** What is the maximum possible number of edges in a bipartite graph with n vertices on each side? **A:** n^2 (complete bipartite graph $K_{n,n}$)
248. **Q:** In linear programming, what does the dual simplex method solve? **A:** The dual problem while maintaining dual feasibility and seeking primal feasibility
249. **Q:** In perceptron learning, what is the bound on the number of mistakes for linearly separable data? **A:** $O(R^2/\gamma^2)$ where $R = \max ||x_i||$ and γ = margin

250. **Q:** What does the Kernel trick allow in SVM? **A:** Implicit mapping to higher-dimensional space without computing the mapping explicitly (via kernel function)
251. **Q:** In number theory, what does Euler's theorem state? **A:** $a^{\varphi(n)} \equiv 1 \pmod{n}$ for $\gcd(a,n) = 1$
252. **Q:** What is the time complexity of the extended Euclidean algorithm on inputs a, b ? **A:** $O(\lg \min(a,b))$ (number of divisions)
253. **Q:** In a B-tree with minimum degree t , what is the maximum number of keys in a node? **A:** $2t - 1$
254. **Q:** In a B-tree with minimum degree t , what is the maximum number of children? **A:** $2t$
255. **Q:** In a B-tree with minimum degree t , what is the minimum number of keys in a non-root node? **A:** $t - 1$
256. **Q:** In a B-tree of height h , what is the maximum number of nodes? **A:** $1 + 2t + (2t)^2 + \dots + (2t)^h = ((2t)^{h+1} - 1)/(2t - 1)$
257. **Q:** In the potential method of amortized analysis, what condition must the potential function satisfy? **A:** $\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ for all i
258. **Q:** What is the minimum number of nodes in a red-black tree of black-height bh ? **A:** $2^{\{bh\}} - 1$ (all black nodes in a perfect binary tree)
259. **Q:** In the analysis of quicksort, what is the probability that z_i and z_j are compared? **A:** $2/(j - i + 1)$
260. **Q:** In a binomial heap, what property must the binomial trees satisfy? **A:** Each binomial tree B_k consists of two B_{k-1} trees linked together (min-heap ordered)
261. **Q:** What is the worst-case time for a FIND-SET operation without path compression? **A:** $O(n)$ (linked list structure)
262. **Q:** In persistent data structures, what is the difference between partial and full persistence? **A:** Partial: query any version, update only latest. Full: query and update any version (functional persistence: immutable)
263. **Q:** In computational geometry, what is the convex hull of a set of points? **A:** The smallest convex set containing all points; the vertices of which are the extreme points
264. **Q:** In the sliding window maximum problem, what data structure gives $O(n)$ time? **A:** Deque (double-ended queue) maintaining monotonic decreasing order of values
265. **Q:** In counting sort, if the range of input values is k , what is the space complexity? **A:** $O(k)$ for the count array + $O(n)$ for output

266. **Q:** In radix sort, if each digit is sorted using counting sort, what is the total time for d digits? **A:** $\Theta(d(n+k))$
267. **Q:** In the fast exponentiation algorithm (square-and-multiply), how many multiplications are needed for exponent k ? **A:** $O(\lg k)$ (at most $2 \cdot \lfloor \lg k \rfloor$ multiplications)
268. **Q:** In modular arithmetic, what is the Legendre symbol (a/p) ? **A:** 1 if a is a quadratic residue mod p , -1 if not, 0 if $p|a$
269. **Q:** In the Verlet integration method, what is the error order? **A:** $O(\Delta t^4)$ — fourth-order accuracy for position; used in molecular dynamics
270. **Q:** In the Simpson's rule for numerical integration, what is the error bound? **A:** $O(h^4 \cdot f^{(4)}(\xi))$ — fourth-order accuracy for smooth functions
271. **Q:** What is the running time of the Kruskal algorithm when edges are already sorted? **A:** $O(E \alpha(V)) \approx O(E)$ (union-find operations dominate)
272. **Q:** In the insertion sort, how many comparisons on a reverse-sorted array of size n ? **A:** $n(n-1)/2 = \Theta(n^2)$
273. **Q:** In the binary search tree, what is the successor of a node with a right subtree? **A:** The minimum node in its right subtree
274. **Q:** In the binary search tree, what is the predecessor of a node with no left subtree? **A:** Go up until finding a node where current is in the right subtree
275. **Q:** In the recursive matrix multiplication D&C, how many matrix additions of size $n/2 \times n/2$? **A:** 4 (2 per recursion level for combining 8 sub-matrix products)
276. **Q:** In the skip list data structure, what is the expected number of levels for n elements? **A:** $O(\lg n)$ (with high probability) when each element is promoted with probability $1/2$
277. **Q:** In dynamic programming for LCS, what is the space-optimized version? **A:** Keep only two rows (previous and current) for $O(\min(m,n))$ space
278. **Q:** In the Floyd-Warshall algorithm, what is the initialization for the predecessor matrix Π ? **A:** $\pi[i,j] = i$ if there is an edge (i,j) , else NIL (or 0 for $i=j$)
279. **Q:** In the SAXPY operation (BLAS level 1), what is the computation? **A:** $y = \alpha \cdot x + y$ (scalar α times x plus y) — single-vector operation
280. **Q:** In the Matrix Market format for sparse matrices, which three arrays are stored? **A:** Row indices, column indices, and non-zero values (COO format); or compressed row/column

281. **Q:** In the maximum subarray problem (Kadane's algorithm), what is the recurrence? **A:** Best ending at $i = \max(\text{arr}[i], \text{best ending at } i-1 + \text{arr}[i])$
282. **Q:** In the longest common substring problem, what is the DP recurrence? **A:** $\text{dp}[i,j] = \text{dp}[i-1,j-1] + 1$ if $x[i]=y[j]$ else 0 (not max like LCS)
283. **Q:** In the N-Queens problem, what is the standard algorithm? **A:** Backtracking with pruning: place queens row by row, ensuring no column, diagonal, or anti-diagonal conflict
284. **Q:** In the maximum independent set problem on a tree, what is the DP recurrence? **A:** $\text{dp}[u] = \max(1 + \sum \text{dp}[\text{grandchild}], \sum \text{dp}[\text{child}])$
285. **Q:** In the AVL tree, what is the balance factor of a node? **A:** $\text{height}(\text{left}) - \text{height}(\text{right})$, with allowed values $\{-1, 0, 1\}$
286. **Q:** In the Fibonacci heap, what happens when a node loses two children? **A:** It is cut from its parent and added to the root list (cascading cut)
287. **Q:** In the Tarjan's off-line LCA algorithm, what data structure is used? **A:** Disjoint-set (union-find) with ancestor tracking
288. **Q:** In the Karger's randomized min-cut algorithm, what is the probability of finding the global min cut? **A:** $\Omega(1/n^2)$ per run; after $O(n^2 \log n)$ runs, success probability $\rightarrow 1$
289. **Q:** In the Chinese Postman Problem, what is the goal? **A:** Shortest closed walk covering every edge at least once (Eulerian trail with duplicates)
290. **Q:** In the Bloom filter, can elements be deleted? **A:** Not with standard Bloom filter (use counting Bloom filter for deletion)
291. **Q:** In the locality-sensitive hashing (LSH), what property defines the hash family? **A:** Similar items hash to same bucket with high probability; dissimilar items with low probability
292. **Q:** In the MinHash algorithm for Jaccard similarity, what is the estimator? **A:** $\Pr[\text{min hash of } A = \text{min hash of } B] = |A \cap B| / |A \cup B| = \text{Jaccard}(A,B)$
293. **Q:** In the SimHash algorithm for cosine similarity, what happens at the end? **A:** Build fingerprint by sign of random projection: positive $\rightarrow 1$, negative $\rightarrow 0$
294. **Q:** In the PageRank algorithm, what is the teleportation parameter typically set to? **A:** $\alpha = 0.85$ (probability of following links, $1-\alpha$ probability of random jump)

295. **Q:** In the Google matrix formulation of PageRank, what is the equation?
A: $\pi = \alpha \cdot \pi \cdot P + (1-\alpha) \cdot e/n$ where π is PageRank vector, P is adjacency stochastic matrix
296. **Q:** In the power iteration method for PageRank, what is the convergence rate? **A:** $O(|\lambda_2/\lambda_1|^k)$ where λ_1, λ_2 are the two largest eigenvalues; converges geometrically
297. **Q:** In the Apriori algorithm for frequent itemset mining, what is the Apriori principle? **A:** Any subset of a frequent itemset must be frequent; pruning: if a itemset is infrequent, all supersets are infrequent
298. **Q:** In the Johnson-Lindenstrauss lemma, what is the target dimension for preserving pairwise distances? **A:** $k = O(\epsilon^{-2} \log n)$ for preserving distances within factor $(1 \pm \epsilon)$
299. **Q:** In the iterative method for solving linear systems (Jacobi), what is the update rule? **A:** $x^{\{(k+1)\}}_i = (1/a_{ii}) \cdot (b_i - \sum_{j \neq i} a_{ij} \cdot x^{\{(k)\}}_j)$
300. **Q:** In the conjugate gradient method, what is the key property of search directions? **A:** They are A-conjugate: $d_i^T \cdot A \cdot d_j = 0$ for $i \neq j$ (mutually orthogonal with respect to A)
301. **Q:** What is the time complexity of the Fast Fourier Transform (FFT) on n points? **A:** $\Theta(n \lg n)$
302. **Q:** In the FFT, how is the DFT computed recursively? **A:** Split into even-indexed and odd-indexed elements, compute DFT of each half ($n/2$ points), combine using ω_n^k factors
303. **Q:** What is the key property used in the KMP prefix function to achieve linear time? **A:** When a mismatch occurs, set $q = \pi[q]$ to skip characters already known to match; never backtrack in T
304. **Q:** In the Rabin-Karp algorithm, what method is used to update the hash value? **A:** Rolling hash: $h_{\{t+1\}} = d \cdot (h_t - d^{\{m-1\}} \cdot T[t+1]) + T[t+m+1] \pmod{q}$ where d is alphabet size
305. **Q:** In the finite automaton string matcher, what is the running time if the transition table is precomputed? **A:** $\Theta(n)$ — constant time per text character
306. **Q:** In the Boyer-Moore-Horspool algorithm, what heuristic determines the shift? **A:** Bad character rule: shift so that the mismatched text character aligns with its last occurrence in the pattern
307. **Q:** In the Z-algorithm for string matching, what does the Z-array represent? **A:** $Z[i] = \text{length of longest common prefix between } S \text{ and } S[i..n]$ (longest substring starting at i that matches prefix)
308. **Q:** In the TEA block cipher, what is the key size and number of rounds? **A:** 128-bit key, 64 rounds (32 cycles of 2 rounds each)

309. **Q:** In the AES block cipher, what are the three key size variants? **A:** AES-128 (10 rounds), AES-192 (12 rounds), AES-256 (14 rounds)
310. **Q:** In the Mersenne Twister PRNG, what is the period? **A:** $2^{\{19937\}} - 1$ (Mersenne prime)
311. **Q:** In the Blum-Blum-Shub PRNG, what is the key operation? **A:** $x_{\{n+1\}} = x_n^2 \bmod M$ where $M = p \cdot q$ (product of two Blum primes)
312. **Q:** In the algorithm for listing all subsets of an n -element set, what is the time complexity? **A:** $\Theta(n \cdot 2^n)$ — each of 2^n subsets takes $O(n)$ to generate
313. **Q:** In the algorithm for generating all permutations of n elements, what is the time complexity? **A:** $\Theta(n \cdot n!)$ — Heap's algorithm or Johnson-Trotter generate each in $O(1)$ amortized
314. **Q:** In the Bell numbers, how many ways to partition an n -element set? **A:** $B_n = \sum_{k=0}^{n-1} C(n-1, k) \cdot B_k$; $B_0 = 1$, $B_1 = 1$, $B_2 = 2$, $B_3 = 5$, $B_4 = 15$
315. **Q:** In the Stirling numbers of the second kind, $S(n, k)$ counts what? **A:** Number of ways to partition n labeled elements into k nonempty unlabeled subsets
316. **Q:** In the Eulerian numbers, $A(n, k)$ counts what? **A:** Number of permutations of $\{1, \dots, n\}$ with exactly k ascents (elements greater than previous)
317. **Q:** In the partition problem, if the total sum is $2S$, what determines a valid partition? **A:** A subset summing to exactly S — the other subset automatically sums to S
318. **Q:** In the bin packing problem with bins of capacity 1, what is the First-Fit-Decreasing approximation ratio? **A:** $11/9 \approx 1.222$ (worst-case; empirical even better)
319. **Q:** In the subset sum problem, what is the time complexity of the meet-in-the-middle algorithm? **A:** $O(2^{\{n/2\}} \cdot n)$ — split into two halves of $n/2$ elements, compute all subset sums for each, combine
320. **Q:** In the edit distance problem, what is the recurrence for the cost of substituting character a to b ? **A:** Cost = 0 if $a = b$, else 1 (Levenshtein distance); or arbitrary cost (generalized)
321. **Q:** In the Needleman-Wunsch global alignment algorithm, what gap penalty scheme is typically used? **A:** Affine gap penalty: $\text{gap_open} + (\text{length}-1) \cdot \text{gap_extend}$; prefers fewer but longer gaps
322. **Q:** In the Smith-Waterman local alignment algorithm, how does the DP table differ from global alignment? **A:** Values never go below 0; best local alignment = maximum value in table (not bottom-right corner)

323. **Q:** In the Viterbi algorithm for HMM decoding, what is the time complexity? **A:** $O(T \cdot K^2)$ where T = sequence length, K = number of hidden states
324. **Q:** In the Baum-Welch algorithm for HMM training, what is the time complexity per iteration? **A:** $O(T \cdot K^2)$
325. **Q:** In the CYK algorithm for CFG parsing, what is the time complexity? **A:** $O(n^3 \cdot |G|)$ where n = string length, $|G|$ = grammar size (requires Chomsky Normal Form)
326. **Q:** In the Earley parser for CFG parsing, what is the time complexity? **A:** $O(n^3)$ worst-case; $O(n^2)$ for unambiguous grammars; $O(n)$ for LR(k) grammars
327. **Q:** In the LR parsing algorithm, what data structure is the primary stack? **A:** A stack of states (DFA states of the LR automaton)
328. **Q:** In the SHA-256 hash function, what is the output length in bits? **A:** 256 bits (internal state: 8 32-bit words)
329. **Q:** In the RANDOMIZED-QUICKSORT partition, what is the expected number of comparisons? **A:** $2n \cdot H_n - 2(n-1) \approx 2n \cdot \ln n - O(n)$
330. **Q:** In the algorithm for counting inversions using merge sort, what is the running time? **A:** $\Theta(n \lg n)$ — count during merge step: when $A[i] > A[j]$, add $(\text{mid}-i+1)$ to inversion count
331. **Q:** In the arithmetic coding algorithm, how is the interval updated for each symbol? **A:** $\text{low} = \text{low} + \text{range} \cdot \text{cum_prob}[\text{prev}]$; $\text{high} = \text{low} + \text{range} \cdot \text{prob}[\text{curr}]$; $\text{range} = \text{high} - \text{low}$
332. **Q:** In the Lagrange interpolation formula, how many points are needed for degree d polynomial? **A:** $d+1$ distinct points uniquely determine the polynomial
333. **Q:** In the Newton's method for root finding, what is the convergence rate? **A:** Quadratic convergence near the root (number of correct digits doubles each iteration)
334. **Q:** In the k -nearest neighbors (k -NN) classification, what is the time complexity for a single query? **A:** $O(n \cdot d)$ for brute force; $O(\lg n)$ with KD-tree in low dimensions
335. **Q:** In the principal component analysis (PCA), what mathematical operation is central? **A:** Eigenvalue decomposition of the covariance matrix (or SVD of the data matrix)
336. **Q:** In the SVD decomposition $A = U \cdot \Sigma \cdot V^T$, what do columns of U and V represent? **A:** U : left singular vectors (eigenvectors of AA^T); V : right singular vectors (eigenvectors of A^TA)

337. **Q:** In the Monte Carlo method for estimating π , how many samples are needed for ε error? **A:** $O(1/\varepsilon^2)$ — variance reduction techniques reduce constant but not rate
338. **Q:** In the Metropolis-Hastings algorithm, what is the acceptance probability formula? **A:** $\alpha(x \rightarrow y) = \min(1, \pi(y) \cdot q(x|y) / (\pi(x) \cdot q(y|x)))$ where π is target, q is proposal
339. **Q:** In the Gibbs sampling algorithm, what is the key property of each update? **A:** Sample each variable conditioned on all others (full conditional); accepted with probability 1
340. **Q:** In the EM algorithm (Expectation-Maximization), what is the E-step? **A:** Compute expected value of latent variables given current parameters and observed data
341. **Q:** In the k-fold cross-validation, what is the typical value of k? **A:** 5 or 10 (balance between bias and variance of the estimate)
342. **Q:** In the stratified cross-validation, what is the key difference from standard CV? **A:** Each fold preserves class proportions (useful for imbalanced datasets)
343. **Q:** In the algorithm for finding strongly connected components (Tarjan's algorithm), what data structure tracks ancestors? **A:** Stack of vertices currently in the DFS path; lowlink values track ancestor reachability
344. **Q:** In the algorithm for finding strongly connected components (Kosaraju's algorithm), how many DFS passes are needed? **A:** Two passes: first on G (record finish times), second on G^T (decreasing finish time)
345. **Q:** In the depth-first search, what is the asymptotic running time? **A:** $O(V+E)$ — each vertex and edge processed once
346. **Q:** In the breadth-first search, what is the key invariant regarding distances? **A:** Queue d-values differ by at most 1 (processing by layers)
347. **Q:** In the topological sort, what is the maximum number of valid orderings for a DAG? **A:** Depends on structure; can be exponential in worst case (e.g., independent vertices: $n!$ orderings)
348. **Q:** In the algorithm for finding the k-th smallest element in a BST, what attribute is needed? **A:** Subtree size (augmentation): $size[x] = size[left[x]] + size[right[x]] + 1$
349. **Q:** In the Van Emde Boas tree, what is the recursion depth for an operation? **A:** $O(\lg \lg u)$ — the universe size u is repeatedly square-rooted
350. **Q:** In the union-find data structure with union by size, what is the size of the larger tree after k merges? **A:** At least 2^k (doubling argument ensures height $\leq \lg n$)

351. **Q:** What is the time complexity of constructing a suffix array using the prefix-doubling method? **A:** $O(n \cdot \lg n)$ (sort by increasing length prefixes: 1, 2, 4, 8, ...)
352. **Q:** In the Lempel-Ziv (LZ77) compression, what does each output token consist of? **A:** (distance, length, next character) — pointer to earlier occurrence + next new character
353. **Q:** What is the time complexity of building a suffix tree using Ukkonen's algorithm? **A:** $O(n)$ (online algorithm with suffix links and active point)
354. **Q:** In the MD5 hash function, what is the output length? **A:** 128 bits (4 32-bit words, broken collision resistance)
355. **Q:** In the SHA-1 hash function, what is the output length? **A:** 160 bits (5 32-bit words; collision resistance broken in 2017)
356. **Q:** In SHA-256, how many rounds of compression are there per 512-bit block? **A:** 64 rounds (SHA-512: 80 rounds)
357. **Q:** In the Merkle-Damgård construction, how is padding handled? **A:** Append '1' bit, then '0' bits until $\text{length} \equiv 448 \pmod{512}$ (or $896 \pmod{1024}$ for SHA-512), then append 64-bit length
358. **Q:** In the Davies-Meyer construction for block cipher-based hash, what is the formula? **A:** $H_i = E_{\{M_i\}}(H_{i-1}) \oplus H_{i-1}$ (encrypt state with message block, XOR with previous state)
359. **Q:** In the HMAC construction, what is the formula? **A:** $\text{HMAC}(K, M) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel M))$
360. **Q:** In the RSA-OAEP padding scheme, what problem does it prevent? **A:** Chosen-ciphertext attacks; adds randomness via Feistel network with hash functions
361. **Q:** In the Diffie-Hellman key exchange, what is the shared secret computation? **A:** Alice sends g^a ; Bob sends g^b ; shared secret = g^{ab} (both compute: $(g^b)^a = (g^a)^b = g^{ab}$)
362. **Q:** In the ElGamal encryption scheme, what is the ciphertext size relative to plaintext? **A:** 2:1 expansion (ciphertext = $(g^k, m \cdot h^k)$ where $h = g^x$ is public key)
363. **Q:** In the Digital Signature Algorithm (DSA), what two values form the signature? **A:** (r, s) where $r = (g^k \pmod p) \pmod q$, $s = k^{-1} \cdot (H(m) + x \cdot r) \pmod q$
364. **Q:** In the elliptic curve cryptography (ECC), what is the key advantage over RSA? **A:** Smaller key sizes for equivalent security (256-bit ECC $\approx$ 3072-bit RSA $\approx$ 128-bit symmetric)

365. **Q:** In the year 2038 problem, what does the signed 32-bit Unix timestamp overflow to? **A:** $2^{31} - 1 = 2147483647$ seconds after Jan 1 1970 = Jan 19 2038; overflows to Dec 13 1901
366. **Q:** In the IEEE 754 floating-point standard, how many bits for the exponent of double precision? **A:** 11 bits (exponent bias = 1023; 52-bit mantissa; total 64 bits)
367. **Q:** In the Fisher-Yates shuffle, what is the time complexity? **A:** $O(n)$; swap each element from end with random earlier element (or from beginning forward)
368. **Q:** In the reservoir sampling algorithm for selecting k random elements from a stream, what is the time per element? **A:** $O(1)$ per element; replace i -th element ($i > k$) with probability k/i
369. **Q:** In the algorithm for finding the majority element (Boyer-Moore), what is the time and space? **A:** $O(n)$ time, $O(1)$ space; cancel pairs of different elements, remainder is candidate
370. **Q:** In the algorithm for compute prefix sums (scan), what is the time complexity? **A:** $O(n)$ sequential, $O(\lg n)$ parallel with tree reduction; $\text{output}[i] = \sum_{j=1}^i \text{input}[j]$

Short Answer

- Q:** State the five red-black tree properties. **Rubric:** (1) Each node red/black; (2) Root black; (3) Leaves (NIL) black; (4) Red's children black; (5) Equal black count on all paths.
- Q:** Show that any comparison sort requires $\Omega(n \lg n)$ comparisons. **Rubric:** Decision tree model: $n! \leq \text{leaves} \leq 2^h \rightarrow h \geq \lg(n!) = \Omega(n \lg n)$ by Stirling.
- Q:** Why is counting sort able to beat the $\Omega(n \lg n)$ lower bound? **Rubric:** It is not a comparison sort; uses array indices, not comparisons.
- Q:** State the four steps for augmenting a data structure. **Rubric:** (1) Choose underlying DS; (2) Determine additional info; (3) Verify maintainability; (4) Develop new operations.
- Q:** Explain the difference between Lomuto and Hoare partitioning. **Rubric:** Lomuto: $A[r]$ pivot, single pass, returns pivot index. Hoare: $A[p]$ pivot, two pointers from ends, returns j with $p \leq j < r$. Hoare more efficient for equal elements.
- Q:** Why does deletion in open-addressed hash tables require special handling? **Rubric:** Emptying slot breaks probe sequences for keys that probed through it. Need DELETED marker or relocation algorithm.
- Q:** State the interval trichotomy. **Rubric:** For any two intervals: they overlap, one is left of the other, or one is right of the other.

8. **Q:** Compare Bellman-Ford and Dijkstra: when would you use each?
Rubric: Bellman-Ford handles negative edges, detects cycles, $O(VE)$. Dijkstra requires nonnegative, $O(E \lg V)$. Dijkstra faster when applicable.
9. **Q:** What is the purpose of the super-source in Johnson's algorithm?
Rubric: Ensure all vertices reachable for Bellman-Ford to compute $h(v)$. Reweighting $\hat{w}(u,v) = w(u,v) + h(u) - h(v)$ makes all edges nonnegative.
10. **Q:** Define work, span, and parallelism in parallel algorithms. **Rubric:** T_1 = total operations; T_∞ = critical path length; parallelism = T_1/T_∞ . Brent: $T_P \leq T_1/P + T_\infty$.
11. **Q:** What is the competitive ratio of an online algorithm? **Rubric:** $C_{\text{ONLINE}} \leq \rho \cdot C_{\text{OPT}}$ (or $+\alpha$). ρ = competitive ratio. E.g., LRU is k -competitive.
12. **Q:** State the Master Theorem cases. **Rubric:** For $T(n) = aT(n/b) + f(n)$:
(1) $f = O(n^{\log_b a - \epsilon}) \rightarrow \Theta(n^{\log_b a})$; (2) $f = \Theta(n^{\log_b a} \lg^k n) \rightarrow \Theta(n^{\log_b a} \lg^{k+1} n)$; (3) $f = \Omega(n^{\log_b a + \epsilon})$ AND $af(n/b) \leq cf(n) \rightarrow \Theta(f(n))$.
13. **Q:** Explain the difference between DP and divide-and-conquer. **Rubric:** D&C partitions into disjoint subproblems (merge sort). DP has overlapping subproblems (Fibonacci). DP uses memoization or bottom-up table; D&C uses recursion only.
14. **Q:** Describe the cut property of MSTs. **Rubric:** For any cut $(S, V-S)$ that respects A (no crossing edges in A), the lightest edge crossing the cut is safe for A . Proof via cut-and-paste: assume heavier edge in MST, swap $\rightarrow$ contradiction.
15. **Q:** How does the Edmonds-Karp algorithm differ from Ford-Fulkerson?
Rubric: Ford-Fulkerson picks any augmenting path; Edmonds-Karp always uses BFS (shortest path in terms of edges). Result: $O(VE^2)$ polynomial bound vs $O(E \cdot |f^*|)$ which can be exponential.
16. **Q:** Explain the three methods of amortized analysis. **Rubric:** Aggregate (same amortized cost for all ops, sum/n), Accounting (assign different costs, credit must stay nonnegative), Potential ($\Phi(D_0)=0$, $\Phi(D_i) \geq 0$, $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$).
17. **Q:** Why does SELECT use groups of 5 rather than 3? **Rubric:** Groups of 3 give $T(n) \leq T(n/3) + T(2n/3) + \Theta(n) = O(n \lg n)$ — not linear. Groups of 5 guarantee $\geq 3n/10$ elements $\leq$ pivot $\rightarrow$ recursion on $\leq 7n/10 \rightarrow T(n) = \Theta(n)$.
18. **Q:** What is the significance of the regularity condition in Master Theorem Case 3? **Rubric:** $af(n/b) \leq cf(n)$ for some $c < 1$ and large n ensures that work does not grow as we descend. Without it, case 3 fails (e.g., $f(n) = n \cdot (2 - \sin n)$ could be larger at subtrees than at root).

19. **Q:** Compare adjacency lists vs adjacency matrices for graph representation. **Rubric:** Lists: $O(V+E)$ space, $O(\deg(v))$ edge enumeration, $O(V)$ edge query. Matrix: $O(V^2)$ space, $O(1)$ edge query, $O(V)$ enumeration. Lists better for sparse graphs; matrix for dense.
20. **Q:** Explain the difference between weak and strong duality in linear programming. **Rubric:** Weak duality: $c^T x \leq b^T y$ for any feasible primal (max) and dual (min). Strong duality: at optimality, $c^T x^* = b^T y^*$. Complementary slackness relates optimal primal/dual variables.
21. **Q:** What is the purpose of the π function in KMP? **Rubric:** $\pi[q]$ = longest proper prefix of P that is suffix of $P[1:q]$. On mismatch at $q+1$, set $q = \pi[q]$ to skip characters that are already known to match, avoiding backtracking in T .
22. **Q:** Describe the structure of a B-tree node. **Rubric:** Each node contains n keys ($n+1$ children pointers). Leaf: all children are NIL. Root: $1 \leq n \leq 2t-1$. Internal: $t-1 \leq n \leq 2t-1$. All leaves at same depth.
23. **Q:** Prove that the running time of BUILD-MAX-HEAP is $O(n)$. **Rubric:** $\sum_{h=0}^{\lceil \lg n \rceil} [n/2^{h+1}] \cdot O(h) = O(n \cdot \sum h/2^h) = O(n)$. Key: at height h , $\leq n/2^{h+1}$ nodes, MAX-HEAPIFY $O(h)$.
24. **Q:** Explain the difference between Prim's and Kruskal's MST algorithms. **Rubric:** Prim: vertex-based, grows single tree from root, uses priority queue, $O(E \lg V)$ or $O(V^2)$. Kruskal: edge-based, builds forest, sorts edges, uses union-find, $O(E \lg V)$. Both based on cut property.
25. **Q:** Describe how to find strongly connected components using Kosaraju's algorithm. **Rubric:** (1) DFS on G , record finish times. (2) Compute G^T (reverse edges). (3) DFS on G^T in decreasing finish time. Each DFS tree in step 3 = one SCC. $O(V+E)$. Correctness: component graph of G^T is reverse of G 's; finish order ensures correct partition.
26. **Q:** Explain polynomial-time reduction and show how to prove a problem is NP-complete. **Rubric:** Reduction $f: x \in L_1 \Leftrightarrow f(x) \in L_2$ in polynomial time. To prove L NP-complete: (1) show $L \in \text{NP}$ (certificate + verifier); (2) reduce known NPC $L' \leq_p L$ (show L is NP-hard). Reductions are transitive.
27. **Q:** Compare and contrast aggregate, accounting, and potential methods of amortized analysis. **Rubric:** Aggregate: $T(n)/n$, same for all ops. Accounting: assign different costs, maintain nonnegative credit. Potential: $\Phi(D_i)$, $\hat{c}_i = c_i + \Delta\Phi$, maintain $\Phi(D_i) \geq \Phi(D_0)$. All give same amortized costs but different perspectives.
28. **Q:** Explain how KMP string matching achieves $O(n+m)$ time. **Rubric:** Computes prefix function π (failure function) in $O(m)$. When mismatch occurs, uses π to skip characters already known to match — never backtracks in T . Each character in T compared at most once.

29. **Q:** Describe the secretary problem (online hiring) and its solution. **Rubric:** Interview n candidates sequentially, must hire/reject on the spot. Strategy: reject first k candidates (record best), then hire first candidate better than record. Optimal $k = n/e$, success probability $= 1/e \approx 37\%$.
30. **Q:** Why does the FFT reduce polynomial multiplication from $O(n^2)$ to $O(n \lg n)$? **Rubric:** Evaluate polynomials at roots of unity ($O(n \lg n)$ via FFT), pointwise multiply ($O(n)$), interpolate via inverse FFT ($O(n \lg n)$). Total: $O(n \lg n)$ vs $O(n^2)$ for coefficient multiplication.
31. **Q:** Compare the divide-and-conquer matrix multiplication with Strassen's algorithm. **Rubric:** Naive D&C: 8 recursive multiplications, $T(n) = 8T(n/2) + O(n^2) = \Theta(n^3)$. Strassen: 7 recursive multiplications ($S_1 \dots S_{10}$ matrices), $T(n) = 7T(n/2) + O(n^2) = \Theta(n^{\lg 7}) \approx O(n^{2.81})$. Strassen better for large n ; constant factors make naive better for small.
32. **Q:** Describe the structure and operations of a Fibonacci heap. **Rubric:** Collection of min-heap-ordered trees. INSERT $O(1)$, EXTRACT-MIN $O(\lg n)$ amortized, DECREASE-KEY $O(1)$ amortized (cascading cut). Key operations: consolidate (merge equal-degree trees during EXTRACT-MIN), cut and cascading cut during DECREASE-KEY.
33. **Q:** How does the Rabin-Karp string matching algorithm work? **Rubric:** Rolling hash: compute hash of pattern ($O(m)$), then sliding window hash over text ($O(n)$ total). When hashes match, verify directly ($O(m)$ worst-case per match). Expected $O(n+m)$, worst-case $\Theta(nm)$ (many spurious hits). Choose large prime modulus q to reduce collisions.
34. **Q:** Explain the concept of competitive analysis for online algorithms. **Rubric:** Compare online algorithm (no future knowledge) against optimal offline algorithm (full future knowledge). $C_ONLINE(I) \leq \rho \cdot C_OPT(I) + \alpha$ for all input sequences I . ρ = competitive ratio. Examples: LRU is k -competitive for paging; ski-rental deterministic $= 2 - 1/B$.
35. **Q:** Explain the computation of the LIS (Longest Increasing Subsequence) in $O(n \lg n)$. **Rubric:** Maintain array M where $M[k]$ = smallest ending value of an increasing subsequence of length k . For each element x , binary search for largest k with $M[k] < x$. Set $M[k+1] = x$. $O(n \lg n)$. Also maintain predecessor array for reconstruction. Example: $[10, 9, 2, 5, 3, 7, 101, 18] \rightarrow \text{LIS} = [2, 3, 7, 101]$ length 4.
36. **Q:** Describe the Graham-Brent scheduling theorem for parallel algorithms. **Rubric:** For any greedy scheduler, $T_P \leq T_1/P + T_\infty$. Proof: In each time step with more than P ready tasks, all P processors work. In steps with $\leq P$ ready tasks, the longest chain reduces. Total work done in first type $\leq T_1/P$; at most T_∞ steps of second type. Implication: linear speedup if parallelism $T_1/T_\infty \geq P$. Example: P-MERGE with $T_1 = \Theta(n)$, $T_\infty = \Theta(\lg^2 n)$ achieves linear speedup for $n/P \gg \lg^2 n$.

37. **Q:** Prove the cut property of minimum spanning trees. **Rubric:** Let $(S, V-S)$ be any cut that respects A (no crossing edges in A). Let e be lightest edge crossing the cut. Show e is safe for A . Proof: Suppose T is an MST containing A but not e . Adding e to T creates a cycle. Some other edge e' in the cycle crosses the cut. Since e is lightest, $w(e) \leq w(e')$. Replace e' with e to get $T' = T - e' + e$, which is also an MST containing A and e . So e is safe.
38. **Q:** Explain the amortized analysis of a dynamic array that doubles when full. **Rubric:** Aggregate: cost of i -th insert = 1 normally, i when $i-1$ is power of 2. Total = $n + \sum_{j=0}^{\lfloor \lg(n-1) \rfloor} 2^j = 3n = O(n)$. Accounting: charge \$3 per insert. Potential: $\Phi = 2 \cdot \text{num} - \text{size}$. When table full, $\Phi = \text{size}$, after doubling $\Phi = 0$, so $\Delta\Phi = -\text{size}$, $\hat{c} = (\text{size}+1) + (-\text{size}) = 1$.
39. **Q:** Describe the difference between Dijkstra's algorithm and Bellman-Ford. **Rubric:** Dijkstra: greedy, requires nonnegative weights, uses priority queue, $O(E \lg V)$. Correct because extracted vertices have final distances. Bellman-Ford: relaxes all edges $|V|-1$ times, handles negative weights, detects negative cycles, $O(VE)$. Use Dijkstra when weights nonnegative and faster time needed; use Bellman-Ford when negative edges present.
40. **Q:** Explain the Miller-Rabin primality test and its error probability. **Rubric:** Write $n-1 = 2^s \cdot t$ with t odd. Choose random $a \in [1, n-1]$. Compute $a^t \bmod n$. If $\equiv \pm 1$, PASS. Otherwise, repeatedly square $s-1$ times: if any becomes -1 , PASS. Otherwise n is composite. Error: for composite n , at most $1/4$ of a 's give false pass. After k independent tests, error $\leq 4^{-k}$.
41. **Q:** Describe how to solve the all-pairs shortest paths problem using Floyd-Warshall. **Rubric:** DP on intermediate vertices $\{1, \dots, k\}$. $d^{\{(k)\}}[i, j] = \min(d^{\{(k-1)\}}[i, j], d^{\{(k-1)\}}[i, k] + d^{\{(k-1)\}}[k, j])$. In-place: maintain single D matrix overwritten in each iteration. Initialize $D^0 =$ adjacency matrix (0 on diagonal, ∞ for non-edges). $\Theta(V^3)$ time, $\Theta(V^2)$ space. Can also compute predecessor matrix Π for path reconstruction.
42. **Q:** Explain the concept of universal hashing and why it avoids worst-case behavior. **Rubric:** Family H of hash functions where $\Pr[h(k_1)=h(k_2)] \leq 1/m$ for any distinct k_1, k_2 (over random choice of h). Key property: for ANY input, expected chain length $\leq 1 + \alpha$. Without randomization, adversary can force all n keys into same slot, making search $\Theta(n)$. With universal hashing, even worst-case input gives expected $O(1 + \alpha)$ performance.
43. **Q:** Show that the decision tree model gives a lower bound on sorting. **Rubric:** Any comparison sort can be modeled as a decision tree (each internal node = comparison, leaf = output permutation). $n!$ possible outputs $\rightarrow$ at least $n!$ leaves. Binary tree of height h has $\leq 2^h$ leaves. So $n! \leq 2^h \rightarrow h \geq \lg(n!) = \Omega(n \lg n)$ by Stirling. Therefore any comparison sort requires $\Omega(n \lg n)$ comparisons in worst case.

44. **Q:** Explain the structure of the proof that CIRCUIT-SAT is NP-complete. **Rubric:** (1) CIRCUIT-SAT $\in$ NP: certificate is satisfying assignment, verify by evaluating gates in $O(\text{gate count})$. (2) For any $L \in \text{NP}$, reduce $L \leq_p \text{CIRCUIT-SAT}$. Given a polynomial-time verifier M for L , construct a boolean circuit that simulates M 's computation on input x and certificate y . The circuit is satisfiable iff there exists y such that M accepts (x,y) . The reduction is polynomial because M runs in polynomial time $\rightarrow$ circuit size $O(T^2)$ where T is M 's running time.
45. **Q:** Compare the three methods for solving recurrences. **Rubric:** Substitution: guess + inductive proof. Works for any recurrence but requires good guess. Recursion-tree: draw tree, sum per level, total across levels. Good for generating guesses but imprecise. Master Theorem: plug and chug for $T(n)=aT(n/b)+f(n)$. Fast when applicable but has gaps between cases. Akra-Bazzi: generalizes Master for different subproblem sizes ($\sum a_i T(n/b_i) + f(n)$).
46. **Q:** Explain the Hopcroft-Karp bipartite matching algorithm. **Rubric:** Each phase: BFS from unmatched left vertices to find shortest augmenting path distances; then find maximal set of vertex-disjoint shortest augmenting paths via DFS on layered graph. Augment all paths simultaneously. Each phase increases shortest path length by at least 2. At most $O(\sqrt{V})$ phases. Total $O(\sqrt{V} \cdot E)$. Key invariant: path lengths strictly increase each phase.
47. **Q:** Describe the competitive analysis of random marking for paging. **Rubric:** Random Marking: on fault, if there are unmarked pages, evict a random unmarked one; if all marked, clear marks. Competitive ratio: $O(\lg k)$ where k = cache size. Lower bound $\Omega(\lg k)$ — no randomized paging algorithm beats this. The marking approach ensures that at least one page from each k requests in a phase remains in cache. Connect to coupon collector.
48. **Q:** Compare and contrast the role of the spawn and sync keywords in the fork-join parallel model. **Rubric:** spawn indicates the called function may execute in parallel with the caller. sync waits for all spawned children to complete. Nested parallelism: spawns within spawn $\rightarrow$ trees of tasks. Scheduling: greedy scheduler handles load balancing. Examples: P-MERGE spawns two merge calls; P-MATRIX-MULTIPLY spawns 8 recursive multiplications.
49. **Q:** Explain how to use the substitution method to solve recurrences. **Rubric:** (1) Guess the form of the solution (e.g., $T(n) = O(n \lg n)$). (2) Assume $T(k) \leq c \cdot k \cdot \lg k$ for $k < n$. (3) Substitute into recurrence, show $T(n) \leq c \cdot n \cdot \lg n$. (4) Base case: choose c large enough for small n . Key technique: subtract a lower-order term if induction gets stuck (e.g., guess $T(n) \leq c \cdot n \cdot \lg n$ fails, try $c_1 \cdot n \cdot \lg n - c_2 \cdot n$). Avoid: asymptotic notation in inductive hypothesis without explicit constants.
50. **Q:** Discuss the relationship between amortized analysis and worst-case analysis. **Rubric:** Worst-case analysis: bound on single operation, regardless of sequence (e.g., TABLE-INSERT worst-case $O(n)$).

Amortized analysis: bound on average cost over a sequence of operations (e.g., TABLE-INSERT amortized $O(1)$). Individual operations may be expensive, but sequence total is bounded. Potential method: Φ captures “prepaid” work. Application: without amortized analysis, dynamic arrays would appear $O(n)$ per insert, hiding the fact that inserts are $O(1)$ amortized.

51. **Q:** Explain the concepts of decision tree models and their use in proving lower bounds. **Rubric:** Decision tree: each internal node represents a comparison, leaves represent outputs. Height = number of comparisons in worst case. For sorting: $n!$ possible outputs $\rightarrow$ at least $n!$ leaves $\rightarrow$ height $\geq \lg(n!) = \Omega(n \lg n)$. For selection: decision tree height gives lower bound on comparisons for finding i -th smallest element. For finding min: $n-1$ lower bound via adversary argument.
52. **Q:** Describe the adversarial method for proving algorithm lower bounds. **Rubric:** Adversary answers comparisons adaptively to force worst-case path. Maintains invariant about possible orderings consistent with past answers. Example: finding minimum needs $n-1$ comparisons because each non-winner must lose at least once (adversary ensures each element loses at most once). For finding both min and max: $3\lceil n/2 \rceil - 2$ comparisons lower bound achieved by tournament method.
53. **Q:** Explain the concept of a competitive ratio for online algorithms, using ski-rental as an example. **Rubric:** Competitive ratio $\rho = \sup_I C_{\text{ONLINE}}(I)/C_{\text{OPT}}(I)$. Ski-rental: rent costs \$1/day, buy costs \$ B . If we know ski trip lasts D days, $\text{OPT} = \min(D, B)$. Deterministic strategy: rent $B-1$ days then buy $\rightarrow$ cost = $B-1 + \min(B, D)$. Worst-case when $D = B$: $C_{\text{ONLINE}} = 2B-1$, $C_{\text{OPT}} = B \rightarrow \rho = 2-1/B$. Randomized strategy can achieve $e/(e-1) \approx 1.582$.
54. **Q:** How does the Lenstra-Lenstra-Lovász (LLL) lattice basis reduction algorithm relate to cryptography? **Rubric:** LLL finds a short (not necessarily shortest) vector in a lattice in polynomial time. Applications: breaking knapsack-based cryptosystems (Merkle-Hellman), finding integer relations (PSLQ predecessor), solving subset-sum problems. Given a basis of n vectors in m -dimensional space, LLL produces a reduced basis where the first vector is within factor $2^{\{(n-1)/2\}}$ of the shortest. This factor is exponential, so LLL has limited applications for high dimensions.
55. **Q:** Describe the Chinese Remainder Theorem and its application to RSA. **Rubric:** CRT: given pairwise coprime $n_1, \dots, n_k$ and remainders a_i , there exists unique x modulo $N = \prod n_i$ with $x \equiv a_i \pmod{n_i}$. For RSA: compute $m^d \pmod{N}$ where $N = pq$. Instead of computing mod N directly, compute $m^d \pmod{p}$ and $m^d \pmod{q}$ separately, then use CRT to combine. This is $\sim 4\times$ faster. Specifically, compute $m_p = m \pmod{p}$, $d_p = d \pmod{p-1}$, $m_q = m \pmod{q}$, $d_q = d \pmod{q-1}$. Result = $m_p^{d_p} \pmod{p}$ and $m_q^{d_q} \pmod{q}$ combined via Garner’s formula.

56. **Q:** Explain the relationship between maximum bipartite matching and maximum flow. **Rubric:** Construct flow network: source $\rightarrow$ each left vertex (capacity 1), each left vertex $\rightarrow$ connected right vertices (capacity 1), each right vertex $\rightarrow$ sink (capacity 1). Maximum flow = maximum cardinality matching. Capacity integrality ensures integer flow $\rightarrow$ integer matching. Runtime: Ford-Fulkerson gives $O(VE)$ (since $|f^*| \leq V/2$, each augmenting path increases flow by 1). Hopcroft-Karp improves to $O(\sqrt{V} \cdot E)$ using layered graph and maximal set of vertex-disjoint augmenting paths.
57. **Q:** Describe the concept of a matroid and its relationship to greedy algorithms. **Rubric:** Matroid $M = (S, I)$ where S is finite set, I is collection of independent subsets, with heredity (subsets of independent sets are independent) and augmentation (if $|A| < |B|$ with A, B independent, $\exists x \in B$ with $A \cup \{x\}$ independent). Greedy algorithm (sort by weight, add if independent) finds maximum-weight independent set in any matroid. Examples: graphic matroid ($F \subseteq E$ forests), uniform matroid ($|X| \leq k$). Matroid-intersection problems (find max common independent set) are solvable in polynomial time.
58. **Q:** Compare and contrast the three cases of the Master Theorem with examples. **Rubric:** $T(n) = aT(n/b) + f(n)$. Compare $f(n)$ to $n^{\log_b a}$. Case 1 (f polynomially smaller): $T(n) = \Theta(n^{\log_b a})$. Example: $T(n) = 9T(n/3) + n \rightarrow n^{\log_3 9} = n^2 \rightarrow \Theta(n^2)$. Case 2 ($f = \Theta(n^{\log_b a} \lg^k n)$): $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$. Example: $T(n) = 2T(n/2) + n \lg n \rightarrow \Theta(n \lg^2 n)$. Case 3 (f polynomially larger with regularity): $T(n) = \Theta(f(n))$. Example: $T(n) = 3T(n/4) + n \lg n \rightarrow n \lg n = \Omega(n^{\log_4 3})$ and $3 \cdot (n/4) \lg(n/4) \leq c \cdot n \lg n$ for $c = 3/4 \rightarrow \Theta(n \lg n)$.
59. **Q:** Explain the concept of the residual network in the context of maximum flow. **Rubric:** Residual network $G_f = (V, E_f)$ where E_f contains forward edges (u, v) with residual capacity $c_f(u, v) = c(u, v) - f(u, v)$ and backward edges (v, u) with capacity $c_f(v, u) = f(u, v)$. Forward edges represent unused capacity; backward edges represent flow that can be reversed. Key property: if augmenting path exists in G_f , flow is not maximum. Lemma: $|f| + |f'| = |f + f'|$ in residual network. Augmenting along path increases total flow by bottleneck capacity.
60. **Q:** Describe the structure and properties of a van Emde Boas tree. **Rubric:** vEB tree stores integers in $\{0, \dots, u-1\}$ where $u = 2^{\{2^k\}}$. Contains: min (minimum element, stored separately), max, summary (vEB over high bits), clusters (array of vEB over low bits). Operations: INSERT/DELETE/MEMBER $O(\lg \lg u)$. SUCCESSOR/PREDECESSOR $O(\lg \lg u)$. MINIMUM/MAXIMUM $O(1)$. Key insight: recursive decomposition reduces universe size from u to $\sqrt{u}$, giving double-logarithmic time. Space $O(u)$ for basic version; can be reduced to $O(n)$ with hash tables for clusters.
61. **Q:** Explain the concept of linear programming duality and the relationship between primal and dual problems. **Rubric:** Given primal $(\min c^T x \text{ s.t. } Ax \geq b, x \geq 0)$, dual is $(\max b^T y \text{ s.t. } A^T y \leq c, y \geq 0)$. Weak duality: $c^T x \geq b^T y$ for any feasible pair. Strong duality: at

optimality $c^T x^* = b^T y$. Complementary slackness: for optimal x, y , we have $x_j(c_j - \sum_i a_{ij} y_i) = 0$ and $y_i(\sum_j a_{ij} x_j - b_i) = 0$. Economic interpretation: dual variables are shadow prices. Sensitivity analysis: dual variable measures rate of change of objective with constraint relaxation.

62. **Q:** Describe the AKS primality test and its significance. **Rubric:** AKS (Agrawal-Kayal-Saxena, 2002): first deterministic polynomial-time primality test. Uses identity $(x-a)^n \equiv x^n - a \pmod{n}$ iff n is prime. Algorithm: check $n = a^b$ (perfect power), then verify $(x+1)^n \equiv x^n + 1 \pmod{n, x^{r-1}}$ for carefully chosen $r = O(\lg^6 n)$. Runtime $O(\lg^{12} n)$ originally, improved to $O(\lg^6 n)$. Significance: primality is in P unconditionally (not relying on unproven conjectures like GRH for Miller's test). Practical: still slower than Miller-Rabin; used for theoretical completeness.
63. **Q:** Explain the exponential-time algorithm for SUBSET-SUM and its connection to dynamic programming. **Rubric:** SUBSET-SUM: given multiset S of n integers and target t , is there subset summing to t ? Pseudo-polynomial DP: let $P[i, s] = 1$ if subset of first i elements sums to s . $O(n \cdot t)$. This is not polynomial because input size is $\lg(t)$, not t . Meet-in-the-middle: split S into two halves of $n/2$ elements. Compute all $2^{n/2}$ subset sums for each half, sort one list, for each sum in first list binary-search for t -sum in second. $O(2^{n/2} \cdot \log(2^{n/2})) = O(2^{n/2} \cdot n)$. Much better than $O(2^n)$.
64. **Q:** Explain the concept of an approximation-preserving reduction (AP-reduction). **Rubric:** AP-reduction: given instance x of problem A and error ε , produces instance x' of problem B such that any ε' -approximation to B gives an ε -approximation to A . This preserves approximability: if B has PTAS, so does A . For APX-completeness: problems where a PTAS would collapse APX to PTAS, implying $P=NP$. E.g., MAX-3-SAT is APX-complete. Distinguishes problems with PTAS (knapsack, Euclidean TSP) from those without (MAX-3-SAT, set cover, general TSP).
65. **Q:** Describe the three methods for amortized analysis and compare them using the example of a dynamic table. **Rubric:** Aggregate: analyze total cost $T(n)$ of n operations, amortized cost $= T(n)/n = O(1)$ for dynamic table. Accounting: charge \$3 per insertion (\$1 for current insert, \$1 for when table doubles and element moves, \$1 for when table doubles and paired element moves). Sum of charges $\sum \$3 = \$3n$, credit always nonnegative. Potential: $\Phi(T) = 2 \cdot \text{num} - \text{size}$. For insert when not full: $\Delta\Phi = 2$, $\hat{c} = 2$. For insert when full: $\Phi_{\text{before}} = 2m - m = m$, $\Phi_{\text{after}} = 2(m+1) - 2m = 2$, $\Delta\Phi = 2 - 2m$, $\hat{c} = (m+1) + (2 - 2m) = 3 - m + 1 \dots$ actually $\hat{c} = 1 + (2(\text{num}+1) - 2\text{size}) - (2\text{num} - \text{size}) = 1 + 2 = 3$ for non-doubling; for doubling: $\hat{c} = (m+1) + (2(m+1) - 2m) - (2m - m) = m+1 + 2 - 2m - m = 3$. Amortized $O(1)$.
66. **Q:** Explain the relationship between Strassen's matrix multiplication and the computational complexity of matrix multiplication in general. **Rubric:** Standard 3-loop multiplication: $O(n^3)$. Strassen: $O(n^{\{2.81\}})$

by reducing 8 recursive multiplications to 7. The exponent ω (matrix multiplication exponent) is defined as infimum such that multiplication is $O(n^{\omega+\epsilon})$. Currently $\omega < 2.37287$ (Alman-Williams, 2024). Lower bound: $\omega \geq 2$ (since output has $O(n^2)$ entries). Conjecture: $\omega = 2$ (there exists $O(n^{2+\epsilon})$ algorithm). Practical: Strassen for $n > 1000$; group-theoretic algorithms (Coppersmith-Winograd family) have large constants and are not practical for any n .

67. **Q:** Describe how to prove that a problem is NP-complete using a reduction from 3-CNF-SAT to CLIQUE. **Rubric:** Given 3-CNF formula $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ with variables $x_1, \dots, x_n$. Construct graph G : for each clause $C_i = (l_1 \vee l_2 \vee l_3)$, create a triangle of three vertices, one per literal. Add edges between vertices in different triangles unless they represent complementary literals (x_i and $\neg x_i$). Claim: φ is satisfiable iff G has a clique of size k . ($\Rightarrow$) satisfying assignment picks at least one literal per clause $\rightarrow$ those k vertices form a clique (no complementary literals). ($\Leftarrow$) k -clique must have one vertex per clause; set those literals true $\rightarrow$ consistent since no complementary literals in clique.
68. **Q:** Explain the concept of a Bloom filter and analyze its false positive rate. **Rubric:** Bloom filter: a k -hash probabilistic data structure for representing a set. Array of m bits, initially all 0. Insert: hash element with k independent hash functions, set hash positions to 1. Query: check if all k hash positions are 1 — if any is 0, element definitely not in set; if all are 1, element may be in set (false positive possible). False positive rate: $(1 - (1-1/m)^{\{kn\}})^k \approx (1 - e^{-kn/m})^k$. Optimal $k = (m/n) \cdot \ln 2 \approx 0.693 \cdot (m/n)$. No false negatives. Applications: spellcheckers, network routers, databases. Can't delete (use counting Bloom filter instead).
69. **Q:** Compare the approach to solving recurrences using substitution, recursion-tree, and the Master Theorem. **Rubric:** Recursion-tree: draw tree, compute cost per level. Good for developing guess (e.g., $T(n) = T(n/3) + T(2n/3) + \Theta(n) \rightarrow$ root $\Theta(n)$, each level $\Theta(n)$, depth $\log_{3/2} n \rightarrow$ guess $\Theta(n \lg n)$). Substitution: verify guess by induction (e.g., $T(n) = T(n/3) + T(2n/3) + \Theta(n) \leq c \cdot n \cdot \lg n \rightarrow$ substitute, choose c large enough). Master Theorem: direct formula for $T(n) = aT(n/b) + f(n)$. When Master doesn't apply (e.g., $T(n) = T(n-1) + n$), use substitution or iteration. Akra-Bazzi generalizes Master for different subproblem sizes.
70. **Q:** Explain the structure of a suffix tree and its applications to string problems. **Rubric:** Suffix tree of string S : compressed trie of all suffixes of S . Each node stores substring label. Primitive applications: substring search $O(m)$ after $O(n)$ preprocessing. Compute longest common substring (LCS) of two strings: build suffix tree of $S_1 \# S_2$, traverse to find deepest node with both markers. Longest repeated substring: find deepest internal node. String matching with wildcards: break into substrings separated by wildcards. Suffix array: space-efficient alternative. LCP array enables many applications (pattern matching, k -common substring).
71. **Q:** Describe the concept of a pseudorandom generator (PRG) and its connection to computational indistinguishability. **Rubric:** PRG:

deterministic function $G: \{0,1\}^s \rightarrow \{0,1\}^n$ with $n > s$ such that $G(U_s) \approx_c U_n$ (output is computationally indistinguishable from uniform). Distinguisher D : $\Pr[D(G(U_s))=1] - \Pr[D(U_n)=1] \leq \epsilon$ for all PPT D . PRGs exist $\Leftrightarrow$ one-way functions exist. Yao's theorem: next-bit unpredictability characterizes PRGs. Application: can replace random bits in any efficient algorithm while preserving correctness. This justifies using PRNGs in randomized algorithms like Miller-Rabin.

72. **Q:** Explain how to use DP to compute edit distance (Levenshtein distance) between two strings. **Rubric:** Edit distance = minimum number of insertions, deletions, and substitutions to convert string $A[1..m]$ to $B[1..n]$. DP recurrence: $D[i,j] = \min(D[i-1,j] + 1 \text{ (delete } A[i]), D[i,j-1] + 1 \text{ (insert } B[j]), D[i-1,j-1] + (A[i]=B[j]?0:1) \text{ (substitute)})$. Base: $D[0,j]=j$, $D[i,0]=i$. $O(mn)$ time, $O(mn)$ space. Space optimization to $O(\min(m,n))$ retaining only two rows — but then cannot reconstruct alignment. Needleman-Wunsch is the global alignment variant (affine gap penalties). Applications: spell checking, DNA sequence alignment, plagiarism detection.
73. **Q:** Describe the operation of Dinic's algorithm for maximum flow. **Rubric:** (1) From residual network, compute level graph using BFS (distance from s). (2) Find blocking flow using DFS on level graph — push flow from s to t along edges in level graph, never revisit dead-end paths. (3) Augment flow by blocking flow, repeat. Each phase increases distance from s to t in level graph. At most $O(V)$ phases. Finding blocking flow: $O(VE)$ per phase (if naive) or $O(E)$ with dynamic trees. Total $O(V^2E)$ worst-case, $O(E\sqrt{V})$ for unit capacities. Faster than Edmonds-Karp $O(VE^2)$ for dense graphs.
74. **Q:** Compare and contrast the Bellman-Ford and Floyd-Warshall algorithms. **Rubric:** Bellman-Ford: single-source, negative edges OK, $O(VE)$. Detects negative cycles reachable from source. Floyd-Warshall: all-pairs, negative edges OK, $\Theta(V^3)$. Detects any negative cycle (negative diagonal after V iterations). Both relax edges, but differently: Bellman-Ford relaxes all edges $|V|-1$ times; Floyd-Warshall considers intermediate vertices incrementally. Floyd-Warshall uses DP: $D^{\{(k)\}}[i,j] = \min(D^{\{(k-1)\}}[i,j], D^{\{(k-1)\}}[i,k] + D^{\{(k-1)\}}[k,j])$.
75. **Q:** Explain how the k-means clustering algorithm works and its convergence properties. **Rubric:** (1) Initialize k centroids randomly. (2) Assign each point to nearest centroid. (3) Recompute centroids as means of assigned points. (4) Repeat until assignments don't change. Convergence: k-means always terminates because distortion (sum of squared distances to centroids) strictly decreases each iteration (or stays same). However, it converges to a local minimum, not global optimum (NP-hard). Random restarts or k-means++ initialization (spread-out initial centroids) improves quality. Runtime $O(n \cdot k \cdot t \cdot d)$ where t = number of iterations, d = dimensionality. Elbow method for choosing k .
76. **Q:** Describe the structure of the proof that compares 3-CNF-SAT and CLIQUE to show CLIQUE is NP-complete. **Rubric:** (1) CLIQUE $\in$ NP:

certificate is set of k vertices; verify all edges exist between them in $O(k^2)$ time. (2) Reduce 3-CNF-SAT $\leq_p$ CLIQUE: Given ϕ with k clauses, each a triple of literals: create $3k$ vertices (one per literal occurrence). Edges: connect vertices in different clauses unless literals are complementary (x and $\neg x$). ϕ satisfiable $\Leftrightarrow$ graph has k -clique. ($\Rightarrow$) satisfying assignment picks one true literal per clause $\rightarrow k$ vertices with no complementary literals $\rightarrow$ form clique. ($\Leftarrow$) k -clique has exactly one vertex per clause, assign literals consistently $\rightarrow$ satisfying assignment.

77. **Q:** Explain how the Dijkstra algorithm handles nonnegative edge weights and why it fails with negative edges. **Rubric:** Intuition: Dijkstra extracts vertex with minimum tentative distance, and since all remaining edges are nonnegative, no shorter path to that vertex can be found later. Formal proof: when u is extracted, $u.d = \delta(s,u)$ because any alternative path to u must go through a vertex v still in Q with $v.d \geq u.d$, and edge (v,u) has nonnegative weight. With negative edges: extracted vertex u could later be reached via a negative-weight edge from v (still in Q), contradicting optimality. Example: $s \rightarrow a(10)$, $s \rightarrow b(1)$, $b \rightarrow a(-9)$. Dijkstra extracts $b.d=1$, then $a.d=10$, but true shortest $s \rightarrow a$ is $s \rightarrow b \rightarrow a = 1 + (-9) = -8$. Bellman-Ford handles this.
78. **Q:** Describe the process of building a Huffman code tree from character frequencies. **Rubric:** (1) Create leaf node per character with frequency as weight. (2) Build min-heap of nodes. (3) While >1 node: extract two smallest, create parent with sum frequency, set extracted as left/right children, insert parent into heap. (4) Root = final remaining node. (5) Assign 0 to left edges, 1 to right edges. (6) Code for character = path from root to leaf. Optimality: lowest-frequency characters get deepest leaves.
79. **Q:** Compare the simplex method and interior-point methods for linear programming. **Rubric:** Simplex: walks along edges of feasible polytope (vertices), exponential worst-case but practical $O(m \cdot \text{operations})$. Interior-point: follows central path through interior of feasible region, polynomial $O(n^3 \cdot L)$. Simplex better for small-med problems; interior-point for large-scale. Karmarkar's algorithm pioneered interior-point methods. Most modern solvers use both (crossover from interior-point to simplex for warm-start).
80. **Q:** Explain the relationship between NP-complete problems, co-NP, and NP-hard. **Rubric:** NP: verifiable in polynomial time. co-NP: complement in NP (e.g., TAUTOLOGY is co-NP-complete). $L \in \text{co-NP}$ iff $\bar{L} \in \text{NP}$. $\text{NPC} \cap \text{co-NP} \neq \emptyset$ would imply $\text{NP} = \text{co-NP}$. NP-hard: every NP problem reduces to it (may not be in NP). Optimization problems (e.g., TSP-OPT) are NP-hard but not NP-complete. Factoring and Graph Isomorphism are believed NP-intermediate (in NP but not P nor NPC, unless $P=NP$).
81. **Q:** Describe the algorithm for finding the Most Significant Digit (MSD) in radix sort. **Rubric:** (1) Sort by most significant digit using stable bucket sort. (2) Recursively sort elements within each bucket by the next digit. (3) Concatenate sorted buckets. Unlike LSD (iterative by least significant), MSD is recursive. MSD can stop early when bucket

has only one element. Both use stable sort per digit. LSD requires same number of digits for all elements; MSD works with variable-length strings.

82. **Q:** Explain the concept of a random-access machine (RAM) model and its assumptions. **Rubric:** RAM model: (1) Each simple operation (+, -, *, /, =, if) takes constant time. (2) Loops and subroutines are sum of times of their operations. (3) Memory access is constant time. (4) Each datum fits in one machine word. No assumptions about pipelining, cache, or parallelism. This model works well for most algorithm analysis because constants are dominated by asymptotic growth. Limitations: doesn't account for cache effects, memory hierarchy, or vectorization.
83. **Q:** Compare and contrast the counting of operations for insertion sort and merge sort in practice. **Rubric:** Insertion sort: $\Theta(n^2)$ worst-case, but $\Theta(n)$ best-case (nearly sorted). Lower constants, good for small n (< 50 elements). Merge sort: $\Theta(n \lg n)$ worst-case, but requires $\Theta(n)$ extra space. Higher constants due to function calls and merge overhead. In practice: insertion sort beats merge sort for n up to ~ 20 -50 in typical implementations. Many library sorts (Timsort, Introsort) use this by applying insertion sort for small subarrays.
84. **Q:** Describe the characteristics of an approximation algorithm and when it is preferred over exact optimization. **Rubric:** Approximation algorithm finds solution within proven factor ρ of optimal in polynomial time. Preferred when: (1) Problem is NP-hard (TSP, Vertex Cover). (2) Exact exponential algorithm is infeasible. (3) Near-optimal solution is acceptable (e.g., delivery routing). Factor ρ can be constant (vertex cover 2), logarithmic (set cover $\ln n$), or polynomial (clique). FPTAS (e.g., knapsack) achieves $(1+\epsilon)$ factor for any $\epsilon > 0$ in time polynomial in n and $1/\epsilon$.
85. **Q:** Explain the importance of the Church-Turing thesis in the context of algorithm analysis. **Rubric:** Church-Turing thesis: any function computable by an effective procedure is computable by a Turing machine. Significance: the Turing machine defines the boundary of what is computable. In algorithm analysis, this means the RAM model is equivalent to the Turing machine up to polynomial factors. Therefore, the class P is well-defined independent of the computational model. Quantum computers violate the strong thesis but are still bounded by BQP. No hypercomputation (oracle machines, time travel) has been physically realized.
86. **Q:** Explain how to construct a suffix array from a string and compute the LCP array. **Rubric:** Suffix array $SA[1..n]$: sorted order of suffixes. Naive construction: sort suffixes $\rightarrow O(n^2 \lg n)$. Better: prefix-doubling (Manber-Myers): $O(n \lg n)$. Linear: SA-IS or DC3 (difference cover) $O(n)$. LCP array via Kasai's algorithm: $LCP[i] = \text{lcp between } SA[i] \text{ and } SA[i-1]$. For i from $1..n$, compute $rank[i] = \text{position of suffix } i \text{ in } SA$. For $i, j = rank[SA[i-1]]$, while $S[i+k] = S[j+k]$: $k++$. Set $LCP[rank[i]] = k$. Decrease k by 1 (invariant: $k \geq 0$ after decrement). Total $O(n)$.

Applications: pattern matching, longest repeated substring, longest common substring.

87. **Q:** Prove that the problem of determining whether a graph has a Hamiltonian cycle is NP-complete. **Rubric:** (1) HAM-CYCLE $\in$ NP: certificate = ordered list of vertices; verifier checks each edge exists in graph and vertices are distinct and includes all n vertices. $O(n)$ verification. (2) Reduce VERTEX-COVER $\leq_p$ HAM-CYCLE: construct graph with selector paths representing vertices, edge gadgets representing edges. A Hamiltonian cycle picks exactly one endpoint from each edge gadget selector path, corresponding to a vertex cover. Cycle length constraints ensure cover size = k . The gadget is polynomial size ($O(k \cdot |E|)$). Correctness: vertex cover of size $k \Leftrightarrow$ Hamiltonian cycle in constructed graph.

Trace / Apply

- Input:** $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, $n=10$. Apply BUILD-MAX-HEAP. **Expected:** $i=5$: no change; $i=4$: $[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$; $i=3$: $[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$; $i=2$: $[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$; $i=1$: $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$.
- Input:** Insert keys 41, 38, 31, 12, 19, 8 into empty RB tree. Show tree after each. **Expected:** Trace RB-INSERT-FIXUP cases at various steps; final valid RB tree.
- Input:** Graph $s \rightarrow a(3)$, $s \rightarrow b(5)$, $a \rightarrow b(2)$, $b \rightarrow c(1)$, $a \rightarrow c(6)$. Run Dijkstra from s . **Expected:** $s.d=0$, $a.d=3$, $b.d=5$, $c.d=6$.
- Input:** $T(n) = 3T(n/4) + n \lg n$. Apply Master Theorem. **Expected:** $a=3$, $b=4$, $n^{\{\log_4 3\}} = n^{\{0.793\}}$, $f(n) = n \lg n = \Omega(n^{\{0.793+\epsilon\}})$, check regularity $\rightarrow$ Case 3 $\rightarrow \Theta(n \lg n)$.
- Input:** Sequence of numbers for COUNTING-SORT: $A=[2, 5, 3, 0, 2, 3, 0, 3]$, $k=5$. **Expected:** C after counts $[2, 0, 2, 3, 0, 1]$; after running sums $[2, 2, 4, 7, 7, 8]$; final $B=[0, 0, 2, 2, 3, 3, 3, 5]$.
- Input:** Ford-Fulkerson on simple graph with capacities $s \rightarrow a(10)$, $s \rightarrow b(10)$, $a \rightarrow t(5)$, $a \rightarrow b(15)$, $b \rightarrow t(10)$. Find max flow. **Expected:** Max flow = 15 (saturate $a \rightarrow t=5$, $s \rightarrow b \rightarrow t=10$, $s \rightarrow a \rightarrow b=5$).
- Input:** Insertion sort on $A=[5, 2, 4, 6, 1, 3]$. Show array after each iteration. **Expected:** $i=2$: $[2, 5, 4, 6, 1, 3]$; $i=3$: $[2, 4, 5, 6, 1, 3]$; $i=4$: $[2, 4, 5, 6, 1, 3]$; $i=5$: $[1, 2, 4, 5, 6, 3]$; $i=6$: $[1, 2, 3, 4, 5, 6]$.
- Input:** $A = [3, 7, 2, 6, 5, 1, 4]$ with Lomuto PARTITION ($p=1$, $r=7$). Show regions after each step. **Expected:** Pivot=4. After $j=1$: $[3, 7, 2, 6, 5, 1, 4]$ $i=1$; $j=2$: $[3, 7, 2, 6, 5, 1, 4]$ $i=1$; $j=3$: $[3, 2, 7, 6, 5, 1, 4]$ $i=2$; $j=4$: $[3, 2, 7, 6, 5, 1, 4]$ $i=2$; $j=5$: $[3, 2, 7, 6, 5, 1, 4]$ $i=2$; $j=6$: $[3, 2, 1, 6, 5, 7, 4]$ $i=3$. Final swap: $[3, 2, 1, 4, 5, 7, 6]$. Return $q=4$.

9. **Input:** $T(n) = 2T(\lfloor n/2 \rfloor) + n \lg n$. Solve the recurrence. **Expected:** Master Theorem Case 2 with $k=1$: $a=2$, $b=2$, $n^{\lg 2} = n^1$, $f(n) = n \lg n = \Theta(n \lg n)$. Solution: $T(n) = \Theta(n \lg^2 n)$.
10. **Input:** AVL tree insertion: keys 10, 20, 30, 40, 50, 25. Show rotations. **Expected:** 10→20→30: left rotation at 10, root=20. 20(10)(30)→40: no rotation. 20(10)(30(·)(40))→50: left rotation at 30. 20(10)(40(30(50)))→25. 25 makes 40 unbalanced: RL rotation at 40. Final: 30(20(10(25)))(40(·)(50)).
11. **Input:** Run Floyd-Warshall on 4-vertex graph with adjacency matrix M where $M[i,j]$ = weight. Compute D^0, D^1, D^2, D^3, D^4 . **Expected:** D^0 = input; $D^1[i,j] = \min(D^0[i,j], D^0[i,1]+D^0[1,j])$; ... D^4 = all-pairs shortest paths.
12. **Input:** Kruskal's algorithm on graph with edges: (a,b,1), (b,c,2), (a,c,3), (c,d,4), (b,d,5). **Expected:** Sort: (a,b,1), (b,c,2), (a,c,3), (c,d,4), (b,d,5). Add (a,b), (b,c), skip (a,c) (cycle), add (c,d), skip (b,d) (cycle). MST weight = 7.
13. **Input:** Run Bellman-Ford on graph: $s \rightarrow a(4)$, $s \rightarrow b(5)$, $a \rightarrow b(-3)$, $b \rightarrow c(2)$, $c \rightarrow a(4)$. Source s . Detect negative cycle. **Expected:** After $|V|-1 = 3$ relaxations: $s.d=0$, $a.d=4$, $b.d=1$, $c.d=3$. Fourth pass: no changes → no negative cycle.
14. **Input:** Insert keys 5, 3, 7, 1, 4, 6, 8 into empty BST. Show the tree. **Expected:** Root 5. Left: 3(1,4). Right: 7(6,8). Inorder: 1,3,4,5,6,7,8.
15. **Input:** Run DFS on graph: $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 4$. Show discovery/finish times. **Expected:** Depends on adjacency order. One possible: 1(1,8), 2(2,5), 4(3,4), 3(6,7). Or: 1(1,8), 3(2,7), 4(3,4), 2(5,6).
16. **Input:** Apply Strassen's algorithm to multiply 2×2 matrices: $A=[1,3;7,5]$, $B=[6,8;4,2]$. Compute P matrices. **Expected:** $S_1=B_{12}-B_{22}=8-2=6$; $S_2=A_{11}+A_{12}=1+3=4$; $S_3=A_{21}+A_{22}=7+5=12$; $S_4=B_{21}-B_{11}=4-6=-2$; $S_5=A_{11}+A_{22}=1+5=6$; $S_6=B_{11}+B_{22}=6+2=8$; $S_7=A_{12}-A_{22}=3-5=-2$; $S_8=B_{21}+B_{22}=4+2=6$; $S_9=A_{11}-A_{21}=1-7=-6$; $S_{10}=B_{11}+B_{12}=6+8=14$. Then $P_1=6 \cdot 1=6$; etc.
17. **Input:** Show the result of applying MAX-HEAPIFY to $A=[3,14,10,8,7,9,3,2,4,1]$ at $i=1$. **Expected:** Compare 3 with children 14,10 → swap with 14 → $[14,3,10,8,7,9,3,2,4,1]$ → MAX-HEAPIFY at $i=2$ → compare 3 with 8,7 → swap with 8 → $[14,8,10,3,7,9,3,2,4,1]$ → compare 3 with 4 → swap → $[14,8,10,4,7,9,3,2,3,1]$. Done (children of 4 are 2 and 1).
18. **Input:** Apply PARTITION (Lomuto) to $A=[10,80,30,90,40,50,70]$, $p=1$, $r=7$. **Expected:** $x=70$. $j=1$: $10 \leq 70 \rightarrow i=1$, swap $A[1] \leftrightarrow A[1]$; $j=2$: $80 > 70$; $j=3$: $30 \leq 70 \rightarrow i=2$, swap $A[2] \leftrightarrow A[3] \rightarrow [10,30,80,90,40,50,70]$; $j=4$: $90 > 70$; $j=5$: $40 \leq 70 \rightarrow i=3$, swap $\rightarrow [10,30,40,90,80,50,70]$; $j=6$: $50 \leq 70 \rightarrow i=4$, swap $\rightarrow [10,30,40,50,80,90,70]$. Final swap $A[5] \leftrightarrow A[7] \rightarrow [10,30,40,50,70,90,80]$. Return $q=5$.

19. **Input:** Compute Euclidean algorithm for $\gcd(54, 24)$. **Expected:** $54 \bmod 24 = 6$; $24 \bmod 6 = 0 \rightarrow \gcd = 6$. EXTENDED-EUCLID: $(d, x, y) = (6, 1, -2)$ since $54 \cdot 1 + 24 \cdot (-2) = 6$.
20. **Input:** Compute Rabin-Karp hash for pattern "ABC" ($d=256, q=101$). Given $T="ABCABC"$, find matches. **Expected:** Hash of pattern = $(65 \cdot 256^2 + 66 \cdot 256 + 67) \bmod 101 = \text{compute}$. Sliding window: same hash at shifts 0 and 3 $\rightarrow$ verify matches.
21. **Input:** Run topological sort on DAG: edges $A \rightarrow B, A \rightarrow C, B \rightarrow C, B \rightarrow D, C \rightarrow D$. **Expected:** DFS finish times: D first, then C, then B, then A. Topological order: A, B, C, D (or A, C, B, D depending on adjacency).
22. **Input:** Find LCS of "ABCBADAB" and "BDCABA". **Expected:** LCS = "BCBA" or "BCAB" or "BDAB" (multiple optimal). Length = 4. DP table $c[1..7, 1..6]$, $b[i, j]$ with directions for reconstruction.
23. **Input:** Run Prim's MST algorithm on graph with vertices $\{a, b, c, d, e\}$ and edges: $a-b(4), a-c(2), b-c(1), b-d(5), c-d(8), c-e(10), d-e(2)$. Start at a. **Expected:** Select $a-c(2), c-b(1), b-d(5)$ [or $c-d(8)$ not selected since $d-e(2)$ already found], $d-e(2)$. MST = $\{a-c, c-b, b-d, d-e\}$, weight = $2+1+5+2 = 10$.
24. **Input:** Show the state of a binary min-heap after inserting 3, 1, 4, 1, 5, 9 into an initially empty heap. **Expected:** Step-by-step: $[3] \rightarrow [1, 3]$ (bubble 1 up) $\rightarrow [1, 3, 4] \rightarrow [1, 1, 4, 3]$ (bubble 1 up) $\rightarrow [1, 1, 4, 3, 5] \rightarrow [1, 1, 4, 3, 5, 9]$. Final heap: $[1, 1, 4, 3, 5, 9]$.
25. **Input:** Apply Hoare PARTITION to $A=[13, 19, 9, 5, 12, 8, 7, 4, 21]$, $p=1, r=9$. Show steps. **Expected:** $x=13, i=0, j=10, j \rightarrow 9: 21 > 13; j \rightarrow 8: 4 \leq 13; i \rightarrow 1: 13 \geq 13; \text{swap } A[1] \leftrightarrow A[8] \rightarrow [4, 19, 9, 5, 12, 8, 7, 13, 21]. j \rightarrow 7: 13 \geq 13?$ Actually $x=13, j \rightarrow 7: 7 \leq 13; i \rightarrow 2: 19 \geq 13; \text{swap} \rightarrow [4, 7, 9, 5, 12, 8, 19, 13, 21]. j \rightarrow 6: 8 \leq 13; i \rightarrow 3: 9 \leq 13; i \rightarrow 4: 5 \leq 13; i \rightarrow 5: 12 \leq 13; i \rightarrow 6: 8 \leq 13; i \rightarrow 7: 19 \geq 13; i=7, j=6 \rightarrow \text{exit. Return } j=6. A[1:6] \leq 13, A[7:9] \geq 13$.
26. **Input:** Trace the Gale-Shapley algorithm with: men $M=\{m1, m2\}$, women $W=\{w1, w2\}$. $m1$ prefers $[w1, w2]$, $m2$ prefers $[w2, w1]$. $w1$ prefers $[m1, m2]$, $w2$ prefers $[m2, m1]$. **Expected:** Day 1: $m1$ proposes to $w1 \checkmark$, $m2$ proposes to $w2 \checkmark$. Both matched. Stable matching: $(m1, w1), (m2, w2)$.
27. **Input:** Compute the LUP decomposition of $A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$. **Expected:** $PA = LU$. $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ (swap rows). $L = \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix}$. $U = \begin{bmatrix} 1 & 1 \\ 0 & 0.5 \end{bmatrix}$. Check: $PA = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} = LU = \begin{bmatrix} 1 & 1 \\ 0.5 \cdot 1 + 1 \cdot 0.5 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0.5 & 1 \end{bmatrix}$.
28. **Input:** For the polynomial $A(x)=3+2x+x^2$, compute its DFT with $n=4$ at $\omega_4 = i$ (primitive 4th root). **Expected:** $\omega_4 = i, \omega_4^2 = -1, \omega_4^3 = -i, \omega_4^4 = 1$. $A^0 = [3, 1]$ (even), $A^1 = [2, 0]$ (odd, with 0 padding). $A^0(1)=4, A^0(-1)=2, A^1(1)=2, A^1(-1)=2$. FFT: $y_0 = 4+1 \cdot 2=6; y_2 = 4-1 \cdot 2=2; y_1 = 2+i \cdot 2 = 2+2i; y_3 = 2-i \cdot 2 = 2-2i$. DFT = $[6, 2+2i, 2, 2-2i]$.

29. **Input:** Apply the greedy set cover algorithm: $X = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$, $F = \{S_1 = \{1, 2, 3, 7\}, S_2 = \{2, 4, 5\}, S_3 = \{3, 4, 5, 6, 9\}, S_4 = \{1, 6, 8, 10\}, S_5 = \{7, 8, 9, 10\}\}$. **Expected:** Greedy: first pick S_3 (covers 5 $\rightarrow$ 3, 4, 5, 6, 9). Remaining: $\{1, 2, 7, 8, 10\}$. Pick S_1 (covers 3 $\rightarrow$ 1, 2, 3, 7). Remaining: $\{8, 10\}$. Pick S_4 (covers 2 $\rightarrow$ 1, 6, 8, 10) or S_5 . Total: 3 sets.
30. **Input:** Compute the convolution of $a = [1, 2, 3]$ and $b = [4, 5, 6]$ using the direct (non-FFT) method. **Expected:** $c_0 = 1 \cdot 4 = 4$; $c_1 = 1 \cdot 5 + 2 \cdot 4 = 13$; $c_2 = 1 \cdot 6 + 2 \cdot 5 + 3 \cdot 4 = 28$; $c_3 = 2 \cdot 6 + 3 \cdot 5 = 27$; $c_4 = 3 \cdot 6 = 18$. $c = [4, 13, 28, 27, 18]$.
31. **Input:** Solve $T(n) = T(n-1) + n$ with $T(1) = 1$. **Expected:** Unrolling:

$$T(n) = T(n-1) + n = T(n-2) + (n-1) + n = \dots = T(1) + 2 + 3 + \dots + n = 1 + \sum_{i=2}^n i$$

$$= \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2).$$
32. **Input:** For RSA: $p=3$, $q=11$, $n=33$, $\phi(n)=20$. Choose $e=7$. Compute d and encrypt $m=2$. **Expected:** $d = e^{-1} \bmod 20 = 7^{-1} \bmod 20 = 3$ (since $7 \cdot 3 = 21 \equiv 1 \bmod 20$). Encrypt: $c = 2^7 \bmod 33 = 128 \bmod 33 = 29$. Decrypt: $m = 29^3 \bmod 33 = 24389 \bmod 33 = 2$. ✓
33. **Input:** Run Edmonds-Karp (BFS-based augmenting path) on a flow network: $s \rightarrow a$ (capacity 16), $s \rightarrow b$ (13), $a \rightarrow b$ (10), $a \rightarrow c$ (12), $b \rightarrow c$ (14), $b \rightarrow t$ (20), $c \rightarrow t$ (4). Find max flow. **Expected:** First augmenting path: $s \rightarrow a \rightarrow c \rightarrow t$, bottleneck $\min(16, 12, 4) = 4$. Second: $s \rightarrow a \rightarrow b \rightarrow c \rightarrow t$, bottleneck $\min(12, 10, 14 - 4, 0) = 0$? check residual: $s \rightarrow a$ (12), $a \rightarrow b$ (10), $b \rightarrow c$ (14), $c \rightarrow t$ (4) saturated? after first, $c \rightarrow t$ full. Actually need to trace carefully.
34. **Input:** Insertion sort on $A = [7, 3, 5, 1, 9, 2]$. Show array after each outer loop iteration. **Expected:** $i=2$: $[3, 7, 5, 1, 9, 2]$; $i=3$: $[3, 5, 7, 1, 9, 2]$; $i=4$: $[1, 3, 5, 7, 9, 2]$; $i=5$: $[1, 3, 5, 7, 9, 2]$; $i=6$: $[1, 2, 3, 5, 7, 9]$.
35. **Input:** Run QUICKSORT (with Lomuto partition) on $A = [9, 7, 5, 11, 12, 2, 14, 3, 10, 6]$. Show first partition step only. **Expected:** $x=6$. $i=0$. $j=1$: $9 > 6$; $j=2$: $7 > 6$; $j=3$: $5 \leq 6 \rightarrow i=1$, swap $A[1]=5 \leftrightarrow A[3]=9$ (no change); $j=4$: $11 > 6$; $j=5$: $12 > 6$; $j=6$: $2 \leq 6 \rightarrow i=2$, swap $A[2]=7 \leftrightarrow A[6]=2 \rightarrow [6, 5, 2, 11, 12, 9, 14, 3, 10, 6]$; $j=7$: $14 > 6$; $j=8$: $3 \leq 6 \rightarrow i=3$, swap $A[3]=11 \leftrightarrow A[8]=3 \rightarrow [9, 5, 2, 3, 12, 9, 14, 11, 10, 6]$; $j=9$: $10 > 6$. Final swap $A[4]=12 \leftrightarrow A[10]=6 \rightarrow [9, 5, 2, 3, 6, 9, 14, 11, 10, 12]$. Return $q=5$.
36. **Input:** Find an Euler tour in graph: edges (1,2), (1,3), (2,3), (3,4), (3,5), (4,5). Show path. **Expected:** Check all vertices have even degree (2: deg=2, 3: deg=4, 4: deg=2, 5: deg=2, 1: deg=2). Euler tour: 1-2-3-1-3-4-5-3 (or any valid). Fleury's algorithm: avoid bridges when possible.
37. **Input:** Apply the Knuth-Morris-Pratt prefix function to $P = \text{"ABABAC"}$. Compute π array. **Expected:** $\pi[1]=0$; $\pi[2]=0$ ($A \neq B$); $\pi[3]=1$ ($P[3]=A$ matches $P[1]=A$); $\pi[4]=2$ ($P[4]=B$ matches $P[2]=B$, $k=2$); $\pi[5]=3$ ($P[5]=A$ matches $P[3]=A$, $k=3$); $\pi[6]=0$ ($P[6]=C \neq P[4]=B$, fallback $k=\pi[3]=1$, $P[6]=C \neq P[1]=A \rightarrow 0$). $\pi = [0, 0, 1, 2, 3, 0]$.
38. **Input:** Solve $T(n) = T(\sqrt{n}) + 1$ by substitution. **Expected:** Let $n = 2^m$. Then $T(2^m) = T(2^{\lceil m/2 \rceil}) + 1$. Let $S(m) = T(2^m)$. Then $S(m) = S(m/2) + 1$.

$2) + 1 = \Theta(\lg m) = \Theta(\lg \lg n)$. So $T(n) = \Theta(\lg \lg n)$. This corresponds to the recursion in van Emde Boas tree operations.

39. **Input:** Find LIS of [3, 10, 2, 1, 20] using DP. **Expected:** Let $L[i]$ = LIS ending at i . $L[1]=1$; $L[2]=2$ ($3 < 10$); $L[3]=1$; $L[4]=1$; $L[5]=3$ ($3 < 20$). LIS length = 3. Subsequence: [3,10,20] or [2,3,20]. For $O(n \lg n)$: maintain M where $M[k]$ = smallest ending value for LIS of length k .
40. **Input:** Show the adjacency matrix of the transitive closure for graph: $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 1$. **Expected:** All vertices reachable from each other. Transitive closure matrix: all 1s (except diagonal can be 1 or 0 depending on definition). $T^* = [[1,1,1],[1,1,1],[1,1,1]]$.
41. **Input:** Compute the modular inverse of 5 modulo 11. **Expected:** Extended Euclid: $\gcd(11,5) = 1$. $11 = 2 \cdot 5 + 1 \rightarrow 1 = 11 - 2 \cdot 5$. So $-2 \equiv 9 \pmod{11}$ is the inverse. Check: $5 \cdot 9 = 45 \equiv 1 \pmod{11}$. ✓
42. **Input:** Apply BFS to find shortest path from s to t in unweighted graph: $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow c$, $b \rightarrow c$, $c \rightarrow t$. **Expected:** BFS layers: $s.d=0$; $\{a,b\}.d=1$; $\{c\}.d=2$; $t.d=3$. Path: $s \rightarrow a \rightarrow c \rightarrow t$ or $s \rightarrow b \rightarrow c \rightarrow t$, length 3.
43. **Input:** Find SCCs using DFS on graph: $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 1$, $3 \rightarrow 4$, $4 \rightarrow 5$, $5 \rightarrow 4$, $4 \rightarrow 6$, $6 \rightarrow 7$, $7 \rightarrow 6$. **Expected:** SCCs: $\{1,2,3\}$, $\{4,5\}$, $\{6,7\}$. Kosaraju: DFS order depends on start. On G^T , start from highest finish vertex, get SCCs. Component graph: $\{1,2,3\} \rightarrow \{4,5\} \rightarrow \{6,7\}$. This is a DAG.
44. **Input:** Apply the deletion algorithm to B-tree ($t=2$): root [10,20,30], delete 20. **Expected:** If 20 is in node with at least t keys (≥ 2 for $t=2$), can delete directly. If 20 has fewer keys than t , may need to merge or borrow. Full trace depends on tree structure but should maintain B-tree properties: leaves at same depth, each node has $t-1$ to $2t-1$ keys.
45. **Input:** Compute $7^5 \pmod{13}$ using modular exponentiation (square-and-multiply). **Expected:** 5 in binary = 101. $x=7$, result=1. 5 LSB=1: result= $1 \cdot 7=7$; $7^2=49 \equiv 10 \pmod{13}$. Next bit=0: result stays 7; $10^2=100 \equiv 9 \pmod{13}$. Next bit=1: result= $7 \cdot 9=63 \equiv 11 \pmod{13}$. Answer: $7^5 \equiv 11 \pmod{13}$. Verify: $7^2=49 \equiv 10$, $7^4=10^2=100 \equiv 9$, $7^5=7^4 \cdot 7=9 \cdot 7=63 \equiv 11$.
46. **Input:** Show the result of applying Graham Scan on points: (0,0), (1,1), (2,2), (0,3), (3,0). **Expected:** Sort by angle from lowest point (0,0). Order: (0,0), (1,1), (2,2), (3,0), (0,3). Stack operations: push (0,0), (1,1), (2,2); (3,0): left turn? cross product of $(1,1) \rightarrow (2,2)$ and $(2,2) \rightarrow (3,0) = (1,1) \times (1,-2) = 1 \cdot (-2) - 1 \cdot 1 = -3 < 0 \rightarrow$ right turn $\rightarrow$ pop (2,2). Cross $(0,0) \rightarrow (1,1)$ and $(1,1) \rightarrow (3,0) = (1,1) \times (2,-1) = -1 - 2 = -3 \rightarrow$ pop (1,1). Push (3,0). Now (0,3): left turn $\rightarrow$ push. Hull: (0,0), (3,0), (0,3). Actually (2,2) is inside.
47. **Input:** Show the state of a max-heap after extracting the maximum once: $A=[16,14,10,8,7,9,3,2,4,1]$. **Expected:** Extract 16. Replace with last element 1: [1,14,10,8,7,9,3,2,4]. MAX-HEAPIFY(1): $14 > 1 \rightarrow$ swap $\rightarrow$ [14,1,10,8,7,9,3,2,4]. MAX-HEAPIFY(2): 1 vs 8,7 $\rightarrow$ swap with 8 $\rightarrow$ [14,8,10,1,7,9,3,2,4]. MAX-HEAPIFY(4): 1 vs 2,4 $\rightarrow$ swap with 4 $\rightarrow$ [14,8,10,4,7,9,3,2,1]. Done. New heap size = 9.

48. **Input:** Run depth-first search on graph: $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 4, 4 \rightarrow 5, 5 \rightarrow 2$. Show discovery/finish times and detect back edge. **Expected:** Starting from 1: 1(1,10); 2(2,9); 3(3,6); 4(4,5); 5(7,8). Edge $5 \rightarrow 2$: 2 is gray (discovered but not finished) $\rightarrow$ this is a BACK EDGE $\rightarrow$ graph has a cycle. Edge types: tree: all others; back: $5 \rightarrow 2$.
49. **Input:** Apply the subset-sum DP algorithm for $S = \{3, 5, 7, 9\}$, $t = 12$. **Expected:** Table $P[0..4, 0..12]$. Initialize $P[0,0] = T$, rest F. For $i = 1..4$, $s = 0..12$: $P[i,s] = P[i-1,s] \text{ OR } P[i-1,s-S[i]]$. Entry $P[4,12] = T$. Found $3+9=12$. Also $5+7=12$. Reconstruction: trace back from $P[4,12]$ to find actual subset.
50. **Input:** Compute ω and ω^2 for $n=8$ primitive 8th roots of unity. **Expected:** $\omega_8 = e^{\{2\pi i/8\}} = \cos(\pi/4) + i \cdot \sin(\pi/4) = \sqrt{2}/2 + i \cdot \sqrt{2}/2$. $\omega_8^2 = e^{\{2\pi i \cdot 2/8\}} = e^{\{\pi i/2\}} = i$. $\omega_8^4 = e^{\{\pi i\}} = -1$. $\omega_8^8 = 1$. These satisfy cancellation lemma: $\sum_{j=0}^{n-1} (\omega_n^k)^j = 0$ for k not divisible by n .
51. **Input:** Compute the convolution of $[1, 2, 3]$ and $[4, 5, 6]$ using the FFT method (conceptually). **Expected:** Pad to length ≥ 5 (e.g., 8 for power-of-two). Compute FFT of both padded vectors: $A' = \text{FFT}([1, 2, 3, 0, 0, 0, 0, 0])$, $B' = \text{FFT}([4, 5, 6, 0, 0, 0, 0, 0])$. Multiply pointwise: $C'[i] = A'[i] \cdot B'[i]$. Compute inverse FFT: $C = \text{IFFT}(C')$. Result: $[4, 13, 28, 27, 18, 0, 0, 0]$. Truncate to first 5 elements.
52. **Input:** Apply the Bellman-Ford algorithm to detect a negative cycle: $s \rightarrow a(1)$, $a \rightarrow b(-2)$, $b \rightarrow c(-3)$, $c \rightarrow a(-1)$. **Expected:** After $|V|-1 = 3$ passes: $s.d=0$, $a.d=1$, $b.d=-1$, $c.d=-4$. Fourth pass: $a \rightarrow b$ relax: $b.d = \min(-1, 1+(-2)) = -1$; $b \rightarrow c$: $c.d = \min(-4, -1+(-3)) = -4$; $c \rightarrow a$: $a.d = \min(1, -4+(-1)) = -5$ (CHANGED from 1). Since $a.d$ changed in V -th pass, negative cycle detected. Cycle: $a \rightarrow b \rightarrow c \rightarrow a$ has weight $1+(-2)+(-3)+(-1) = -5 < 0$.
53. **Input:** Merge two sorted lists $[1, 3, 5]$ and $[2, 4, 6]$ using P-MERGE conceptual steps. **Expected:** P-MERGE($A[1..3]$, $B[1..3]$, $C[1..6]$): find median of longer = $A[2]=3$. Binary search B for insertion point of 3 $\rightarrow B[2]=4$. Recursively spawn P-MERGE($A[1..1]$, $B[1..1]$, $C[1..2]$) and P-MERGE($A[3..3]$, $B[3..3]$, $C[4..5]$) in parallel. Set $C[3]=3$. Work $T_1 = \Theta(n)$, Span $T_\infty = \Theta(\lg^2 n)$.
54. **Input:** Show the steps of linear probing hash table with $m=7$, hash $h(k)=k \bmod 7$, keys: 50, 21, 58, 17, 15, 49, 56. **Expected:** Insert: 50 mod 7=1; 21 mod 7=0; 58 mod 7=2; 17 mod 7=3; 15 mod 7=1 $\rightarrow$ collision, probe 2 $\rightarrow$ collision, probe 3 $\rightarrow$ collision, probe 4; 49 mod 7=0 $\rightarrow$ collision, probe 1 $\rightarrow$ collision, probe 2 $\rightarrow$ collision, probe 3 $\rightarrow$ collision, probe 4 $\rightarrow$ collision, probe 5; 56 mod 7=0 $\rightarrow$ collision, probe 1 $\rightarrow$ collision, probe 2 $\rightarrow$ collision, probe 3 $\rightarrow$ collision, probe 4 $\rightarrow$ collision, probe 5 $\rightarrow$ collision, probe 6. Final table: $[21, 50, 58, 17, 15, 49, 56]$.
55. **Input:** Find min and max of $[3, 7, 1, 9, 5, 2, 8]$ using the tournament method. **Expected:** Pair elements: $(3, 7) \rightarrow \min=3, \max=7$; $(1, 9) \rightarrow \min=1, \max=9$; $(5, 2) \rightarrow \min=2, \max=5$; $(8) \rightarrow \min=8, \max=8$. Compare mins: $\min(3, 1, 2, 8)=1$. Compare maxs: $\max(7, 9, 5, 8)=9$. Total

comparisons: $3 \cdot (n/2) - 2 = 3 \cdot 3.5 - 2 = 3 \cdot 4 - 2 = 10$ comparisons? Actually $\lceil 3n/2 \rceil - 2 = \lceil 3 \cdot 7/2 \rceil - 2 = \lceil 10.5 \rceil - 2 = 11 - 2 = 9$. Naive: $2 \cdot (n-1) = 12$. Tournament saves 3 comparisons.

56. **Input:** Show the result of applying Bucket sort to uniformly distributed numbers [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68] ($n=10$). **Expected:** Create 10 buckets. Distribute: $B_1=[0.17,0.12]$, $B_2=[0.26,0.21,0.23]$, $B_3=[0.39]$, $B_6=[0.68]$, $B_7=[0.78,0.72]$, $B_9=[0.94]$. Sort each (insertion sort): $B_1=[0.12,0.17]$, $B_2=[0.21,0.23,0.26]$, $B_3=[0.39]$, $B_6=[0.68]$, $B_7=[0.72,0.78]$, $B_9=[0.94]$. Concatenate: [0.12,0.17,0.21,0.23,0.26,0.39,0.68,0.72,0.78,0.94].
57. **Input:** Apply the Gale-Shapley algorithm with 3 men and 3 women. Men preferences: $M1=[W1,W2,W3]$, $M2=[W1,W3,W2]$, $M3=[W2,W1,W3]$. Women preferences: $W1=[M3,M1,M2]$, $W2=[M1,M2,M3]$, $W3=[M2,M3,M1]$. **Expected:** Day 1: $M1 \rightarrow W1$, $M2 \rightarrow W1$ ($W1$ keeps $M1$, rejects $M2$), $M3 \rightarrow W2$ ($W2$ accepts). Day 2: $M2 \rightarrow W3$ ($W3$ accepts). All matched: ($M1,W1$), ($M2,W3$), ($M3,W2$). This is proposer-optimal and stable.
58. **Input:** Compute the S and P matrices for 2×2 Strassen: $A=[[2,4],[6,8]]$, $B=[[1,3],[5,7]]$. **Expected:** $S_1=3-7=-4$, $S_2=2+4=6$, $S_3=6+8=14$, $S_4=5-1=4$, $S_5=2+8=10$, $S_6=1+7=8$, $S_7=4-8=-4$, $S_8=5+7=12$, $S_9=2-6=-4$, $S_{10}=1+3=4$. $P_1=2 \cdot (-4)=-8$, $P_2=6 \cdot 7=42$, $P_3=14 \cdot 1=14$, $P_4=8 \cdot 4=32$, $P_5=10 \cdot 8=80$, $P_6=-4 \cdot 12=-48$, $P_7=-4 \cdot 4=-16$. $C_{11}=P_5+P_4-P_2+P_6=80+32-42-48=22$. $C_{12}=P_1+P_2=-8+42=34$. $C_{21}=P_3+P_4=14+32=46$. $C_{22}=P_5+P_1-P_3-P_7=80-8-14-(-16)=74$. Check: direct multiplication: $[[2 \cdot 1 + 4 \cdot 5, 2 \cdot 3 + 4 \cdot 7], [6 \cdot 1 + 8 \cdot 5, 6 \cdot 3 + 8 \cdot 7]] = [[22, 34], [46, 74]]$. ✓
59. **Input:** Apply Horner's rule to evaluate $P(x)=2x^3+x^2-3x+1$ at $x=2$. **Expected:** Rewrite: $P(x) = (((2 \cdot x + 1) \cdot x) - 3) \cdot x + 1$. Evaluate: $y=2$; $y=2 \cdot 2 + 1=5$; $y=5 \cdot 2 - 3=7$; $y=7 \cdot 2 + 1=15$. $P(2)=15$. Direct: $2 \cdot 8 + 4 - 6 + 1 = 16 + 4 - 6 + 1 = 15$. ✓ Horner's rule: $O(n)$ multiplications instead of $O(n^2)$ for naive.
60. **Input:** Show the state after each operation: MAKE-SET(1), MAKE-SET(2), MAKE-SET(3), UNION(1,2), UNION(2,3), FIND-SET(1). **Expected:** With union by size: After MS(1): {1}. MS(2): {2}. MS(3): {3}. Union(1,2): 1 parent of 2 (size 2). Union(2,3): find $2 \rightarrow 1$, find $3 \rightarrow 3$, $\text{size}(1)=2 \geq \text{size}(3)=1 \rightarrow 1$ parent of 3 (size 3). FIND-SET(1): returns 1 (no change, already root). FIND-SET(3): returns 1 with path compression ($3 \rightarrow 1$ directly).
61. **Input:** Compute the prefix function π for $P="AAABAAAA"$. **Expected:** $\pi = [0,1,2,0,1,2,3,3]$. Step-by-step: $i=1, k=0$: $P[1] \neq P[0] \rightarrow \pi[1]=0$. $i=2$: $P[2] = P[0] \rightarrow k=1, \pi[2]=1$. $i=3$: $P[3] \neq P[1] \rightarrow k=2, \pi[3]=2$. $i=4$: $P[4] \neq P[2] \rightarrow k=\pi[2]=1$; $P[4] \neq P[1] \rightarrow k=\pi[1]=0$; $P[4] \neq P[0] \rightarrow \pi[4]=0$. $i=5$: $P[5] = P[0] \rightarrow k=1, \pi[5]=1$. $i=6$: $P[6] = P[1] \rightarrow k=2, \pi[6]=2$. $i=7$: $P[7] = P[2] \rightarrow k=3, \pi[7]=3$. $i=8$: $P[8] = P[3] = \dots$ $P[3]=A$, but $P[3]$ is the 4th character (0-indexed: $P[3]=A$). Actually P is "AAABAAAA", $P[3]=B$.

So $P[8]=A \neq P[3]=B \rightarrow k=\pi[3]=0$; $P[8]=P[0] \rightarrow k=1, \pi[8]=1$. Wait mismatched. Let me recalc:
 $P[0]=A, P[1]=A, P[2]=A, P[3]=B, P[4]=A, P[5]=A, P[6]=A, P[7]=A$. $\pi[0]=0$.
 $i=1$: $k=\pi[0]=0$, $P[1]=A=P[0] \rightarrow k=1, \pi[1]=1$. $i=2$: $k=\pi[1]=1$,
 $P[2]=A=P[1] \rightarrow k=2, \pi[2]=2$. $i=3$: $k=\pi[2]=2$,
 $P[3]=B \neq P[2]=A \rightarrow k=\pi[1]=1$, $P[3]=B \neq P[1]=A \rightarrow k=\pi[0]=0$,
 $P[3]=B \neq P[0]=A \rightarrow \pi[3]=0$. $i=4$: $k=\pi[3]=0$, $P[4]=A=P[0] \rightarrow k=1, \pi[4]=1$.
 $i=5$: $k=\pi[4]=1$, $P[5]=A=P[1] \rightarrow k=2, \pi[5]=2$. $i=6$: $k=\pi[5]=2$,
 $P[6]=A=P[2] \rightarrow k=3, \pi[6]=3$. $i=7$: $k=\pi[6]=3$, $P[7]=A \neq P[3]=B \rightarrow k=\pi[2]=2$,
 $P[7]=A=P[2]=A \rightarrow k=3, \pi[7]=3$. $\pi=[0,1,2,0,1,2,3,3]$.

62. **Input:** Find the minimum cut in a graph with vertices $\{s,a,b,c,t\}$ and edges: $s \rightarrow a(4)$, $s \rightarrow b(2)$, $a \rightarrow b(1)$, $a \rightarrow c(3)$, $b \rightarrow c(2)$, $b \rightarrow t(3)$, $c \rightarrow t(5)$.
Expected: Max flow (using any algorithm) = 6. Minimum cut: cut with capacity 6. Example: $S=\{s,a,b\}$, $T=\{c,t\}$: edges crossing: $a \rightarrow c(3) + b \rightarrow c(2) + b \rightarrow t(3) = 8$. Another: $S=\{s,a\}$, $T=\{b,c,t\}$: $s \rightarrow b(2) + a \rightarrow b(1) + a \rightarrow c(3) = 6$. Min cut = 6. Verify max flow = 6.
63. **Input:** Run the capacity scaling algorithm starting with $\Delta = 8$ on a small flow network. **Expected:** Initialize $\Delta = \text{largest power of } 2 \leq \text{max capacity}$. While $\Delta \geq 1$: find augmenting paths in residual using only edges with capacity $\geq \Delta$. For each Δ -phase: repeatedly augment until no more paths. Double: augment along path with bottleneck $\geq \Delta$. Halve: $\Delta = \Delta/2$. Total phases: $O(\lg C)$. Each phase: $O(E^2)$ augmentations (each at least doubles flow value or saturates edge). Total $O(E^2 \lg C)$.
64. **Input:** Compute the unique fixed point of the Bellman operator for a Markov Decision Process with 2 states. **Expected:** Bellman operator $T(V) = \max_a [R(s,a) + \gamma \sum P(s'|s,a) \cdot V(s')]$. Iterate from $V_0 = 0$: $V_{k+1} = T(V_k)$. Contraction in sup-norm with factor $\gamma < 1$ ensures convergence to unique fixed point (optimal value function). Convergence rate: $\|V_k - V^*\|_\infty \leq \gamma^k \cdot \|V_0 - V^*\|_\infty$. For $\gamma=0.9$, after $k=100$ iterations, $\text{error} \leq \gamma^{100} = \exp(-10.5) \approx 2.7 \times 10^{-5}$.
65. **Input:** Compute the conjugate of $f(x) = x^2$ (quadratic function).
Expected: $f(y) = \sup_x (y \cdot x - f(x)) = \sup_x (y \cdot x - x^2)$. Maximum at $x = y/2$: $f(y) = y \cdot (y/2) - (y/2)^2 = y^2/2 - y^2/4 = y^2/4$. The conjugate of a quadratic is also quadratic. Fenchel duality: optimization $\min_x f(x) + g(Ax) \leftrightarrow \max_y -f(A^T y) - g(-y)$. Used in convex optimization (dual gradient methods, ADMM).

Essay / Long-Form

- Q:** Compare and contrast quicksort, heapsort, and merge sort. **Key points:** Asymptotic (quicksort $\Theta(n^2)$ worst, $O(n \lg n)$ expected; others $O(n \lg n)$ worst). In-place (heapsort/quicksort yes, mergesort no). Stability (mergesort yes, others no). Constants (quicksort smallest, heapsort moderate, mergesort highest). Practical: quicksort fastest, heapsort for worst-case guarantees, mergesort for stability.
- Q:** Prove that the expected running time of RANDOMIZED-QUICKSORT is $O(n \lg n)$. **Key points:** Lemma 7.1 (time = $O(n+X)$, X = comparisons).

Lemma 7.2 (z_i compares z_j iff first pivot from Z_{ij}). Lemma 7.3 ($\Pr = 2/(j-i+1)$). $E[X] = \sum \sum 2/(j-i+1) = O(n \lg n)$. Expected time = $O(n + n \lg n) = O(n \lg n)$.

3. **Q:** Explain how SELECT achieves worst-case linear time. **Key points:** Groups of 5 ensure pivot $\geq 3n/10$ elements and $\leq 3n/10$. Recursive subproblem $\leq 7n/10$. Recurrence $T(n) \leq T(n/5) + T(7n/10) + \Theta(n) \rightarrow T(n) = \Theta(n)$ by substitution. Groups of 3 give $O(n \lg n)$. Theoretical interest only — RANDOMIZED-SELECT more practical.
4. **Q:** Prove the max-flow min-cut theorem. **Key points:** (1) $\Rightarrow$ (2): augmenting path would increase flow, contradicting maximality. (2) $\Rightarrow$ (3): S = vertices reachable from s in G_f ; edges $S \rightarrow T$ saturated, edges $T \rightarrow S$ have 0 flow $\rightarrow |f| = c(S, T)$. (3) $\Rightarrow$ (1): Corollary 24.5 states $|f| \leq c(S, T)$ for any cut, so $|f| = c(S, T)$ is maximum.
5. **Q:** Prove that a red-black tree with n nodes has height at most $2 \lg(n+1)$. **Key points:** Inductive claim: subtree at x has $\geq 2^{\text{bh}(x)} - 1$ internal nodes. Property 4: at least half nodes on path are black $\rightarrow \text{bh}(\text{root}) \geq h/2$. Therefore $n \geq 2^{h/2} - 1 \rightarrow h \leq 2 \lg(n+1)$.
6. **Q:** Analyze the expected performance of hashing with chaining. **Key points:** Load factor $\alpha = n/m$. Unsuccessful search: expected length of chain = α , time $\Theta(1+\alpha)$. Successful search: expected elements before searched element = $(\alpha/2 - \alpha/(2n))$, time $\Theta(1+\alpha)$. Universal hashing guarantees $O(1+\alpha)$ for any input.
7. **Q:** Compare and contrast Kruskal's and Prim's MST algorithms. **Key points:** Kruskal (edge-based, sort then union-find, $O(E \lg V)$, builds forest). Prim (vertex-based, priority queue, $O(E \lg V)$ binary, $O(V^2)$ dense, grows single tree). Both rely on cut property. Kruskal better for sparse, Prim for dense with array.
8. **Q:** Describe the four single-source shortest-path algorithms and when to use each. **Key points:** BFS (unweighted, $O(V+E)$). DAG-SP (DAG only, negative OK, $\Theta(V+E)$, topological sort). Dijkstra (nonnegative, $O(E \lg V)$). Bellman-Ford (negative OK, detects cycles, $O(VE)$). Dijkstra for most cases with nonnegative weights; Bellman-Ford when negative edges present.
9. **Q:** Explain the relationship between P, NP, NP-complete, and NP-hard. Give examples. **Key points:** P = polynomial-time decidable (shortest path, MST). NP = polynomial-time verifiable (SAT, TSP, CLIQUE). NPC = $\text{NP} \cap \text{NP-hard}$ (SAT, 3-CNF, CLIQUE, VERTEX-COVER, HAM-CYCLE, TSP, SUBSET-SUM). If any $\text{NPC} \in \text{P}$, then $\text{P} = \text{NP}$. NP-hard includes unconstrained optimization (TSP optimization).
10. **Q:** Compare DP vs greedy algorithms with examples of where each works. **Key points:** DP: optimal substructure + overlapping subproblems (matrix-chain, LCS, OBST). Greedy: greedy-choice property + optimal substructure (activity selection, Huffman, MST). Key difference: DP explores all subproblem solutions; greedy commits

to one choice. 0-1 knapsack needs DP; fractional knapsack works with greedy.

11. **Q:** Explain the analysis of disjoint-set forests with union by rank and path compression. **Key points:** Union by rank ensures tree height $\leq \lg n$. Path compression flattens trees during FIND-SET. Together achieve $O(m \alpha(n))$ for m operations. Inverse Ackermann $\alpha(n) \leq 4$ for all practical n ($\leq 10^{80}$). Tarjan proved tight bound via potential function with $\text{level}(x)$ and $\text{iter}(x)$.
12. **Q:** Describe the Ford-Fulkerson method and prove the max-flow min-cut theorem. **Key points:** Augment along paths in residual network until none exist. Max-flow min-cut: three equivalent conditions (max flow, no augmenting path, flow = cut capacity). Corollary: integrality theorem for integer capacities. Applications to bipartite matching and edge-disjoint paths.
13. **Q:** Explain how BFS computes shortest paths in unweighted graphs and prove its correctness. **Key points:** Queue ensures processing by distance layers. Invariants: (1) $v.d \geq \delta(s,v)$ always; (2) queue contains vertices with d values differing by at most 1; (3) BFS tree gives shortest paths. Theorem 20.5: $v.d = \delta(s,v)$ for all reachable v . Proof by induction on distance.
14. **Q:** Discuss the P vs NP problem and its implications for algorithm design. **Key points:** P = polynomial-time solvable; NP = polynomial-time verifiable. NP-complete hardest in NP. $P \neq NP$ means no polynomial algorithm for TSP, SAT, etc. Implication: approximation algorithms, heuristics, and exponential algorithms for small instances. Oracle separation results and circuit complexity lower bounds.
15. **Q:** Analyze the expected running time of RANDOMIZED-SELECT. **Key points:** Indicator variables for comparisons. Partition step $O(n)$. Expected size reduction: with probability $1/2$, pivot falls in middle 50% $\rightarrow$ subproblem $\leq 3n/4$. $E[T(n)] \leq T(3n/4) + O(n) \rightarrow E[T(n)] = \Theta(n)$. Formal analysis via recurrence with indicator for pivot rank.
16. **Q:** Compare and contrast open addressing and chaining for hash tables. **Key points:** Chaining: α can exceed 1, easy deletion, pointer overhead. Open addressing: $\alpha < 1$ required, deletion hard (DELETED marker), better cache performance. Expected probes: chain search $\Theta(1+\alpha)$, open addr unsuccessful $\leq 1/(1-\alpha)$. Open addressing preferred for in-memory tables with known n .
17. **Q:** Explain the competitive analysis framework with examples from caching and ski-rental. **Key points:** Competitive ratio ρ : $C_{\text{ONLINE}} \leq \rho \cdot C_{\text{OPT}} + \alpha$. Ski-rental: rent $B-1$ days then buy $\rightarrow 2-1/B$ competitive (optimal deterministic). Paging: LRU, FIFO, Marking are k -competitive (k = cache size); no deterministic can beat k . Randomization helps: Marking is $O(\lg k)$ -competitive; $\Omega(\lg k)$ lower bound.
18. **Q:** Derive and explain the Master Theorem for solving recurrences. **Key points:** For $T(n) = aT(n/b) + f(n)$: compare $f(n)$ to $n^{\log_b a}$. Three

cases: (1) leaves dominate: $f = O(n^{\log_b a - \epsilon}) \rightarrow T = \Theta(n^{\log_b a})$; (2) tied: $f = \Theta(n^{\log_b a} \lg^k n) \rightarrow T = \Theta(n^{\log_b a} \lg^{k+1} n)$; (3) root dominates: $f = \Omega(n^{\log_b a + \epsilon})$ AND regularity $af(n/b) \leq cf(n) \rightarrow T = \Theta(f(n))$. Intuition from recursion tree: cost at root vs leaves.

19. **Q:** Analyze Strassen's matrix multiplication algorithm and its place in the evolution of matrix multiplication. **Key points:** Classic: 8 multiplications, $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$. Strassen: 7 multiplications, $T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}) \approx \Theta(n^{2.81})$. Improvement via 10 $S_1 \dots S_{10}$ matrices. Coppersmith-Winograd: $O(n^{2.376})$. Alman-Williams: $O(n^{2.37287})$. Practical: Strassen used for large n ; naive better for small.
20. **Q:** Explain the relationship between maximum flow, minimum cut, and bipartite matching. **Key points:** Max-flow min-cut: flow value = min cut capacity. Bipartite matching reduces to flow: source $\rightarrow L(1)$, $L \rightarrow R(1)$, $R \rightarrow \text{sink}(1)$. Integrality ensures integer matching. Hall's theorem as consequence: perfect matching exists iff $|A| \leq |N(A)|$ for all $A \subseteq L$. Hopcroft-Karp faster for matching $O(\sqrt{V} \cdot E)$ vs flow $O(VE^2)$.
21. **Q:** Compare and contrast counting sort, radix sort, and bucket sort. When would you use each? **Key points:** Counting sort: integers in $[0, k]$, $\Theta(n+k)$, stable. Use when $k = O(n)$. Radix sort: d -digit numbers, $\Theta(d(n+k))$, use when d is small and digits have limited range. Bucket sort: uniform $[0, 1)$, expected $\Theta(n)$, use for floating-point uniform data. All beat $\Omega(n \lg n)$ bound by not using comparisons.
22. **Q:** Prove the correctness of Dijkstra's algorithm. **Key points:** Invariant: when vertex u is extracted from Q , $u.d = \delta(s, u)$. Proof by contradiction: suppose first violation is u . Let y be the first vertex on the true shortest path from s to u that isn't in S (processed set). Show $y.d \leq \delta(s, u)$ and since u is picked first, $u.d \leq y.d \leq \delta(s, u) = \delta(s, u)$. Contradiction since $u.d > \delta(s, u)$ assumed. Requires nonnegative weights.
23. **Q:** Describe the concept of universal hashing and explain why it's important. **Key points:** Family H of hash functions where $\Pr[h(k_1) = h(k_2)] \leq 1/m$ for distinct keys. This is a probabilistic guarantee independent of input (worst-case keys). Without it, adversary can construct keys that all hash to same slot $\rightarrow \Theta(n)$ per operation. Universal families exist (H_{pm} , multiply-shift). Application: guarantee expected $O(1 + \alpha)$ search for any input.
24. **Q:** Analyze the amortized cost of operations in a binary counter. **Key points:** Aggregate: bit $A[i]$ flips $\lfloor n/2^i \rfloor$ times, total flips $\sum \lfloor n/2^i \rfloor < 2n$, so amortized $O(1)$ per INCREMENT. Potential: $\Phi = \text{number of 1s}$. If t trailing 1s flip to 0 and 1 flips to 1: $\Delta\Phi = 1 - t$, $\hat{c} = (t+1) + (1-t) = 2$. DECREMENT breaks $O(1)$: alternating increments/decrements can flip $\Theta(\lg n)$ bits per operation.
25. **Q:** Explain how the SELECT algorithm achieves worst-case linear time. **Key points:** Groups of 5 ensure median-of-medians pivot splits input so worst-case size $\leq 7n/10$. Recurrence: $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$.

Solve via substitution: assume $T(k) \leq ck$, then $T(n) \leq cn/5 + 7cn/10 + \Theta(n) = 9cn/10 + \Theta(n)$. Choose c large enough so $c \cdot n/10$ dominates $\Theta(n) \rightarrow T(n) \leq cn$ for $c = 10 \cdot \Theta(1)$. Why not 3? Groups of 3 give $T(n) \leq T(n/3) + T(2n/3) + \Theta(n) \rightarrow O(n \lg n)$.

26. **Q:** Compare and contrast the four string-matching algorithms covered in CLRS. **Key points:** Naive: $O(nm)$, no preprocessing. Rabin-Karp: $O(n+m)$ expected, rolling hash, good for multiple patterns. Finite automaton: $O(m|\Sigma|)$ preprocessing, $O(n)$ matching, large space. KMP: $O(m)$ preprocessing (prefix function), $O(n)$ matching, no backtracking. KMP best for single pattern; Rabin-Karp for multiple patterns; FA for small Σ .
27. **Q:** Describe the Cook-Levin theorem and its significance. **Key points:** CIRCUIT-SAT is NP-complete. Proof: given any NP language L and polynomial-time verifier M , construct a boolean circuit that simulates M on input x and certificate. The circuit is satisfiable iff there exists a certificate that makes M accept. This shows every NP problem reduces to CIRCUIT-SAT. Significance: first NP-complete problem, establishing the existence of a hardest problem in NP.
28. **Q:** Discuss the interplay between data structure choice and algorithm design for graph problems. **Key points:** Adjacency list vs matrix affects asymptotic complexity: Prim's $O(V^2)$ with array for dense, $O(E \lg V)$ with binary heap for sparse. Dijkstra: $O(V^2)$ array vs $O(E \lg V)$ heap. Floyd-Warshall requires matrix representation. Edmonds-Karp: adjacency list for BFS. Choosing the right data structure is critical for achieving optimal algorithm performance.
29. **Q:** Explain how the potential method is used to analyze the amortized cost of disjoint-set forest operations. **Key points:** Potential function based on ranks and levels. Union-by-rank ensures tree height $O(\lg n)$. Path compression reduces future costs. Amortized analysis shows FIND-SET takes $O(\alpha(n))$ time. The inverse Ackermann function grows so slowly that for all practical n , $\alpha(n) \leq 4$. Tight bound proven by Tarjan using multiple-level potential.
30. **Q:** Derive the expected running time of RANDOMIZED-QUICKSORT. **Key points:** Let X = total comparisons. $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$ where $X_{ij} = 1$ if z_i, z_j compared. $\Pr[z_i \text{ compares } z_j] = 2/(j-i+1)$ (occurs only when first pivot from $\{z_i, \dots, z_j\}$). $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2/(j-i+1) = \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} 2/k < \sum_{i=1}^{n-1} 2 \cdot H_{n-i} = 2n \cdot H_n = O(n \lg n)$. Partition costs $O(n) \rightarrow$ total expected $O(n \lg n)$.
31. **Q:** What is the cycle property of MSTs and how is it used? **Key points:** For any cycle in a weighted graph, the heaviest edge on that cycle is in no minimum spanning tree. Proof: assume heaviest edge e is in some MST T . Removing e splits T into two components; there exists another edge e' on the cycle crossing the cut. Replacing e with e' gives a lighter or equal tree, contradicting e being heaviest. Used in reverse-delete

algorithm: sort edges descending, remove if removal doesn't disconnect.

32. **Q:** Explain the Ford-Fulkerson method and prove the max-flow min-cut theorem. **Key points:** Start with zero flow. While there exists an augmenting path in residual network, augment flow by bottleneck capacity. Max-flow min-cut: three conditions are equivalent — (1) f is max flow; (2) no augmenting path in G_f ; (3) $|f| = c(S,T)$ for some cut. Proof: (1)→(2): if path existed, augmenting would increase flow. (2)→(3): let S = vertices reachable from s in G_f ; all edges from S to $V \setminus S$ are saturated, so $|f| = c(S,T)$. (3)→(1): for any cut, $|f| \leq c(S,T)$, so equality implies maximum.
33. **Q:** Discuss the design and analysis of Huffman coding for data compression. **Key points:** Algorithm: build min-heap of character frequencies. While >1 node: extract two smallest, merge (sum frequencies), insert back. Assign 0/1 to edges. Optimality proofs: Lemma 15.2 (lowest-frequency characters can be deepest siblings), Lemma 15.3 (optimal substructure), Theorem 15.4 (greedy produces optimal prefix code). Full binary tree required: if not full, one could remove a leaf and reduce code length. Compression ratio: Huffman reduces file size by 20-90% depending on distribution.
34. **Q:** Compare and contrast the three hash collision resolution strategies: chaining, linear probing, and double hashing. **Key points:** Chaining: external storage (linked lists), α can exceed 1, easy deletion, pointer overhead. Linear probing: $O(1)$ with 5-independent hash and $\alpha \leq 2/3$, primary clustering (long runs), hard deletion. Double hashing: $\Theta(m^2)$ probe sequences, avoids primary clustering, $h_2(k)$ must be nonzero and coprime to m . Expected probes: chaining $\Theta(1+\alpha)$, open addr unsuccessful $\leq 1/(1-\alpha)$.
35. **Q:** Analyze the expected running time of Rabin-Karp string matching. **Key points:** Preprocessing $O(m)$ for pattern hash. Rolling hash update $O(1)$ per shift. For $n-m+1$ shifts, total $O(n+m)$ expected time. Worst-case $\Theta(nm)$ when hash collisions cause many false positives. Spurious hits probability = $1/q$ (with modulus q). Choose large prime q ($\sim 2^{31}$) to make probability negligible. Expected number of verified matches = $O(1)$ when hash collisions rare.
36. **Q:** Prove the optimality of the furthest-in-future (clairvoyant) caching algorithm. **Key points:** Furthest-in-future evicts the page whose next access is furthest in the future. Optimality proof via exchange argument: let OPT be optimal algorithm, transform step by step to match FIF behavior without increasing faults. Key lemma: for any request sequence, there exists an optimal algorithm that evicts the page with furthest next access. Adversary argument shows no deterministic online algorithm can beat k -competitive.
37. **Q:** Explain how dynamic programming solves the matrix-chain multiplication problem. **Key points:** Problem: find optimal parenthesization of $A_1 \cdot A_2 \cdot \dots \cdot A_n$ to minimize scalar multiplications.

Optimal substructure: optimal parenthesization splits at some k , and both subchains must be optimally parenthesized. Recurrence: $m[i,j] = \min_{i \leq k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j)$. Compute table by chain length ($\ell=2..n$). Build $s[i,j]$ table for reconstruction. $\Theta(n^3)$ time, $\Theta(n^2)$ space. Example: $A_1(30 \times 35)$, $A_2(35 \times 15)$, $A_3(15 \times 5)$: $m[1,3] = \min(m[1,1]+m[2,3]+30 \cdot 35 \cdot 5, m[1,2]+m[3,3]+30 \cdot 15 \cdot 5)$.

38. **Q:** Contrast deterministic and randomized online algorithms using ski-rental and paging as examples. **Key points:** Deterministic ski-rental: rent $B-1$ days then buy, $2-1/B$ competitive. Randomized: expected $e/(e-1) \approx 1.582$ competitive — worse-case randomization helps. Deterministic paging: no algorithm can beat k -competitive. Randomized paging: Random Marking achieves $O(\lg k)$ -competitive, with matching $\Omega(\lg k)$ lower bound. Randomization breaks adversarial input patterns.
39. **Q:** Describe how to apply the Master Theorem to solve recurrences, including cases where it does not apply. **Key points:** For $T(n) = aT(n/b) + f(n)$: compare $f(n)$ to $n^{\{\log_b a\}}$. Case 1 (leaves dominate): $f = O(n^{\{\log_b a\} - \epsilon}) \rightarrow T = \Theta(n^{\{\log_b a\}})$. Case 2 (tied): $f = \Theta(n^{\{\log_b a\}} \lg^k n) \rightarrow T = \Theta(n^{\{\log_b a\}} \lg^{k+1} n)$. Case 3 (root dominates): $f = \Omega(n^{\{\log_b a\} + \epsilon})$ and $af(n/b) \leq cf(n)$ (regularity) $\rightarrow T = \Theta(f(n))$. When Master doesn't apply: $f(n)$ falls between cases (e.g., $n/\lg n$), use Akra-Bazzi or recursion tree. Example: $T(n) = 2T(n/2) + n \lg n$ falls in Case 2 with $k=1 \rightarrow \Theta(n \lg^2 n)$.
40. **Q:** Analyze the parallel mergesort (P-MERGE-SORT) using work and span. **Key points:** P-MERGE: Work $\Theta(n)$, Span $\Theta(\lg^2 n)$. Binary search on longer half ($O(\lg n)$), then recursively spawn two merges in parallel. P-MERGE-SORT: $T_1(n) = 2T_1(n/2) + \Theta(n) = \Theta(n \lg n)$. $T_\infty(n) = T_\infty(n/2) + \Theta(\lg^2 n) = \Theta(\lg^3 n)$ [or $\Theta(\lg n \cdot \lg \lg n)$ with optimized version]. Parallelism = $\Theta(n/\lg^2 n)$. Greedy scheduling: $T_P \leq T_1/P + T_\infty$. Linear speedup when $n/P \gg \lg^2 n$.
41. **Q:** Explain the concept of competitive ratio and analyze the Move-to-Front (MTF) list accessing algorithm. **Key points:** MTF: after accessing element, move it to front of linked list. Theorem: MTF is 2-competitive. Proof via potential function: Φ = number of inversions between MTF list and optimal list. Each access: pay c (position) + $2 \cdot (c-1)$ (inversions created) - inversions destroyed by moving to front. Amortized cost = $2 \cdot c_{OPT} - 1$. Ratio: 2. Lower bound: no deterministic online list accessing algorithm can be better than 2-competitive.
42. **Q:** Describe the relationship between P, NP, NP-complete, and NP-hard. Give examples of each class and explain what it means if $P=NP$. **Key points:** P: solvable in polynomial time (sorting, MST, shortest path). NP: verifiable in polynomial time (SAT, TSP, CLIQUE). NP-complete: in NP and every NP problem reduces to it (SAT, CLIQUE, VERTEX-COVER, HAM-CYCLE). NP-hard: every NP problem reduces to it, but may not be in NP (TSP-OPT, SAT-OPT). If $P=NP$: cryptography breaks, optimization problems become tractable, mathematics becomes mechanizable. If $P \neq NP$: there are inherently hard problems requiring approximation or exponential algorithms.

43. **Q:** Discuss the Hopcroft-Karp algorithm for maximum bipartite matching and explain why it achieves $O(\sqrt{V} \cdot E)$. **Key points:** Bipartite matching modeled as flow: source $\rightarrow L(1)$, $L \rightarrow R(1 \text{ if edge})$, $R \rightarrow \text{sink}(1)$. Ford-Fulkerson: $O(V \cdot E)$ because each augmenting path increases flow by 1, at most V paths. Hopcroft-Karp: each phase finds maximal set of vertex-disjoint shortest augmenting paths using BFS + DFS. Paths are shortest in terms of edges. Each phase increases shortest path length by at least 2. Since path length $\leq 2\sqrt{V} + 1$, only $O(\sqrt{V})$ phases needed. Each phase $O(E)$. Total $O(\sqrt{V} \cdot E)$. Key innovation: augmenting multiple disjoint paths per phase reduces number of phases.
44. **Q:** Compare deterministic and randomized algorithms for solving the selection problem (find k -th smallest element). **Key points:** Deterministic SELECT: uses groups of 5, median-of-medians pivot, $O(n)$ worst-case. High constant factor ($\approx 20n$). Randomized SELECT: random pivot, $O(n)$ expected. Practical preference: randomized version due to lower constants and simpler implementation. Both use recursive partitioning. Expected vs worst-case tradeoff: SELECT guaranteed $O(n)$ but overhead of median-of-medians computation. For small n , simple sorting works. For streaming, use heap-based sampling.
45. **Q:** Explain the concept of a PTAS (Polynomial-Time Approximation Scheme) and FPTAS (Fully PTAS). **Key points:** PTAS: for any $\epsilon > 0$, finds $(1+\epsilon)$ -approximation in time $O(n^{f(\epsilon)})$ where f is some function (e.g., $O(n^{2/\epsilon})$). FPTAS: time polynomial in both n and $1/\epsilon$ (e.g., $O(n^3/\epsilon)$). Knapsack has FPTAS via rounding/scaling: round values to multiples of $\epsilon \cdot v_{\max}/K$, run DP in $O(n \cdot K/\epsilon)$ time. TSP (general) has no PTAS unless $P=NP$ (APX-hard). TSP with triangle inequality has PTAS (Arora's algorithm for Euclidean TSP). EPTAS: stronger, time $O(n^{g(\epsilon)})$ for some function g .
46. **Q:** Explain the analysis of the dynamic table using the potential method. **Key points:** Potential function $\Phi(T) = 2 \cdot \text{num} - \text{size}$. $\Phi_{\text{initial}} = 0$. Before doubling: $\text{size} = \text{num} = m \rightarrow \Phi = m$. After doubling: $\text{size} = 2m$, $\text{num} = m+1 \rightarrow \Phi = 2(m+1) - 2m = 2$. $\Delta\Phi = 2 - 2m$. Actual cost of doubling: $m+1$ (copy m elements + insert). Amortized cost: $\hat{c} = (m+1) + (2 - 2m) = 3 - m + 2 = \dots$ wait, recompute: $\hat{c} = \text{actual} + \Delta\Phi = (m+1) + (2 - 2m) = 3 - m$. That can't be right. Let's redo proper: $\Phi(D_i) = 2 \cdot \text{num}_i - \text{size}_i$. When no doubling: $\text{num}_i = \text{num}_{i-1} + 1$, $\text{size}_i = \text{size}_{i-1}$. $\Delta\Phi = 2(\text{num}_{i-1} + 1 - \text{size}_{i-1}) - 2(\text{num}_{i-1} - \text{size}_{i-1}) = 2$. $\hat{c} = 1 + 2 = 3$. When doubling: $\text{num}_i = \text{num}_{i-1} + 1$, $\text{size}_i = 2 \cdot \text{size}_{i-1} = 2 \cdot \text{num}_{i-1}$. $\Phi_{i-1} = 2 \cdot \text{num}_{i-1} - \text{size}_{i-1} = 2 \cdot \text{num}_{i-1} - \text{num}_{i-1} = \text{num}_{i-1}$. $\Phi_i = 2 \cdot (\text{num}_{i-1} + 1) - 2 \cdot \text{num}_{i-1} = 2$. $\Delta\Phi = 2 - \text{num}_{i-1}$. $\hat{c} = (\text{num}_{i-1} + 1) + (2 - \text{num}_{i-1}) = 3$. Amortized $O(1)$ per insertion.
47. **Q:** Describe the relationship between linear programming duality and the max-flow min-cut theorem. **Key points:** Max flow can be formulated as a linear program: maximize flow subject to capacity and conservation constraints. The dual LP corresponds to finding a minimum cut. Specifically, dual variables on vertices give a cut capacity. Strong duality ensures optimal flow = optimal cut. This gives an

alternative proof of max-flow min-cut. Karp's result: strong separation oracle for dual polyhedron is equivalent to min-cut computation. This connection generalizes to multicommodity flow and sparsest cut.

48. **Q:** Compare the structure and properties of red-black trees and AVL trees. **Key points:** Balance: RB trees guarantee $h \leq 2 \cdot \lg(n+1)$; AVL trees guarantee $h \leq 1.44 \cdot \lg(n+1)$ (tighter). Insertion: RB trees $O(1)$ amortized rotations (at most 2); AVL trees $O(\lg n)$ rotations (in worst case). Deletion: RB trees $O(1)$ amortized (at most 3 rotations); AVL trees $O(\lg n)$. Lookup: AVL trees $O(\lg n)$ with better constants due to tighter balance. Practical: RB trees used in most language libraries (C++ map, Java TreeMap); AVL trees preferred for lookup-intensive workloads (databases). Both use similar coloring/balance strategies but different invariants.
49. **Q:** Explain the concept of the AC-0 circuit class and its role in computational complexity. **Key points:** AC-0: problems solvable by constant-depth, polynomial-size circuits with AND/OR gates of unbounded fan-in. Examples: PARITY is NOT in AC-0 (proven by Furst-Saxe-Sipser, Håstad). AC-0 has low expressive power. SMajority is not in AC-0, but is in TC-0 (with majority gates). The depth hierarchy: AC¹ has logarithmic-depth polynomial-size circuits. NC¹ has $O(\lg n)$ -depth bounded fan-in circuits. NC: problems solvable with polylogarithmic depth and polynomial size (efficient parallel computation). Relevance: shows limitations of constant-depth parallel computation.
50. **Q:** Discuss the use of randomization in online algorithms and how it can break deterministic lower bounds. **Key points:** Deterministic paging: no algorithm can beat k -competitive (k = cache size). Randomized Marking: $O(\lg k)$ -competitive. The improvement comes from making the adversary's job harder — a randomized algorithm's behavior is unpredictable. For a given input sequence, the expected cost may be lower than the worst-case cost over random choices. Lower bound for randomized paging: $\Omega(\lg k)$ (by Rabani et al.). Ski-rental: deterministic $2-1/B$, randomized $e/(e-1) \approx 1.582$. For list accessing, BIT (randomized) achieves 1.75-competitive vs 2 lower bound for deterministic.
51. **Q:** Explain the concept of a binary decision diagram (BDD) and its applications. **Key points:** BDD: a directed acyclic graph representation of a boolean function. Variables are ordered; each node represents a variable with two outgoing edges (0 and 1). Leaves are 0 and 1. Reduced-ordered BDD (ROBDD): canonical form for a given variable ordering — two ROBDDs are isomorphic iff they represent the same function. Properties: function composition, quantification, and satisfiability check all polynomial in BDD size. Applications: circuit verification, symbolic model checking (McMillan), CAD tools. Limitation: size can be exponential in variable count for some functions (e.g., multiplication output bits).
52. **Q:** Describe the quicksort analysis using indicator random variables. **Key points:** Let X = total comparisons in quicksort. $X = \sum_i \sum_{\{j>i\}} X_{\{ij\}}$, where $X_{\{ij\}} = 1$ if z_i and z_j are compared directly. Two

elements are compared iff one is the first pivot chosen from $Z_{\{ij\}}$. $\Pr[X_{\{ij\}}=1] = 2/(j-i+1)$. By linearity: $E[X] = \sum_{\{i=1\}^{\{n-1\}}} \sum_{\{j=i+1\}^{\{n\}}} 2/(j-i+1) = \sum_{\{i=1\}^{\{n-1\}}} \sum_{\{k=2\}^{\{n-i+1\}}} 2/k \leq \sum_{\{i=1\}^{\{n-1\}}} 2 \cdot H_n = 2(n-1) \cdot H_n = O(n \lg n)$. Each comparison costs $O(1)$, and partition cost $O(n)$ per level, so total expected $O(n \lg n)$. This matches the more intuitive analysis using expected recurrence.

53. **Q:** Compare and contrast Bellman-Ford and Dijkstra for single-source shortest paths. **Key points:** Dijkstra: greedy, requires nonnegative weights, $O(E \lg V)$ with binary heap, cannot detect negative cycles. Correctness relies on the fact that extracted vertex has final shortest-path weight. Bellman-Ford: scans all edges each pass, $O(VE)$, handles negative weights, can detect negative cycles. In practice, Dijkstra is used when edge weights are nonnegative (most routing scenarios). Bellman-Ford is used for graphs known to have or suspected of having negative edges. SPFA (queue-based variant) is often faster in practice but has worst-case $O(VE)$.
54. **Q:** Analyze the expected performance of randomized quicksort as a function of input distribution. **Key points:** Randomized quicksort gives $O(n \lg n)$ expected time regardless of input distribution. This is because the randomness is in the algorithm, not in the input — the expectation is over the algorithm's random choices, not the input. Even for adversarially chosen input, expected time is $O(n \lg n)$. In contrast, deterministic quicksort depends on input: sorted input gives $\Theta(n^2)$ worst-case. The distinction: randomized algorithm achieves good expected performance against ANY input; average-case analysis assumes a specific input distribution.
55. **Q:** Explain the relationship between the RSA cryptosystem and the integer factorization problem. **Key points:** RSA security relies on the assumption that factoring large composite numbers (products of two large primes) is computationally hard. If factoring is easy, RSA is broken (since we can compute $\phi(n)$ from $n=pq$). The converse (if RSA is broken, factoring is easy) is not known to be true. There exist attacks on RSA that don't require factoring (e.g., low-exponent attacks, side-channel attacks). Best factoring algorithms: General Number Field Sieve (GNFS) runs in $\sim \exp(O((\lg n)^{1/3}(\lg \lg n)^{2/3}))$ time. For 2048-bit RSA, this is computationally infeasible. Shor's algorithm factors in polynomial time on a quantum computer.
56. **Q:** Discuss the concept of a deterministic finite automaton-based string matching and its space complexity. **Key points:** Build a DFA from pattern P : states $0..m$ (m = pattern length). State q means longest prefix of P matched so far. On character c : transition to longest prefix of P that is suffix of $P[1:q]+c$. Precomputation: for each state q ($0..m$) and each character c in Σ , compute $\delta(q,c)$. Building transitions: $O(m|\Sigma|)$ time. Matching: $O(n)$ time — process text one character at a time, follow transitions. If state = m , pattern ends at current position. Space: $\Theta(m|\Sigma|)$ for transition table. For large alphabets (Unicode), this is impractical. KMP achieves same runtime with $O(m)$ space by not storing transitions for all characters.

57. **Q:** Analyze the relationship between suffix trees, suffix arrays, and the LCP array. **Key points:** Suffix tree: compact trie of all suffixes, $O(n)$ nodes/edges, substring search in $O(m)$. Suffix array: sorted order of suffixes, $O(n)$ space ($4n$ bytes). LCP array: $LCP[i]$ = longest common prefix between suffix at rank i and $i-1$. Suffix array + LCP array can simulate many suffix tree operations. Construction: suffix tree $\rightarrow$ suffix array via DFS ($O(n)$); suffix array $\rightarrow$ LCP via Kasai's algorithm ($O(n)$); suffix array $\rightarrow$ suffix tree via LCP ($O(n)$). Applications: pattern matching in $O(m + \lg n)$, longest repeated substring (max LCP), longest common substring (max LCP across two strings).
58. **Q:** Compare the computational complexity of matrix multiplication with that of matrix inversion. **Key points:** Matrix multiplication and matrix inversion have the same asymptotic complexity (up to constant factors). Strassen algorithm: multiply in $O(n^{\{2.81\}})$, invert by multiply-based methods (solve $Ax = b$ via Gaussian elimination-style reduction). The equivalence: Inversion $\leq_p$ Multiplication: block matrix inversion using Schur complement requires two multiplications and two inversions of half-size matrices $\rightarrow T(n) = 2T(n/2) + O(M(n)) \rightarrow T(n) = O(M(n))$. Multiplication $\leq_p$ Inversion: multiply $A \times B$ by inverting $[I, A; 0, I]^{\{-1\}}$ $[I, 0; -B, I]$ or using $[I, A, 0; 0, I, B; 0, 0, I]^{\{-1\}}$. So ω = exponent for both problems.
59. **Q:** Explain the adversarial lower bound for the minimum element problem. **Key points:** To find minimum of n elements using comparisons, at least $n-1$ comparisons are needed. Proof: adversary maintains possible minima set. Initially all n elements are potential minima. Each comparison between elements x and y : if $x > y$, x can no longer be minimum (adversary declares $y < x$). At least $n-1$ elements must lose at least once. Each comparison can eliminate at most one element from being minimum (the loser). So at least $n-1$ comparisons needed. Optimal algorithm: sequential tournament ($n-1$ comparisons). Extension to min+max: $3\lceil n/2 \rceil - 2$ lower bound via tournament pairing.
60. **Q:** Discuss the relationship between Nisan-Wigderson pseudorandom generators (PRGs) and circuit lower bounds. **Key points:** NW PRG: if there exists an EXP function with hardness $2^{\{\Omega(n)\}}$ (no circuit of size $2^{\{\delta n\}}$ computes it), then for any $\varepsilon > 0$, there exists a PRG stretching n bits to n^ε bits that fools circuits of size $n^{\{\varepsilon'\}}$ (for some ε'). Key idea: the hard function's truth table is used as the generator's seed. Hardness = unpredictability against small circuits. Conversely, if PRGs exist, then circuit lower bounds follow (derandomization implies circuit lower bounds — Karp-Lipton, Impagliazzo-Wigderson). Connection: strong enough derandomization collapses P and BPP, but requires circuit lower bounds assumptions.
61. **Q:** Explain why the Ford-Fulkerson algorithm may fail to terminate when capacities are irrational. **Key points:** Ford-Fulkerson: while augmenting path exists in residual network, augment flow. For integer capacities: always terminates in at most $|f^*| \leq C \cdot |E|$ iterations ($C = \max$ capacity). For rational capacities: multiply by LCM to make integer. For irrational capacities: there exist pathological examples where Ford-

Fulkerson runs forever, converging to a non-maximum flow. Example (Zwick, 1995): carefully constructed network with exponentially growing denominators. Edmonds-Karp (BFS-based) always terminates because each augmentation increases flow while distances strictly increase, bounding iterations by $O(VE)$. Dinic's algorithm also terminates regardless of capacity values.

62. **Q:** Compare the entropy-based analysis of Huffman coding with Shannon-Fano coding. **Key points:** Shannon's source coding theorem: optimal compression achieves average code length between $H(p)$ and $H(p)+1$ where $H(p) = -\sum p_i \lg p_i$. Huffman code: optimal prefix code (minimum weighted external path length). Shannon-Fano: partition symbols into two groups with approximately equal probability, assign leading bit 0/1, recurse. Neither always achieves Shannon bound, but Huffman is optimal. For symbol probabilities that are powers of $1/2$, Huffman achieves exactly $H(p)$. The redundancy (average length - $H(p)$) is at most 1 for Huffman, can be reduced to $\leq 1/m$ by coding m -symbol blocks (extended source). Arithmetic coding beats the per-symbol bound by using fractional bits.
63. **Q:** Explain the concept of BPP and the relationship between randomness and deterministic computation. **Key points:** BPP = problems solvable by randomized polynomial-time algorithms with error $\leq 1/3$. Amplification: run k times, take majority — error $\leq 2^{-\Omega(k)}$ for two-sided error. RP: one-sided error 0 for NO, $\leq 1/2$ for YES. co-RP: complementary. ZPP: zero-error, expected polynomial time = $RP \cap co-RP$. Known: $P \subseteq ZPP \subseteq RP \subseteq BPP$, $P \subseteq co-RP \subseteq BPP$. Derandomization: if EXP requires super-polynomial circuit size, then $P = BPP$ (Impagliazzo-Wigderson). Unconditionally: $BPP \subseteq P/poly$ (Adleman's theorem) and $BPP \subseteq PH \subseteq PSPACE$. Practical: randomized algorithms (Miller-Rabin, Quicksort, Karger's min-cut) are essential despite possibility of derandomization.
64. **Q:** Analyze the competitive ratio of the Move-to-Front (MTF) list accessing heuristic. **Key points:** MTF: after accessing an element, move it to the front of the list. Potential function: Φ = number of inversions — pairs (x,y) where x precedes y in MTF list but y precedes x in optimal (static) list. Initial Φ : at most $C(n,2)$. Each access: MTF pays c (position in list). After move-to-front, at most $c-1$ inversions are destroyed (elements before x that should be after x) and at most $c-1$ new inversions created (elements before x that should be before x — but after the move they're after x). Actually: elements before x : a_j that are before x in optimal list $\rightarrow$ NO inversion created; a_j after x in optimal $\rightarrow$ INVERSION destroyed. Elements after x : create at most $c-1$ inversions. Net: amortized cost $\leq 3 \cdot c \cdot OPT - 2$. This gives competitive ratio 3. Improved analysis shows actual ratio is 2.
65. **Q:** Discuss the concept of linear programming relaxation for NP-hard problems and provide examples. **Key points:** Integer Linear Programming (ILP) is NP-hard. LP relaxation: drop integrality constraints to get a polynomial-time solvable problem. The LP optimum provides a bound on the ILP optimum: for minimization, $LP_{opt} \leq$

ILP_opt; for maximization, $LP_opt \geq ILP_opt$. Rounding: convert fractional solution to integer solution with performance guarantee. Examples: (1) Vertex Cover: ILP $\min \sum x_i$ s.t. $x_i + x_j \geq 1$ for each edge, $x_i \in \{0,1\}$. LP relaxation gives half-integral solution. Rounding threshold $1/2$ gives 2-approximation. (2) Set Cover: LP rounding gives $O(\lg n)$ -approximation. (3) Max-SAT: random 0/1 rounding gives $3/4$ -approximation (Goemans-Williamson). (4) Facility location: LP rounding achieves 1.5-approximation via filtering technique.

66. **Q:** Prove that sorting networks work using the 0-1 Sorting Lemma. **Key points:** A sorting network is a sequence of comparators (i,j) that compare and swap elements at positions i,j . 0-1 Sorting Lemma: an oblivious comparison-exchange network that sorts all binary sequences (0s and 1s) also sorts all arbitrary sequences. Proof: if a monotone function f (like threshold) is applied to a sequence, the network processes the transformed sequence identically. Since any incorrect order after sorting an arbitrary sequence would survive a threshold transformation to produce a binary sequence that the network fails to sort, contradiction. Application: proving bitonic sort, odd-even merge sort, etc., are correct by testing only 0-1 sequences.
67. **Q:** Discuss the relationship between consensus (distributed computing) and leader election. **Key points:** Consensus: n processes each propose a value, must agree on a single value (agreement, validity, termination). Leader election: special case of consensus where processes agree on who is leader. FLP impossibility: in an asynchronous system with even one crash failure, consensus cannot be solved deterministically. Paxos/Raft: practical consensus algorithms in partially synchronous systems. Leader election: Bully algorithm (highest-ID process becomes leader), Ring algorithm (processes pass election messages around ring). Connection: consensus protocols often elect a leader first (viewstamped replication, Raft). Leader-based consensus simplifies agreement: proposer sends value to leader, leader broadcasts to acceptors.
68. **Q:** Describe how the cuckoo hashing data structure works and analyze its performance. **Key points:** Cuckoo hashing: two hash tables T_1, T_2 with hash functions h_1, h_2 . Insert: try $T_1[h_1(x)]$, if occupied, evict y , put x in T_1 , try inserting y into $T_2[h_2(y)]$, repeat. If cycle detected (too many evictions), rehash with new hash functions. Search: check $O(1)$ positions ($T_1[h_1(x)]$ and $T_2[h_2(x)]$) — deterministic $O(1)$ worst-case lookup. Deletion: $O(1)$. Analysis: with load factor $\alpha < 1/2$, expected insertion time $O(1)$ with high probability. Space: $2n/\alpha$ (wasteful but fast). Variants: d -ary cuckoo hashing (d hash tables), cuckoo filters (Bloom filter alternative). Application: high-performance hash tables where lookup speed is critical.
69. **Q:** Explain the simplex algorithm for linear programming and its geometric interpretation. **Key points:** Simplex algorithm: start at a vertex (basic feasible solution) of the feasible polytope defined by $Ax = b, x \geq 0$. At each iteration, move along an edge to an adjacent vertex that improves the objective. Edge direction: increasing a nonbasic variable while keeping others at 0. Pivot: variable enters basis

(increases from 0), another leaves (decreases to 0). Degeneracy: when more than n constraints pass through a vertex, causing potential cycling. Bland's rule avoids cycling. Geometric interpretation: optimal vertex occurs at intersection of n constraints. The path follows edges of the polytope. Exponential worst-case (Klee-Minty cube), but practical for most problems.

70. **Q:** Compare backtracking, branch-and-bound, and dynamic programming for combinatorial optimization. **Key points:** Backtracking: systematic enumeration with pruning when partial solution cannot be extended to feasible solution. No guarantee of optimality bounds. Example: N-Queens, Sudoku. Branch-and-bound: backtracking with lower/upper bound pruning — discard branches whose bound is worse than current best bound. Guarantees optimality if bounds are valid. Example: TSP, integer programming. Dynamic programming: optimal substructure + overlapping subproblems, builds table of subproblem solutions. Guarantees optimality when DP applies. Example: knapsack, LCS, edit distance. Choose DP when optimal substructure holds and state space is manageable; choose branch-and-bound for NP-hard problems with good bounds; choose backtracking for small constraint satisfaction.
71. **Q:** Prove that the Vertex Cover problem is NP-complete. **Key points:** (1) VERTEX-COVER $\in$ NP: certificate = set of k vertices. Verifier checks each edge (u,v) : if $u \notin C$ and $v \notin C$, reject. $O(|E|)$ time. (2) Reduce CLIQUE $\leq_p$ VERTEX-COVER: given instance (G,k) of CLIQUE, produce $\bar{G}$ (complement of G), $k' = |V| - k$. (3) G has clique of size $k \Leftrightarrow \bar{G}$ has vertex cover of size $|V|-k$. Proof: $S \subseteq V$ is a clique in $G \Leftrightarrow$ every non-edge in $\bar{G}$ has at least one endpoint outside $S \Leftrightarrow V \setminus S$ is a vertex cover in $\bar{G}$. (4) Reduction is polynomial: $\bar{G}$ computed by complementing adjacency matrix. (5) Since CLIQUE is NP-complete and reduces to VERTEX-COVER, VERTEX-COVER is NP-complete.
72. **Q:** Analyze the expected performance of hash table with open addressing using linear probing. **Key points:** Load factor $\alpha = n/m < 1$ required. Expected number of probes: unsuccessful search $\leq 1/(1-\alpha)$, successful search $\leq (1/\alpha) \cdot \ln(1/(1-\alpha))$. For $\alpha = 0.5$: unsuccessful ≤ 2 , successful ≤ 1.39 . For $\alpha = 0.9$: unsuccessful ≤ 10 , successful ≤ 2.56 . Primary clustering: consecutive occupied slots form long runs, degrading performance. Expected probe count for linear probing with $\alpha = 0.9$ is worse than double hashing (where it's $-\ln(1-\alpha)/\alpha \approx 2.56$ for both). Double hashing: $\Theta(m^2)$ different probe sequences avoid clustering. Uniform hashing assumption: each probe sequence equally likely. In practice: use linear probing for $\alpha \leq 2/3$ with good hash function; double hashing for higher load.
73. **Q:** Discuss the relationship between the Karatsuba multiplication algorithm and the divide-and-conquer paradigm. **Key points:** Karatsuba multiplies two n -digit numbers in $O(n^{\lg 3}) \approx O(n^{1.585})$ time. Standard: 4 recursive multiplications of $n/2$ -digit numbers $\rightarrow T(n) = 4T(n/2) + O(n) = \Theta(n^2)$. Karatsuba: rewrite $a \cdot b = (a_1 \cdot 10^{\lfloor n/2 \rfloor} + a_0) \cdot (b_1 \cdot 10^{\lfloor n/2 \rfloor} + b_0) = z_2 \cdot 10^n + z_1 \cdot 10^{\lfloor n/2 \rfloor} + z_0$,

where $z_2 = a_1 \cdot b_1$, $z_0 = a_0 \cdot b_0$, and $z_1 = (a_1 + a_0) \cdot (b_1 + b_0) - z_2 - z_0$. Only 3 multiplications: $T(n) = 3T(n/2) + O(n) = \Theta(n^{\lg 3})$. Generalized divide-and-conquer reduces 4 to 3 multiplications. Toom-Cook further generalizes to k -way split (5 vs 3 for 3-way). Schönhage-Strassen: $O(n \lg n \lg \lg n)$ using FFT. FFT-based multiplication: $O(n \lg n)$ for large n .

74. **Q:** Explain the concept of the PCP theorem and its implications for inapproximability. **Key points:** PCP Theorem (Arora-Safra, Arora-Lund-Motwani-Sudan-Szegedy): $NP = PCP(O(\lg n), O(1))$. Every NP problem has a probabilistically checkable proof where the verifier reads only $O(\lg n)$ random bits and $O(1)$ bits of the proof. Interpretation: NP problems have proofs checkable with constant query complexity and logarithmic randomness. Implication for approximation: MAX-3-SAT has no PTAS unless $P=NP$. In fact, any MAX-SNP problem has a constant inapproximability threshold. The PCP theorem shows that approximate versions of NP-complete problems can be as hard as exact ones. UGC (Unique Games Conjecture) sharpens these thresholds: e.g., Max-Cut is NP-hard to approximate better than ~ 0.878 (Goemans-Williamson threshold).
75. **Q:** Derive the expected number of comparisons in RANDOMIZED-QUICKSORT and prove its optimality. **Key points:** Let C_n = expected comparisons for n elements. $C_0 = C_1 = 0$. For $n \geq 2$: $C_n = n-1 + (1/n) \cdot \sum_{k=1}^{n-1} (C_{k-1} + C_{n-k})$. By symmetry: $\sum_{k=1}^{n-1} C_{k-1} = \sum_{k=1}^{n-1} C_{n-k}$. So $n \cdot C_n = n(n-1) + 2 \cdot \sum_{k=1}^{n-1} C_{k-1}$. Substitute $n-1$: $(n-1) \cdot C_{n-1} = (n-1)(n-2) + 2 \cdot \sum_{k=1}^{n-2} C_k$. Subtract: $n \cdot C_n - (n-1) \cdot C_{n-1} = 2(n-1) + 2 \cdot C_{n-1}$. So $n \cdot C_n = (n+1) \cdot C_{n-1} + 2(n-1)$. Divide by $n(n+1)$: $C_n/(n+1) = C_{n-1}/n + 2(n-1)/(n(n+1))$. Summation: $C_n/(n+1) = 2 \cdot \sum_{k=2}^n (k-1)/(k(k+1)) \approx 2 \cdot \ln n + O(1)$. Therefore $C_n \approx 2n \ln n \approx 1.39 \cdot n \lg n$. This is asymptotically optimal (within constant of $\Omega(n \lg n)$ lower bound).

Reference Appendices

Appendix A — Recurrence Master Table

Recurrence	Algorithm	Solution
$T(n) = 2T(n/2) + \Theta(n)$	Merge sort	$\Theta(n \lg n)$
$T(n) = T(n-1) + \Theta(n)$	Quicksort worst-case	$\Theta(n^2)$
$T(n) = 2T(n/2) + \Theta(1)$	Binary search	$\Theta(\lg n)$
$T(n) = 8T(n/2) + \Theta(1)$	Recursive mat mult	$\Theta(n^3)$
$T(n) = 7T(n/2) + \Theta(n^2)$	Strassen	$\Theta(n^{\lg 7})$
$T(n) = 9T(n/3) + \Theta(n)$	Example	$\Theta(n^2)$
$T(n) = T(2n/3) + \Theta(1)$	MAX-HEAPIFY	$\Theta(\lg n)$
$T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$	SELECT	$\Theta(n)$

Recurrence	Algorithm	Solution
$T(n) = 2T(n/2) + n \lg n$	Quicksort tied	$\Theta(n \lg^2 n)$
$T(n) = 3T(n/4) + n \lg n$	Example	$\Theta(n \lg n)$
$T(n) = T(n/3) + T(2n/3) + \Theta(n)$	Unbalanced D&C	$\Theta(n \lg n)$
$T(n) = 4T(n/2) + \Theta(n)$	Example	$\Theta(n^2)$
$T(n) = 2T(n/2) + \Theta(1)$	Binary search	$\Theta(\lg n)$
$T(n) = T(n/4) + T(3n/4) + \Theta(n)$	Example	$\Theta(n \lg n)$
$T(n) = 2T(n-1) + 1$	Towers of Hanoi	$\Theta(2^n)$
$T(n) = aT(n-b) + f(n)$	Decrease-and-conquer	depends on a,b
$T(n) = 2T(\sqrt{n}) + \lg n$	Variable substitution	$\Theta(\lg n \cdot \lg \lg n)$
$T(n) = T(n^2) + O(n)$	Nonexistent	Must check: $n^2 > n \rightarrow T$ undefined
$T(n) = 1T(n-1) + 1$	Ackermann initialization	$\Theta(n)$

Appendix B — Asymptotic Notation Cheat Sheet

Notation	Formal Definition	Intuition	Limit Test
$f = \Theta(g)$	$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for $n \geq n_0$	$f \approx g$ (tight bound)	$\lim f/g = c > 0$
$f = O(g)$	$0 \leq f(n) \leq c g(n)$ for $n \geq n_0$	$f \leq g$ (upper bound)	$\lim \sup f/g < \infty$
$f = \Omega(g)$	$0 \leq c g(n) \leq f(n)$ for $n \geq n_0$	$f \geq g$ (lower bound)	$\lim \inf f/g > 0$
$f = o(g)$	$\forall c > 0 \exists n_0: 0 \leq f(n) < c g(n)$ for $n \geq n_0$	$f < g$ (not tight)	$\lim f/g = 0$
$f = \omega(g)$	$\forall c > 0 \exists n_0: 0 \leq c g(n) < f(n)$ for $n \geq n_0$	$f > g$ (not tight)	$\lim f/g = \infty$

Common pitfalls: - $O(n^2) = \Theta(n^2)$ is FALSE: O is upper bound, Θ is tight bound. $2n = O(n^2)$ but $2n \neq \Theta(n^2)$. - Using Θ for worst-case time is wrong if best and worst differ: insertion sort worst $\Theta(n^2)$, best $\Theta(n)$, overall not $\Theta(n^2)$. - Asymptotic notation in equations: $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ means $\exists f(n) \in \Theta(n)$ such that $2n^2 + 3n + 1 = 2n^2 + f(n)$. - **Sums:** If $\sum f(k) = \Theta(\sum g(k))$, combine carefully; the “hidden” constant must work for all terms.

Growth hierarchy (each dominates previous): - $O(1) \subset O(\lg n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \lg n) \subset O(n\sqrt{n}) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$

Functions with multiple variables: - $f(m,n) = O(g(m,n))$ if $\exists c, m_0, n_0: f(m,n) \leq c \cdot g(m,n)$ for all $m \geq m_0, n \geq n_0$ - Careful with ordering: $O(m+n) \neq O(\max(m,n))$ in general (though often same). For graphs: $O(V+E)$ is standard.

Limit-based comparison rules: - $f = o(g) \Leftrightarrow \lim_{n \rightarrow \infty} \{f(n)/g(n)\} = 0$ - $f = \Theta(g) \Leftrightarrow 0 < \lim_{n \rightarrow \infty} \{f(n)/g(n)\} < \infty$ (limit exists) - $f = \omega(g) \Leftrightarrow \lim_{n \rightarrow \infty} \{f(n)/g(n)\} = \infty$ - L'Hôpital: if $f, g \rightarrow \infty$, $\lim f/g = \lim f'/g'$ (when limit exists)

Appendix C — Common Graph Algorithm Complexities

Algorithm	Time	Space	Notes
BFS	$O(V+E)$	$O(V)$	Unweighted shortest paths
DFS	$O(V+E)$	$O(V)$	Edge classification, topological order
Topological sort (DFS)	$\Theta(V+E)$	$O(V)$	DAG only
Topological sort (Kahn)	$\Theta(V+E)$	$O(V)$	In-degree removal
SCC (Kosaraju)	$\Theta(V+E)$	$O(V)$	Two DFS passes
SCC (Tarjan)	$\Theta(V+E)$	$O(V)$	Single pass, lowlinks
MST (Kruskal)	$O(E \lg V)$	$O(V)$	Sort + union-find
MST (Prim binary heap)	$O(E \lg V)$	$O(V)$	DECREASE-KEY $\times E$
MST (Prim Fibonacci)	$O(E + V \lg V)$	$O(V)$	Faster for dense
MST (Prim array)	$O(V^2)$	$O(V)$	Best for dense graphs
Bellman-Ford	$O(VE)$	$O(V)$	Negative edges OK
BFS for unweighted SP	$O(V+E)$	$O(V)$	Unweighted
DAG Shortest Paths	$\Theta(V+E)$	$O(V)$	Topological sort + relax
Dijkstra (binary heap)	$O(E \lg V)$	$O(V)$	Nonnegative only
Dijkstra (Fibonacci)	$O(V \lg V + E)$	$O(V)$	Theoretical best
Dijkstra (array)	$O(V^2)$	$O(V)$	For dense graphs
Floyd-Warshall	$\Theta(V^3)$	$\Theta(V^2)$	Dense, all-pairs
Johnson	$O(V^2 \lg V + VE)$	$O(V^2)$	Sparse, negative edges OK
FASTER-APSP (squaring)	$\Theta(V^3 \lg V)$	$\Theta(V^2)$	min-plus matrix mult
Transitive closure (boolean FW)	$\Theta(V^3)$	$\Theta(V^2)$	Boolean matrix
Ford-Fulkerson	$O(E \cdot f^*)$	f^*	)
Edmonds-Karp	$O(VE^2)$	$O(V+E)$	BFS augmenting paths
Push-relabel (generic)	$O(V^2E)$	$O(V+E)$	Local operations
Push-relabel (FIFO)	$O(V^3)$	$O(V+E)$	Practical
Dinic	$O(V^2E)$	$O(V+E)$	Layered networks
Capacity scaling	$O(E^2 \lg C)$	$O(V+E)$	$C = \max$ capacity
Hopcroft-Karp	$O(\sqrt{V} \cdot E)$	$O(V+E)$	Bipartite matching
Gale-Shapley	$O(n^2)$	$O(n^2)$	Stable marriage
Hungarian	$O(n^3)$	$O(n^2)$	Assignment problem
Maximum bipartite matching (flow)	$O(VE)$	$O(V+E)$	Via Dinic on unit cap

Appendix D — Data Structure Operation Complexities

Structure	SEARCH	INSERT	DELETE	MIN/ MAX	SUCCESSOR	EXTRACT- MIN	DECREASE- KEY
Sorted array	$O(\lg n)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Unsorted array	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Singly linked list	$O(n)$	$O(1)$	$O(n)^\dagger$	$O(n)$	$O(n)$	$O(n)$	$O(1)^\dagger^\ddagger$
Doubly linked list	$O(n)$	$O(1)$	$O(1)^\dagger$	$O(n)$	$O(n)$	$O(n)$	$O(1)^\dagger^\ddagger$
Stack (array)	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	—
Queue (array)	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(1)$	—
BST (unbalanced)	$O(h)$	$O(h)$	$O(h)$	$O(h)$	$O(h)$	$O(h)$	$O(h)$
BST (balanced/ AVL)	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$
RB tree	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$
B-tree (min deg t)	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$
Hash (chaining, $\alpha = n/m$)	$O(1 + \alpha)$ avg	$O(1)$	$O(1)^\dagger$	$O(n)$	$O(n)$	$O(n)$	$O(1)^\dagger^\ddagger$
Hash (open addr)	$O(1/(1-\alpha))$ avg	—	$O(1)$ marked	$O(n)$	$O(n)$	—	—
Binary heap (min)	$O(n)$	$O(\lg n)$	$O(\lg n)^\dagger^\ddagger^\ddagger^\ddagger$	$O(1)$	$O(n)$	$O(\lg n)$	$O(\lg n)$
d-ary heap (min)	$O(n)$	$O(\log_d n)$	$O(\log_d n)$	$O(1)$	$O(n)$	$O(d \log_d n)$	$O(\log_d n)$
Fibonacci heap (min)	$O(n)$	$O(1)$	$O(\lg n)^\dagger^\ddagger^\ddagger^\ddagger$	$O(1)$	$O(n)$	$O(\lg n)$ amort	$O(1)$ amort
Disjoint-set (forest)	—	$O(1)^\bullet$	—	—	—	—	—
van Emde Boas tree	$O(\lg \lg u)$	$O(\lg \lg u)$	$O(\lg \lg u)$	$O(1)$	$O(\lg \lg u)$	$O(\lg \lg u)$	—

† With pointer to element. ‡ With pointer, key known. $^\ddagger^\ddagger^\ddagger$ With pointer and heap-decrease-key. $^\bullet$ MAKE-SET is $O(1)$. FIND-SET and UNION are $O(\alpha(n))$ amortized with both heuristics.

Note on hash table deletions: Open addressing requires DELETED marker (slows search). Chaining deletion is $O(1)$ with doubly linked list.

Appendix E — Sorting Algorithm Properties

Algorithm	Worst	Avg	Best	Space	Stable	In-place	Comparisons	Memory
Insertion	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$O(1)$	Yes	Yes	$\Theta(n^2)$ worst	Incremental
Selection	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No	Yes	$\Theta(n^2)$ all	Selection
Bubble	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$O(1)$	Yes	Yes	$\Theta(n^2)$ worst	Exchange
Merge	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$O(n)$	Yes	No	$\Theta(n \lg n)$	Divide & Conquer
Heapsort	$O(n \lg n)$	$O(n \lg n)$	$\Omega(n \lg n)$	$O(1)$	No	Yes	$O(n \lg n)$	Selection
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$O(\lg n)^*$	No	Yes	$\Theta(n \lg n)$ avg	Divide & Conquer
Introspective	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	$O(\lg n)$	No	Yes	$O(n \lg n)$	Hybrid
Shell	$O(n^2)$	$O(n^{\{3/2\}})$	$O(n \lg n)$	$O(1)$	No	Yes	varies	Incremental
Timsort	$O(n \lg n)$	$O(n \lg n)$	$O(n)$	$O(n)$	Yes	No	$O(n \lg n)$	Hybrid
Counting	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$O(k)$	Yes	No	0 (non-comp)	Non-comparative
Radix (LSD)	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$O(n+k)$	Yes	No	0	Non-comparative
Bucket	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	Yes	No	0 (expected)	Distribution

*Quicksort stack depth $O(\lg n)$ with tail-recursion elimination; $O(n)$ without.

- **Stable sort guarantees:** When equal keys maintain original relative order.

- **Adaptive:** Insertion, Bubble, Timsort are adaptive (near $\Theta(n)$ for nearly sorted). - **Internal vs external:** Merge sort used for external sorting (tape drives).

Appendix F — Important Graph Properties

Property	Statement	Application
Handshaking Lemma	$\sum \deg(v) = 2 E $	Graph invariants
Euler's formula (planar)	$V - E + F = 2$	Planar graph bounds
Graph connectivity	$\kappa(G) \leq \lambda(G) \leq \delta(G)$	Vertex/edge connectivity
Turán's theorem	max edges in K_{r+1} -free graph: $(1-1/r) \cdot n^2/2$	Extremal graph theory
Mantel's theorem	Triangle-free graph has $\leq n^2/4$ edges	Special case of Turán
Triangle inequality	$\delta(s,v) \leq \delta(s,u) + w(u,v)$	Shortest paths
Directed graph DAG	If G is a DAG $\implies \exists$ topological ordering	Scheduling, dependencies
Undirected DFS	Only tree and back edges	Edge classification
Directed DFS	4 edge types (tree, back, forward, cross)	Cycle detection
Cut property	Lightest edge across cut respecting A is safe	MST

Property	Statement	Application
Cycle property	Heaviest edge on any cycle is in no MST	MST
Max-flow min-cut	Max flow = min cut capacity	Network flow
Hall's theorem	Perfect matching iff $ A \leq N(A) \forall A \subseteq L$	Bipartite matching
König's theorem	In bipartite: max matching = min vertex cover	Equivalence
Berge's lemma	M maximum $\iff$ no M-augmenting path	Matching
Dilworth's theorem	In poset: max antichain size = min chain decomposition	Partial orders
Brooks' theorem	$\chi(G) \leq \Delta(G)$ (except cliques/odd cycles)	Graph coloring
Vizing's theorem	$\Delta(G) \leq \chi'(G) \leq \Delta(G)+1$	Edge coloring

Appendix G — Key Mathematical Formulas

Series & Summations: - **Arithmetic series:** $\sum_{i=1}^n i = n(n+1)/2$ -

Sum of squares: $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$ - **Sum of cubes:**

$\sum_{i=1}^n i^3 = n^2(n+1)^2/4$ - **Geometric series:** $\sum_{i=0}^n c^i =$

$(c^{n+1} - 1)/(c-1)$ for $c \neq 1$ - **Infinite geometric ($c < 1$):** $\sum_{i=0}^{\infty} c^i =$

$1/(1-c)$ - **Arithmetic-geometric:** $\sum_{i=1}^n i \cdot c^i = (c - (n+1)c^{n+1} + nc^{n+2})/(1-c)^2$ - **Sum of $i \cdot 2^i$:** $\sum_{i=1}^n i \cdot 2^i = (n-1)2^{n+1} + 2$

- **Sum of 2^i :** $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ - **Harmonic numbers:** $H_n =$

$\sum_{i=1}^n 1/i = \ln n + \gamma + O(1/n)$, $\gamma \approx 0.57721$ - **Negative powers:**

$\sum_{i=1}^{\infty} 1/i^2 = \pi^2/6$

Logarithms: - **Binary:** $\lg n = \log_2 n$ - **Natural:** $\ln n = \log_e n$ - **Change of**

base: $\log_a n = \log_b n / \log_b a = \Theta(\log n)$ - **Iterated log:** $\lg^* n = \min\{i \geq$

$0: \lg^{(i)} n \leq 1\}$ - **Properties:** $\log(ab) = \log a + \log b$; $\log(a^b) = b \cdot \log a$;

$a^{\log_b c} = c^{\log_b a}$

Combinatorics: - **Binomial coefficient:** $C(n,k) = n!/(k!(n-k)!) = C(n, n-k)$ -

Stirling's approximation: $n! = \sqrt{2\pi n}(n/e)^n(1 + \Theta(1/n))$ - **Catalan**

numbers: $C_n = (1/(n+1)) \cdot C(2n, n) = \Omega(4^{n/2} \{3/2\})$ - **Binomial theorem:**

$(x+y)^n = \sum_{k=0}^n C(n,k) \cdot x^k \cdot y^{n-k}$ - **Pascal's identity:** $C(n,k) =$

$C(n-1,k-1) + C(n-1,k)$ - **Vandermonde:** $C(m+n,r) = \sum_{k=0}^r C(m,k) \cdot C(n,r-k)$

- **Stars and bars:** $C(n+k-1, k-1)$ ways to place n identical items into k bins

Number Theory: - **Fibonacci:** $F_i = (\phi^i - \bar{\phi}^i)/\sqrt{5}$, $F_0=0$, $F_1=1$ - **Golden**

ratio: $\phi = (1+\sqrt{5})/2 \approx 1.61803$, $\bar{\phi} = (1-\sqrt{5})/2 \approx -0.618$ - **gcd properties:**

$\gcd(a,b) = \gcd(b, a \bmod b)$; $\gcd(ka,kb) = k \cdot \gcd(a,b)$ - **lcm:** $\text{lcm}(a,b) = ab/$

$\gcd(a,b)$ - **Euler's totient:** $\phi(n) = n \cdot \prod_{p|n} (1-1/p)$ - **Fermat's theorem:**

$a^{\phi(n)} \equiv 1 \pmod{p}$ for prime $p \nmid a$ - **Euler's theorem:** $a^{\phi(n)} \equiv 1 \pmod{n}$

n) for $\gcd(a,n)=1$ - **Chinese Remainder**: $x \equiv a_i \pmod{n_i}$, n_i coprime $\rightarrow$ unique $x \pmod{\prod n_i}$

Probability: - **Union bound**: $\Pr[\cup A_i] \leq \sum \Pr[A_i]$ - **Conditional**: $\Pr[A|B] = \Pr[A \cap B] / \Pr[B]$ - **Bayes**: $\Pr[A|B] = \Pr[B|A] \cdot \Pr[A] / \Pr[B]$ - **Expectation**: $E[X] = \sum x \cdot \Pr[X=x]$; $E[g(X)] = \sum g(x) \cdot \Pr[X=x]$ - **Linearity**: $E[\sum X_i] = \sum E[X_i]$ (no independence) - **Variance**: $\text{Var}[X] = E[(X-\mu)^2] = E[X^2] - (E[X])^2$ - **Covariance**: $\text{Cov}[X,Y] = E[XY] - E[X]E[Y]$ - **Markov**: $\Pr[X \geq t] \leq E[X]/t$ for $X \geq 0$ - **Chebyshev**: $\Pr[|X-\mu| \geq k\sigma] \leq 1/k^2$ - **Chernoff bound**: $\Pr[X \leq (1-\delta)\mu] \leq e^{-\mu\delta^2/2}$ for $\delta \in [0,1]$; $\Pr[X \geq (1+\delta)\mu] \leq e^{-\mu\delta^2/3}$ - **Birthday**: $\Pr[\text{no collision}] \approx e^{-k(k-1)/(2n)}$; threshold: $k \approx \sqrt{2n \ln 2}$ - **Coupon collector**: Expected $b \cdot H_b \approx b \cdot \ln b$ trials to collect all b coupons

Inequalities & Bounds: - **Cauchy-Schwarz**: $(\sum a_i b_i)^2 \leq (\sum a_i^2)(\sum b_i^2)$ - **Jensen**: $f(E[X]) \leq E[f(X)]$ for convex f - **Boole (union bound)**: $\Pr[\cup A_i] \leq \sum \Pr[A_i]$ - **Floor/ceil**: $x-1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$; $\lfloor x/2 \rfloor + \lceil x/2 \rceil = x$ - **Sum integral**: $\int_{\{0\}^n} f(x) dx \leq \sum_{i=1}^n f(i) \leq \int_{\{1\}^{n+1}} f(x) dx$ for monotone f - **Bernoulli inequality**: $(1+x)^n \geq 1+nx$ for $x \geq -1$, $n \geq 0$ - **AM $\geq$ GM**: $(\sum a_i)/n \geq (\prod a_i)^{1/n}$ for nonnegative a_i

Asymptotic Bounds (for algorithm analysis): - $\lg(n!) = n \lg n - n \lg e + O(\lg n)$ - $\Theta(n \lg n)$ - $H_n = \ln n + \gamma + O(1/n)$ - $\sum_{i=1}^n 1/i = \ln n + \gamma + \varepsilon_n$ where $\varepsilon_n \rightarrow 0$ - $\sum_{i=1}^n i^p = \Theta(n^{p+1})$ for $p > -1$ - $\sum_{i=1}^n 1/i^p = \Theta(1)$ for $p > 1$; $= \Theta(\lg n)$ for $p = 1$; $= \Theta(n^{1-p})$ for $p < 1$

Appendix H — Dynamic Programming Recurrence Summary

Problem	Recurrence	Base Case	Time	Space
Rod Cutting	$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$	$r_0 = 0$	$\Theta(n^2)$	$\Theta(n)$
Matrix-Chain	$m[i,j] = \min_{i \leq k < j} (m[i,k] + m[k+1,j] + p_{i-1} p_k p_j)$	$m[i,i] = 0$	$\Theta(n^3)$	$\Theta(n^2)$
LCS	$c[i,j] = c[i-1,j-1] + 1$ if $x_i = y_j$; else $\max(c[i-1,j], c[i,j-1])$	$c[0,j] = c[i,0] = 0$	$\Theta(mn)$	$\Theta(mn)$
OBST	$e[i,j] = \min_{i \leq r \leq j} (e[i,r-1] + e[r+1,j] + w[i,j])$	$e[i,i-1] = e[i+1,i] = 0$	$\Theta(n^3)$	$\Theta(n^2)$
LIS	$dp[i] = 1 + \max_{j < i, A[j] < A[i]} dp[j]$	$dp[i] = 1$	$O(n^2)$	$O(n)$
Edit Distance	$dp[i,j] = \min(dp[i-1,j] + 1, dp[i,j-1] + 1, dp[i-1,j-1] + \delta(x_i \neq y_j))$	$dp[i,0] = i, dp[0,j] = j$	$\Theta(mn)$	$\Theta(mn)$
0-1 Knapsack	$dp[i,w] = \max(dp[i-1,w], dp[i-1,w-w_i] + v_i)$	$dp[0,w] = dp[i,0] = 0$	$O(nW)$	$O(nW)$
Floyd-Warshall	$d^{\{(k)\}}[i,j] = \min(d^{\{(k-1)\}}[i,j], d^{\{(k)\}}[i,k] + d^{\{(k-1)\}}[k,j])$	$d^{\{(0)\}}[i,j] = w(i,j)$	$\Theta(V^3)$	$\Theta(V^2)$

Problem	Recurrence	Base Case	Time	Space
Bellman-Ford	$d^{\{(k-1)\}}[i,k] + d^{\{(k-1)\}}[k,j]$ $d^{\{(i)\}}[v] = \min(d^{\{(i-1)\}}[v], \min_u \{d^{\{(i-1)\}}[u] + w(u,v)\})$	$d^{\{(0)\}}[s]=0, \text{ else } \infty$	$O(VE)$	$O(V)$
LUP Solve	$Ly=Pb$ (forward), $Ux=y$ (back)	—	$\Theta(n^2)$	$\Theta(n^2)$
Convolution	$c_k = \sum_{i+j=k} a_i \cdot b_j$	—	$O(n^2)/O(n \lg n)$	$O(n)$
Subset Sum (DP)	$P[i,s] = P[i-1,s] \vee P[i-1,s-x_i]$	$P[0,0]=\text{true}$	$O(n \cdot \text{target})$	$O(\text{target})$

Appendix I — Data Structures: When to Use Which

Need	Recommended DS	Reasoning
Fast search, ordered	Balanced BST (RB, AVL)	$O(\lg n)$ search/successor/predecessor
Fast search, unordered	Hash table (chaining)	$O(1+\alpha)$ average, good constants
LIFO	Stack (array)	$O(1)$ push/pop, simple
FIFO	Queue (circular array)	$O(1)$ enqueue/dequeue
Repeated minimum extraction	Min-heap	$O(1)$ min, $O(\lg n)$ extract
Priority queue + DECREASE-KEY	Fibonacci heap	$O(1)$ DECREASE-KEY amortized for Dijkstra/Prim
Dynamic graph connectivity	Disjoint-set forest	$O(\alpha(n))$ per op, near-linear
Order-statistic queries	OS tree (RB + subtree sizes)	$O(\lg n)$ SELECT/RANK
Interval overlap queries	Interval tree (RB + max endpoint)	$O(\lg n)$ find overlapping
Range sum queries (dynamic)	Fenwick tree (BIT)	$O(\lg n)$ update/query, minimal memory
Range min/query (static)	Sparse table	$O(1)$ query, $O(n \lg n)$ build
Disk-optimized dictionary	B-tree	High branching $\rightarrow O(\log_t n)$ disk accesses

Need	Recommended DS	Reasoning
String dictionary (static)	Trie / Patricia trie	$O(m)$ per operation (m = string length)
Small universe (integers)	van Emde Boas tree	$O(\lg \lg u)$ per operation
LRU cache	Doubly linked list + hash map	$O(1)$ get/put
Text pattern matching	Suffix array + LCP	$O(n)$ build, $O(m + \lg n)$ search

Appendix J — Important NP-Completeness Reductions

Chapter 2: Theorem 2.1 (any comparison sort can count inversions in $O(n \lg n)$) **Chapter 3:** Theorem 3.1 ($\Theta = O \cap \Omega$) **Chapter 4:** Master Theorem (4.1); Akra-Bazzi Theorem (4.2) **Chapter 5:** Lemma 5.4 (RANDOMLY-PERMUTE produces uniform permutations) **Chapter 7:** Expected comparisons of RANDOMIZED-QUICKSORT = $O(n \lg n)$ **Chapter 8:** Theorem 8.1 ($\Omega(n \lg n)$ comparison sort lower bound) **Chapter 9:** Theorem 9.1 (SELECT runs in $\Theta(n)$ worst-case) **Chapter 11:** Theorem 11.4 (H_{pm} universal); Theorem 11.9 (perfect hashing $O(n)$ space) **Chapter 12:** Theorem 12.4 (expected height of randomly built BST = $O(\lg n)$) **Chapter 13:** Lemma 13.1 (RB tree height $\leq 2 \lg(n+1)$) **Chapter 15:** Theorem 15.1 (greedy activity selection optimal); Theorem 15.4 (Huffman optimal prefix-free code); Theorem 15.5 (furthest-in-future optimal caching) **Chapter 17:** Theorem 17.1 (augmentable if f depends on $O(1)$ children); Theorem 17.2 (INTERVAL-SEARCH correctness) **Chapter 18:** Theorem 18.1 (B-tree height bound) **Chapter 20:** Theorem 20.5 (BFS shortest paths); Theorem 20.6 (Parenthesis); Theorem 20.7 (White-path) **Chapter 21:** Theorem 21.1 (Cut property) **Chapter 22:** Lemma 22.2 (Bellman-Ford correctness); Theorem 22.10 (Dijkstra correctness) **Chapter 24:** Theorem 24.6 (Max-flow min-cut); Theorem 24.10 (Integrality) **Chapter 25:** Theorem 25.8 (Gale-Shapley produces stable matching) **Chapter 30:** Theorem 30.2 (Convolution Theorem via FFT) **Chapter 31:** Theorem 31.27 (Chinese Remainder); Theorem 31.31 (Fermat) **Chapter 32:** Theorem 32.4 (KMP correctness) **Chapter 34:** Theorem 34.7 (CIRCUIT-SAT is NP-complete — Cook-Levin)

Appendix I — Important NP-Completeness Reductions

Reduction Graph ($L_1 \rightarrow L_2$ means $L_1 \leq_p L_2$):

```

CIRCUIT-SAT
  → SAT
    → 3-CNF-SAT
      → CLIQUE
        → VERTEX-COVER
          → HAM-CYCLE
            → TSP
          → SUBSET-SUM
        → 3-COLOR

```

Source Problem	Target Problem	Gadget/Technique	Key Idea
SAT	3-CNF-SAT	Parse tree + auxiliary vars	Convert each clause to CNF, then 3-CNF
3-CNF-SAT	CLIQUE	Vertices per literal, edges between non-conflicting literals in different clauses	$k = \# \text{clauses}$; clique size $k \Leftrightarrow$ satisfying assignment
3-CNF-SAT	SUBSET-SUM	Base-10 encoding, variable + slack numbers	Each clause/digit position ensures satisfaction
3-CNF-SAT	3-COLOR	OR-gadgets, palette triangle	Encode variables as TRUE/FALSE vertices
CLIQUE	VERTEX-COVER	Complement graph	G has k -clique $\Leftrightarrow \bar{G}$ has
CLIQUE	INDEPENDENT-SET	Same graph	G has k -clique $\Leftrightarrow \bar{G}$ has k independent set
VERTEX-COVER	HAM-CYCLE	Selector paths for edges, cover vertices	Tour visits exactly one endpoint per edge gadget
HAM-CYCLE	TSP	Complete graph, cost 0 if edge in G	HAM-CYCLE in $G \Leftrightarrow$ TSP tour cost ≤ 0
SUBSET-SUM	PARTITION	Add target sum element	Partition exists with specific sum
3-CNF-SAT	HAM-PATH	Complex widget per clause	Path through clause widget if literal true
VERTEX-COVER	DOMINATING-SET	Add degree-1 vertices to enforce constraint	VC of size $k \Leftrightarrow$ DS of size k
3-SAT	MAX-2-SAT	Clause splitting	Exact reduction

Source Problem	Target Problem	Gadget/Technique	Key Idea
			showing MAX-2-SAT is NP-hard

Appendix K — Pseudo-Code Library: Sorting

INSERTION-SORT(A, n)

```

for i = 2 to n
    key = A[i]
    j = i - 1
    while j > 0 and A[j] > key
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = key

```

MERGE-SORT(A, p, r)

```

if p < r
    q = ⌊(p+r)/2⌋
    MERGE-SORT(A, p, q)
    MERGE-SORT(A, q+1, r)
    MERGE(A, p, q, r)

```

HEAPSORT(A, n)

```

BUILD-MAX-HEAP(A, n)
for i = n down to 2
    swap A[1] with A[i]
    heap-size = heap-size - 1
    MAX-HEAPIFY(A, 1)

```

QUICKSORT(A, p, r)

```

if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q-1)
    QUICKSORT(A, q+1, r)

```

COUNTING-SORT(A, n, k)

```

let C[0:k] = 0
for i = 1 to n    C[A[i]]++
for i = 1 to k    C[i] = C[i] + C[i-1]
let B[1:n]
for i = n down to 1
    B[C[A[i]]] = A[i]
    C[A[i]]--
return B

```

Appendix L — Pseudo-Code Library: Graph Algorithms

BFS(G, s)

```
for each  $u \in G.V - \{s\}$ :  $u.color = WHITE$ ,  $u.d = \infty$ ,  $u.\pi = NIL$ 
 $s.color = GRAY$ ,  $s.d = 0$ ,  $s.\pi = NIL$ 
 $Q = \emptyset$ ; ENQUEUE( $Q$ ,  $s$ )
while  $Q \neq \emptyset$ 
     $u = DEQUEUE(Q)$ 
    for each  $v \in G.Adj[u]$ 
        if  $v.color == WHITE$ 
             $v.color = GRAY$ ;  $v.d = u.d + 1$ ;  $v.\pi = u$ ; ENQUEUE( $Q$ ,  $v$ )
     $u.color = BLACK$ 
```

DFS(G)

```
for each  $u \in G.V$ :  $u.color = WHITE$ ,  $u.\pi = NIL$ 
time = 0
for each  $u \in G.V$ 
    if  $u.color == WHITE$     DFS-VISIT( $G$ ,  $u$ )
```

BELLMAN-FORD(G , w , s)

```
INITIALIZE-SINGLE-SOURCE( $G$ ,  $s$ )
for  $i = 1$  to  $|G.V| - 1$ 
    for each  $(u, v) \in G.E$     RELAX( $u$ ,  $v$ ,  $w$ )
for each  $(u, v) \in G.E$ 
    if  $v.d > u.d + w(u, v)$     return FALSE
return TRUE
```

DIJKSTRA(G , w , s)

```
INITIALIZE-SINGLE-SOURCE( $G$ ,  $s$ )
 $S = \emptyset$ ;  $Q = G.V$ 
while  $Q \neq \emptyset$ 
     $u = EXTRACT-MIN(Q)$ 
     $S = S \cup \{u\}$ 
    for each  $v \in G.Adj[u]$     RELAX( $u$ ,  $v$ ,  $w$ )
```

Appendix N — Key NP-Complete Problems Reference

Problem	Input	Question	Classic Reduction From
CIRCUIT-SAT	Boolean circuit	Is there a satisfying assignment?	(First NPC problem)
SAT	CNF formula	Is formula satisfiable?	CIRCUIT-SAT
3-CNF-SAT	3-CNF formula	Is formula satisfiable?	SAT
CLIQUE	Graph G , integer k	Does G contain a	3-CNF-SAT

Problem	Input	Question	Classic Reduction From
VERTEX-COVER	Graph G, integer k	clique of size $\geq k$? Does G contain a vertex cover of size $\leq k$?	CLIQUE
INDEPENDENT-SET	Graph G, integer k	Does G contain an independent set of size $\geq k$?	CLIQUE
HAM-CYCLE	Graph G	Does G contain a Hamiltonian cycle?	VERTEX-COVER
HAM-PATH	Graph G, vertices s,t	Is there a Hamiltonian path from s to t?	HAM-CYCLE
TSP (decision)	Complete graph G, costs, budget B	Is there a tour of cost $\leq B$?	HAM-CYCLE
SUBSET-SUM	Set of integers S, target t	Does some subset sum to t?	3-CNF-SAT
PARTITION	Set of integers S	Can S be partitioned into two equal-sum subsets?	SUBSET-SUM
3-COLOR	Graph G	Can G be properly 3-colored?	3-CNF-SAT
3-D MATCHING	Sets X,Y,Z, triples $T \subseteq X \times Y \times Z$	Is there a matching covering all elements?	3-CNF-SAT
HITTING SET	Sets $S_1, \dots, S_m$, integer k	Is there a set of size $\leq k$ intersecting all?	VERTEX-COVER
KNAPSACK (decision)	Items (v_i, w_i) , capacity W, target V	Can value $\geq V$ be achieved within capacity?	SUBSET-SUM
SET PACKING			CLIQUE

Problem	Input	Question	Classic Reduction From
	Sets $S_1, \dots, S_m$, integer k	Are there k pairwise disjoint sets?	
SET COVER	Universe U , sets S_i , integer k	Can k sets cover all elements?	VERTEX-COVER
BIN PACKING	Items of size s_i , bin capacity B , k bins	Can items fit in k bins?	PARTITION
INTEGER PROGRAMMING	Matrix A , vectors b, c	Is there $x \in \mathbb{Z}^n$ with $Ax \leq b$, $c^T x \geq$ target?	SAT
STEINER TREE	Graph G , terminals T , budget B	Is there tree spanning T with cost $\leq B$?	VERTEX-COVER

Appendix O — Big-O Functions Family Tree

$0(1) \subset 0(\lg n) \subset 0(\sqrt{n}) \subset 0(n) \subset 0(n \lg n) \subset 0(n^2) \subset 0(n^3) \subset 0(2^n) \subset 0(n!)$

Common functions in increasing order: | Function | Name | Example
 Algorithm | |-----|-----|-----| | $\Theta(1)$ | Constant | Array access, PUSH/
 POP | | $\Theta(\lg n)$ | Logarithmic | Binary search, heap operations | | $\Theta(\sqrt{n})$ |
 Square root | Elementary factoring, grid search | | $\Theta(n)$ | Linear | Minimum,
 counting sort ($k=O(n)$), BFS/DFS | | $\Theta(n \lg n)$ | Linearithmic | Merge sort,
 heapsort, quicksort (avg) | | $\Theta(n\sqrt{n})$ | $n \cdot \text{sqrt}$ | Some geometric algorithms | |
 $\Theta(n^2)$ | Quadratic | Insertion sort (worst), naive multiplication | | $\Theta(n^3)$ |
 Cubic | Floyd-Warshall, matrix-chain DP | | $\Theta(2^n)$ | Exponential | Subset
 generation, naive Fibonacci | | $\Theta(n!)$ | Factorial | Permutation generation,
 brute-force TSP | | $\Theta(\lg \lg n)$ | Log-log | van Emde Boas operations | | $\Theta(\lg^2 n)$ |
 Log squared | P-MERGE span, optimal BST | | $\Theta(\alpha(n))$ | Inverse
 Ackermann | Disjoint-set FIND-SET ($\alpha(n) \leq 4$) | | $\Theta(\lg^* n)$ | Iterated log |
 Union-find without rank ($\lg^* n \leq 5$) |

Growth comparison (for $n = 10^6$): $-\lg n \approx 20$ - $\sqrt{n} \approx 1000$ - $n = 10^6$ - $n \lg n \approx 20 \cdot 10^6$ - $n^2 = 10^{12}$ - $n^3 = 10^{18}$ (1 billion billion) - 2^n is infeasible for $n > 60$

Appendix P — Algorithm Design Techniques Cheat Sheet

Technique	When to Use	How	Examples
Brute Force	Small input size	Enumerate all possibilities	Subset generation, TSP ($n \leq 10$)

Technique	When to Use	How	Examples
Divide & Conquer	Independent subproblems	Split, solve recursively, combine	Merge sort, quicksort, FFT, Strassen
Decrease & Conquer	One subproblem	Reduce to smaller instance	Binary search, insertion sort, Euclid
Dynamic Programming	Overlapping subproblems	Memoization or bottom-up table	LCS, matrix-chain, Knapsack
Greedy	Greedy-choice property	Make locally optimal choice	Activity selection, Huffman, Prim
Iterative Improvement	Feasible to optimal	Augment along improving directions	Simplex, Ford-Fulkerson, Hopcroft-Karp
Divide & Conquer + DP	Tree-like structure	DP on binary decomposition	Floyd-Warshall, matrix-chain
Transform & Conquer	Nonstandard form	Reweight, reduce, change representation	Johnson's algorithm, Horner's rule
Binary Search	Monotonic predicate	Check midpoint, eliminate half	Binary search, binary search on answer
Two Pointers	Sorted array traversal	Move pointers toward solution from ends	Hoare partition, 3SUM, merge
Sliding Window	Subarray/substring constraints	Maintain window, expand/shrink	Rabin-Karp, minimum window substring
Union-Find	Dynamic connectivity	Same-set queries, incremental connectivity	Kruskal, connected components
Sweep Line	Geometric problems	Process events in order	Line segment intersection, skyline
Randomized	Large input, need speed	Use randomness for expected guarantees	RANDOMIZED-QUICKSORT, universal hashing
Backtracking	Constraint satisfaction	Recursive enumeration with pruning	N-Queens, graph coloring, SAT
Branch & Bound	Optimization with constraints	Branch with upper/lower	TSP, integer programming

Technique	When to Use	How	Examples
Approximation	NP-hard optimization	bound pruning Guarantee within factor of optimal	Set cover, vertex cover
Local Search	Large combinatorial space	Random initial solution, iteratively improve	k-Means, hill climbing, simulated annealing

Appendix R — Algorithm Families by Problem Domain

Sorting: Insertion (small/nearly sorted), Merge (stable, external), Heap (worst-case guarantee), Quick (fast average), Counting (small integer range), Radix (fixed-length integer keys), Bucket (uniform floats), Timsort (Python’s hybrid), IntroSort (C++ hybrid)

Searching: Linear (unsorted), Binary (sorted array), BST (dynamic), RB/AVL (balanced dynamic), Hash (unordered, fast), B-tree (disk), Suffix tree/array (string), van Emde Boas (integer universe)

Graph Traversal: BFS (shortest path unweighted), DFS (connectivity, topological, SCC), Bidirectional BFS (faster path finding), A* (heuristic shortest path), IDA* (memory-limited A*), Dijkstra (nonnegative shortest paths), Bellman-Ford (negative edges)

Minimum Spanning Tree: Kruskal (edge-based, forest), Prim (vertex-based, tree), Borůvka (parallel MST), Reverse-delete (delete heaviest cycle edges)

Shortest Path: BFS (unweighted), DAG-SP (DAG, negative OK), Dijkstra (nonnegative), Bellman-Ford (negative OK), Floyd-Warshall (all-pairs dense), Johnson (all-pairs sparse), A* (heuristic), Bidirectional Dijkstra (faster unidirectional)

Maximum Flow: Ford-Fulkerson (general, exponential), Edmonds-Karp (BFS, $O(VE^2)$), Dinic (layered, $O(V^2E)$), Push-Relabel (local, $O(V^3)$), Capacity Scaling ($O(E^2 \lg C)$), Micali-Vazirani (general matching, $O(\sqrt{VE})$)

Dynamic Programming: Fibonacci (linear 1D), Rod cutting (linear 1D), LCS (2D), Matrix-chain (interval 2D), Knapsack (bounded 2D), Edit distance (2D), Floyd-Warshall (all-pairs), OBST (interval, Knuth optimization), DP on trees, DP on DAG

Number Theory: Euclid (gcd), Extended Euclid (gcd+coefficients), Modular exponentiation (square-and-multiply), Miller-Rabin (primality), AKS (deterministic primality), Sieve of Eratosthenes (prime enumeration), Baby-step giant-step (discrete log)

String Matching: Naive (no preprocessing), Rabin-Karp (rolling hash), KMP (prefix function), BMH (bad character rule), Finite automaton

(transition table), Suffix array (offline), Suffix tree (full indexing), Z-algorithm (linear pattern matching)

Appendix S — Common Pitfalls & Exam Tips

Asymptotic Analysis: - Don't assume O = worst-case, Ω = best-case. $O/\Omega/\Theta$ describe function growth, not case. - $f(n) = O(n^2)$ does NOT mean $f(n) = \Theta(n^2)$. Always use tightest bound when possible. - Summation: $\sum_{i=1}^n O(f(i)) \neq O(\sum f(i))$ — the constant may vary per term. - When analyzing loops: multiply loop bounds only for independent nested loops (not dependent like triangle loops).

Data Structures: - BST operations are $O(h)$, not $O(\lg n)$. Only balanced BSTs guarantee $O(\lg n)$. - Hash tables with chaining: expected search $O(1+\alpha)$, $\alpha=n/m$. Make $\alpha < 1$ for good performance. - Open addressing requires load factor $\alpha < 1$; typical target $\alpha < 0.75$ for linear probing. - Heap's BUILD-MAX-HEAP is $O(n)$, not $O(n \lg n)$. The tighter analysis uses sum over heights. - Priority queue: EXTRACT-MIN is $O(\lg n)$; FIND-MIN is $O(1)$.

Recurrences: - Master Theorem Case 2: $f(n) = \Theta(n^{\log_b a} \lg^k n) \rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$. Don't forget the extra $\lg$ factor. - Master Theorem Case 3: must CHECK regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ for $c < 1$. - Master Theorem doesn't apply if $f(n)$ is not polynomially smaller/larger (e.g., $n/\lg n$ between cases 1 and 2).

Graph Algorithms: - BFS gives shortest paths in UNWEIGHTED graphs (not weighted). - Dijkstra fails with negative edges — even a single negative edge can cause wrong output. - Bellman-Ford detects only negative cycles REACHABLE from s . - Topological sort works only on DAGs. - Ford-Fulkerson: integer capacities give integer flow (integrality theorem). - MST: cut property chooses light edge crossing ANY cut that respects A .

NP-Completeness: - Proving NP-completeness: (1) Show $L \in \text{NP}$ (certificate + verifier). (2) Reduce known NPC problem to L . Direction MUST be from known NPC to your problem. - Optimization problems (TSP-OPT) are NP-hard but not NP-complete (not in NP). - $\text{SAT} \leq_p \text{CLIQUE}$ means "if CLIQUE is easy, SAT is easy." Contrapositive: if SAT is hard, CLIQUE is hard.

DP & Greedy: - DP subproblems must overlap. If they don't, D&C suffices. - Greedy requires proof of optimality (exchange argument or matroid). Guessing doesn't count. - 0-1 Knapsack is NP-hard (no polynomial algorithm). Fractional Knapsack is P (greedy). - Optimal substructure $\neq$ greedy-choice property (e.g., 0-1 Knapsack has optimal substructure but not greedy-choice).

Online Algorithms: - Competitive ratio is worst-case over ALL inputs, not average. - Lower bounds for online algorithms often use adversarial input construction. - Randomized online algorithms can beat deterministic lower bounds (e.g., paging: k vs $O(\lg k)$).

Parallel Algorithms: - Work T_1 should be asymptotically same as best serial algorithm for efficiency. - Span T_∞ determines maximum speedup regardless of processors. - Greedy scheduler achieves $T_P \leq T_1/P + T_\infty$. - Amdahl's Law: speedup limited by serial fraction.

Asymptotic Notation: - Don't say "insertion sort is $\Theta(n^2)$ " — it's $\Omega(n)$ best case. Be specific: " $\Theta(n^2)$ worst-case" or " $O(n^2)$ ". - $O(f(n)) \neq \Theta(f(n))$. Saying " $O(\Theta(n^2))$ " is meaningless. - In limits: if limit doesn't exist, notation may not apply (e.g., $n^{(1+\sin n)}$).

Recurrences: - Master Theorem doesn't apply when $f(n)$ falls between cases (e.g., $n/\lg n$ between Cases 1 and 2). - Always check regularity condition for Case 3. - Base cases matter: $T(1)=\Theta(1)$ is standard.

Data Structures: - BST: worst-case height is $O(n)$ for unbalanced. "Average" BST $\neq$ randomly built BST. - Hash tables: $\alpha = n/m$. For chaining, search = $\Theta(1+\alpha)$. For open addressing, MUST have $\alpha < 1$. - RB trees: insert fixup has at most 2 rotations; delete fixup has at most 3. - B-trees: minimum degree t , NOT the number of keys. Root can have as few as 1 key.

Graph Algorithms: - Dijkstra: fails with negative edges (even without negative cycles). - Bellman-Ford: must check ALL edges each pass, not just outgoing from changed vertices. - BFS: distances measured in number of edges, not weights. - Topological sort: requires DAG. Run DFS first to detect cycles. - Flow: forward edges get +flow, backward edges get -flow.

Sorting: - Counting sort: only for integers, requires knowing k . - Radix sort: LSD-first needs stable digit sort. MSD-first works lexicographically. - Quick sort: Lomuto partition with all equal elements = $\Theta(n^2)$.

NP-Completeness: - NP $\neq$ "not polynomial." NP = nondeterministic polynomial time (verifiable in poly time). - NP-hard $\neq$ NP-complete (NP-hard may not be in NP). - Optimization problems (TSP-OPT) are NP-hard, not NP-complete.

General Exam Strategy: - For "analyze an algorithm" questions: identify the model (RAM), count operations, find dominant term. - For "prove correctness": write loop invariant, show initialization/maintenance/termination. - For "design an algorithm": state the approach clearly, analyze complexity, prove correctness. - For comparison questions: organize by dimension (time, space, stability, constraints). - Trace problems: show intermediate states clearly, label each step.

Appendix T — Quick Reference: Algorithm Selection by Problem Type

If you need...	Use...	Because...
Shortest path, unweighted	BFS	$O(V+E)$, simple, optimal
Shortest path, nonnegative	Dijkstra (binary heap)	$O((V+E) \lg V)$, optimal for sparse

If you need...	Use...	Because...
Shortest path, negative edges	Bellman-Ford	Detects negative cycles, $O(VE)$
All-pairs shortest, dense	Floyd-Warshall	$\Theta(V^3)$, simple, no negative cycles
All-pairs shortest, sparse	Johnson's	$O(V^2 \lg V + VE)$, reweighting
Sorting, general purpose	Quicksort (randomized)	$\Theta(n \lg n)$ expected, in-place
Sorting, guaranteed worst-case	Heapsort / Mergesort	$\Theta(n \lg n)$ worst-case
Sorting, stable / external	Mergesort	Stable, $O(n)$ merge
Sorting, small range integers	Counting sort	$O(n+k)$, linear
Sorting, fixed-length keys	Radix sort	$O(d(n+k))$, linear
MST	Kruskal (sparse) or Prim (dense)	$O(E \lg V)$ or $O(V^2)$
Maximum flow	Dinic	$O(V^2E)$, fast for unit networks
String matching	KMP	$O(n+m)$ worst-case, no backtracking
Pattern matching, many patterns	Aho-Corasick	$O(n+m+k)$, automaton of patterns
Dynamic connectivity	Union-Find	$O(\alpha(n))$ amortized per op
Dictionary, unordered	Hash table (chaining)	$O(1)$ expected per op
Dictionary, ordered	Red-black tree	$O(\lg n)$ per op, balanced guaranteed
Large on-disk data	B-tree	Minimizes disk I/O, large fanout
Discrete optimization, optimal	Dynamic programming	Exploits optimal substructure
Discrete optimization, fast	Greedy	Locally optimal decisions
NP-hard optimization	Approximation algorithm	Guaranteed factor of optimal
Parallel sorting	P-MERGE	$O(n \lg n)$ work, $O(\lg^2 n)$ span
Integer factorization	Pollard's rho	$O(n^{\{1/4\}})$ expected
Convex hull	Graham scan	$O(n \lg n)$
Linear programming	Simplex (practical) / Ellipsoid (theoretical)	Polynomial-time theoretical, fast practice
Primality testing	Miller-Rabin	Randomized $O(\lg^3 n)$
RSA encryption	Modular exponentiation	$O(\lg n)$ multiplications
Unsorted order statistics	SELECT (quickselect)	$O(n)$ worst-case / expected

If you need...	Use...	Because...
Transitive closure	Floyd-Warshall variant	Boolean matrix multiplication
Pattern recognition / clustering	k-Means	Simple, $O(nkt)$ iterations
Online decision making	Weighted Majority	$O(\lg n)$ regret against best expert
Large-scale optimization	Gradient Descent / SGD	$O(1/\epsilon)$ iterations, scalable
Sequence alignment	Needleman-Wunsch (global) / Smith-Waterman (local)	$O(nm)$ time, DP-based

Appendix U — Important Open Problems in Algorithms

Problem	Description	Known Best	Significance
P vs NP	Can every efficiently verifiable problem be solved efficiently?	$P \subseteq NP$; widely believed $P \neq NP$	Most famous open problem in CS; 1M Clay Prize
Matrix Multiplication Exponent ω	Minimum ω such that $n \times n$ mat mult is $O(n^{\{\omega\}})$	$\omega < 2.37287$; $\omega \geq 2$	Closing gap would revolutionize linear algebra
Unique Games Conjecture	Is there a 2-prover game with hardness of approximation?	Implies tight inapprox results for Max-Cut, Vertex Cover	Would complete UGC-hardness picture
Graph Isomorphism	Is GI solvable in polynomial time?	Quasipolynomial (Babai 2015)	One of few natural problems in NP-intermediate
Integer Factorization	Can $n = p \cdot q$ be factored in polynomial time?	Subexponential (GNFS)	Would break RSA completely
Deterministic Parallel Algorithms	Is $P = NC$?	$NC \subseteq P$; $P \subseteq NC$ is open	Would problems have fast parallel solutions?
Exact Exponential Algorithms	Can 3-SAT be solved in $O(1.99^n)$ time?	Best: $O(1.307^n)$ by PPSZ	ETH implies $\Omega(2^{\{cn\}})$ for 3-SAT
Unique Games Conjecture	Is UGC true?	Related to small-set expansion	Would settle hardness of many approximation problems
Dynamic Optimality Conjecture	Is splay tree $O(\lg n)$ per operation?	Conjectured but unproven	Would explain splay tree performance
Log-Rank Conjecture		Open since 1988	Connection to circuit lower bounds

Problem	Description	Known Best	Significance
NEXP vs P/poly	Is deterministic comm complexity polylog of rank? Are problems doubly-exponential in NP in P/poly?	Open	Implies $EXP \neq NEXP$
BPP vs P	Can every randomized poly-time algorithm be derandomized?	Believed $BPP = P$; follows from circuit lower bounds	Would eliminate need for randomness in efficient algorithms
Approximate Closest Vector	Is there a polynomial-time algorithm for constant-factor CVP?	Exponential in dimension	Lattice problems; related to crypto security
VC Dimension Learning	Is PAC learning with respect to distribution-free model efficiently possible?	Open	Foundation of computational learning theory

Appendix V — Frequently Confused Concepts

Concept A	Concept B	Key Difference
$O(f)$ (upper bound)	$\Theta(f)$ (tight bound)	O is a bound from above; Θ is both upper and lower
Worst-case analysis	Upper bound (O)	Worst-case is about input difficulty; O is about function growth
Running time of insertion sort	$\Omega(n)$ best, $\Theta(n^2)$ worst	The function is not $\Theta(n^2)$ overall; specify per-case
P vs NP vs NP-complete vs NP-hard	$P \subseteq NP$; NPC hardest in NP; NP-hard may not be in NP	NP does NOT mean “not polynomial”
Decision problem vs optimization	Decision: yes/no; Optimization: find best	NP-completeness applies to decision versions
AVL vs Red-Black trees	AVL tighter balance ($h \leq 1.44 \lg n$); RB ($h \leq 2 \lg n$)	AVL faster lookup; RB faster insert/delete
Max flow vs Min cut	Max flow = min cut value	The value equals, but the sets differ
Primal vs Dual LP	Dual variables = shadow prices; dual constraints bound primal objective	Weak duality ($\text{primal} \leq \text{dual}$) always; strong at optimality
Dijkstra vs Prim		

Concept A	Concept B	Key Difference
	Both greedy, both use priority queue	Dijkstra: sum from source; Prim: minimum spanning tree
Topological sort (DFS) vs Kahn's algorithm	DFS uses finish times; Kahn uses in-degrees	Both $O(V+E)$; different approaches
Counting sort vs Radix sort	Counting sort on one pass; Radix sort applies counting sort per digit	Radix extends counting sort to multi-digit numbers
Memoization vs Tabulation	Memoization: top-down recursion + caching; Tabulation: bottom-up iteration	Both DP; tabulation avoids recursion overhead
DP vs Divide & Conquer	DP: overlapping subproblems; D&C: disjoint subproblems	Both use recursion, key difference is subproblem overlap
Competitive ratio vs Approximation ratio	Online vs offline; competitive compares to optimal online, approximation compares to optimal offline	Both are ratios measuring suboptimality
Span (parallel) vs Height (tree)	Span = critical path of computation; Height = tree depth	Span was originally called "depth" in parallel computation
Master Theorem vs Akra-Bazzi	Master: same-size subproblems; Akra-Bazzi: different subproblem sizes	Akra-Bazzi generalizes Master for $\sum a_i \cdot T(n/b_i) + f(n)$
Eulerian path vs Hamiltonian path	Eulerian: visit every edge once; Hamiltonian: visit every vertex once	Eulerian pertains to edges; Hamiltonian to vertices
Edges vs Vertices in graphs	E = edges (connections); V = vertices (nodes)	In complexity: $O(V+E)$ vs $O(V^2)$ depends on graph density
Universal hashing vs Perfect hashing	Universal: random function, expected collisions; Perfect: deterministic function with no collisions	Perfect requires knowing keys in advance; universal works for any input
Path compression (union-find) vs Cascading cut (Fib heap)	Both improve data structure performance through structural change	Different contexts but similar concepts of "optimizing while traversing"
Amortized analysis vs Average-case analysis	Amortized: worst-case over sequence; Average-case:	Different notions of "average": over operations vs over inputs

Concept A	Concept B	Key Difference
Prim's algorithm vs Dijkstra's algorithm	expected over random input Prim: relaxation updates key to minimize weight to tree; Dijkstra: relaxation updates distance from source	Both use similar code but different comparison criteria
Transitive closure vs All-pairs shortest paths	TC: reachability (boolean); APSP: distance (numeric)	Floyd-Warshall computes both with different operations
Undirected vs Directed graph DFS	Undirected: only tree and back edges; Directed: all 4 edge types	Directed DFS has more complex edge classification
Heapsort vs Mergesort	Heapsort: in-place, not stable; Mergesort: stable, not in-place	Both $O(n \lg n)$ worst-case but different properties
Decision tree vs Adversary lower bounds	Decision tree: model-based counting leaves; Adversary: adversarial answers to comparisons	Two different techniques for proving lower bounds
BPP vs RP vs ZPP	BPP: error both sides $\leq 1/3$; RP: error NO side only; ZPP: Las Vegas, expected polynomial	RP is a subset of BPP; $ZPP = RP \cap \text{co-RP}$
Pseudopolynomial vs Polynomial	Pseudo: polynomial in numeric value; Poly: polynomial in input length	0-1 knapsack DP is $O(nW)$ — pseudopolynomial because W is exponential in input bits
PTAS vs FPTAS	PTAS: $O(n^{f(1/\epsilon)})$; FPTAS: $O((n \cdot 1/\epsilon)^c)$	FPTAS is polynomial in both n and $1/\epsilon$; stronger than PTAS

Appendix W — NP-Completeness Reduction Strategies

Standard reduction techniques:

1. **Restriction:** Show problem contains a known NP-complete problem as a special case. E.g., SET COVER contains VERTEX-COVER as a restriction (each set has size 2).
2. **Local replacement:** Replace each component of instance with a small gadget. E.g., $3\text{-CNF-SAT} \leq_p \text{CLIQUE}$: each clause becomes a triangle of literal vertices; edges between literals in different clauses unless contradictory.

3. **Component design:** Build complex gadgets that simulate constraints. E.g., VERTEX-COVER $\leq_p$ HAM-CYCLE: selector paths representing edges and vertices; the tour must pick exactly one endpoint per edge.
4. **Gadget encoding:** Encode combinatorial constraints via gadget properties. E.g., 3-CNF-SAT $\leq_p$ 3-COLOR: each variable becomes a TRUE/FALSE vertex; clause gadgets ensure at least one literal per clause is true.
5. **Numerical encoding:** Map constraints to numeric values. E.g., 3-CNF-SAT $\leq_p$ SUBSET-SUM: encode each clause and variable as base-10 digit positions; satisfying assignment corresponds to subset summing to target.
6. **Graph encoding:** Transform problem to graph variant. E.g., CLIQUE $\leq_p$ VERTEX-COVER: complement graph transforms clique into vertex cover.

Common reduction mistakes to avoid: - Reducing FROM your problem TO a known NPC problem (wrong direction — need FROM known NPC to your problem) - Not showing your problem is in NP (certificate + verifier required for NP-completeness) - Using optimization version instead of decision version - Gadget not enforcing constraints correctly - Reduction not polynomial-time (output size blows up)

Quick reference for reductions:

Your Problem Looks Like Reduce From

Clique-like	3-CNF-SAT (clique per clause)
Partition-like	SUBSET-SUM
Coloring-like	3-CNF-SAT (OR-gadgets)
Path/Cycle-like	VERTEX-COVER or HAM-CYCLE
Set selection constraint	3-CNF-SAT or CLIQUE
Numerical constraint	SUBSET-SUM or PARTITION
Packing/Allocation	SET COVER or BIN PACKING
Graph covering	VERTEX-COVER or DOMINATING-SET

Appendix X — Solving Recurrences: Step-by-Step Worked Examples

Example 1: $T(n) = 2T(n/2) + n$ (Merge sort)

Solution: Master Theorem Case 2. $a=2$, $b=2$, $n^{\{\log_b a\}}=n^1=n$.
 $f(n)=n = \theta(n^{\{\log_2 2\}} \cdot \lg^0 n) \rightarrow$ Case 2 with $k=0$.
 $T(n) = \theta(n \lg n)$.

Verification via recursion tree:

Level 0: n work (root)

Level 1: $2 \cdot (n/2) = n$ work

Level 2: $4 \cdot (n/4) = n$ work

...

Level $\lg n$: $2^{\{\lg n\}} \cdot (n/2^{\{\lg n\}}) = n \cdot 1 = n$ work

Total: $n \cdot (\lg n + 1) = \theta(n \lg n)$

Example 2: $T(n) = 9T(n/3) + n$ (divide into 9 subproblems)

Solution: Master Theorem Case 1. $a=9$, $b=3$, $n^{\{\log_3 9\}}=n^2$.

$f(n) = n = O(n^{\{2-\epsilon\}})$ with $\epsilon=1$.

$T(n) = \theta(n^2)$.

Intuition: Top level costs n . Next level: $9 \cdot (n/3) = 3n$.

Level i : $9^i \cdot (n/3^i) = (9/3)^i \cdot n = 3^i \cdot n$.

Growing geometrically – leaves dominate.

Example 3: $T(n) = 3T(n/4) + n \lg n$

Solution: Master Theorem Case 3. $a=3$, $b=4$, $n^{\{\log_4 3\}}=n^{\{0.793\}}$.

$f(n) = n \lg n = \Omega(n^{\{0.793+\epsilon\}})$ with $\epsilon=0.207$.

Regularity: $a \cdot f(n/b) = 3 \cdot (n/4) \lg(n/4) = 0.75n \cdot (\lg n - \lg 4) \leq (0.75)n \cdot \lg n = c \cdot f(n)$ with $c=0.75 < 1$.

$T(n) = \theta(n \lg n)$.

Intuition: Root dominates – each level does less total work.

Example 4: $T(n) = 2T(n/2) + n \lg n$

Solution: Master Theorem Case 2 with $k=1$.

$a=2$, $b=2$, $n^{\{\log_2 2\}}=n$. $f(n)=n \lg n = \theta(n \cdot \lg^1 n)$.

$T(n) = \theta(n \cdot \lg^2 n)$.

Verification via recursion tree:

Level 0: $n \lg n$ work

Level 1: $2 \cdot (n/2) \cdot \lg(n/2) = n \cdot (\lg n - 1) = n \lg n - n$

Level 2: $4 \cdot (n/4) \cdot \lg(n/4) = n \lg n - 2n$

...

Level k : $n \lg n - kn$

Sum over $\lg n$ levels: $\theta(n \cdot \lg^2 n)$ (sum of arithmetic series of $\lg n$ terms)

Example 5: $T(n) = T(n-1) + n$ (Insertion sort worst-case)

Solution: Not a Master Theorem candidate (decrease, not divide).

Unrolling: $T(n)=T(n-1)+n=T(n-2)+(n-1)+n=\dots=T(1)+2+3+\dots+n$.

$T(1)=1 \rightarrow T(n)=1+2+\dots+n=n(n+1)/2=\theta(n^2)$.

Substitution verification: Guess $T(n)=O(n^2)$.

Assume $T(k) \leq ck^2$ for $k < n$. Then

$T(n) \leq c(n-1)^2 + n = c(n^2 - 2n + 1) + n = cn^2 - 2cn + c + n$.

Choose $c \geq 1$: $T(n) \leq cn^2 + (-2cn + c + n)$. For large n , $-2cn + n \leq 0$ when $c \geq 1/2$.

So $T(n) \leq cn^2$ for $c=1$, $n \geq 1$.

Example 6: $T(n) = 2T(\sqrt{n}) + \lg n$

Solution: Substitute $n = 2^m$.

$T(2^m) = 2T(2^{m/2}) + m$.

Let $S(m) = T(2^m)$. Then $S(m) = 2S(m/2) + m$.

By Master Theorem: $a=2, b=2, n^{\log_2 2}=m, f(m)=m \rightarrow$ Case 2.

$S(m) = \Theta(m \lg m)$.

$T(n) = S(\lg n) = \Theta(\lg n \cdot \lg \lg n)$.

Intuition: Each recursion halves the exponent.

Example 7: $T(n) = T(n/3) + T(2n/3) + \Theta(n)$ (unbalanced parent)

Solution: Master Theorem doesn't apply (different subproblem sizes).

Recursion tree:

- Root: $\Theta(n)$ work
- Level 1: $\Theta(n/3) + \Theta(2n/3) = \Theta(n)$ work
- Level 2: $\Theta(n/9) + 2 \cdot \Theta(2n/9) = \Theta(n)$ work
- Each level: $\Theta(n)$ total work
- Depth: longest path $n \rightarrow (2/3) \cdot n \rightarrow \dots \rightarrow 1$, so $\log_{3/2} n$ levels
- Total: $\Theta(n \cdot \log_{3/2} n) = \Theta(n \lg n)$

Example 8: $T(n) = T(n-1) + T(n-2) + \Theta(1)$ (Fibonacci naive)

Solution: Characteristic equation approach.

$T(n) - T(n-1) - T(n-2) = \Theta(1)$. Homogeneous part: $r^2 - r - 1 = 0$.

$r = (1 \pm \sqrt{5})/2 = \phi$ or $-1/\phi$.

$T(n) = c_1 \cdot \phi^n + c_2 \cdot (-1/\phi)^n + [\text{particular solution}]$.

For Fibonacci: $T(n) = \Theta(\phi^n)$ where $\phi = (1 + \sqrt{5})/2 \approx 1.618$.

Better solution: DP approach yields $T(n) = \Theta(n)$ with memoization.

Appendix Y — Exam Question Templates

Template 1: “Prove lower bound” - Input: [problem, model of computation] - Method: [adversary argument / decision tree / reduction] - Construct: [worst-case input / decision tree leaves] - Bound: [show lower bound of $f(n)$] - Match: [optimal algorithm achieves this bound, or gap exists]

Template 2: “Design an algorithm” - Input: [problem description] - Approach: [greedy/DP/D&C/network flow/other] - Data structures: [required DS with justification] - Pseudocode: [key operations] - Correctness: [invariant/induction/exchange argument] - Time: [$O(f(n))$] Space: [$O(g(n))$] - Edge cases: [what about empty input, duplicates, etc.]

Template 3: “Prove NP-completeness” - $L \in \text{NP}$: [certificate of size $O(f(n))$, verifier runs in $O(g(n))$] - Reduction from known NPC: [pick appropriate problem P] - Transformation f: [map instances of P to instances of L] - $(\Rightarrow)$ If P-instance yes $\rightarrow$ L-instance yes: [argument] - $(\Leftarrow)$ If L-instance yes $\rightarrow$ P-instance yes: [argument] - Polynomial: [size of output polynomial in input] - Conclusion: L is NP-complete

Template 4: “Solve recurrence” - Identify: Does Master Theorem apply? (a, b, f(n)) - If yes: compute $n^{\log_b a}$, compare to f(n), state case - Apply formula: write $T(n) = \Theta(\dots)$ - If no: use recursion tree / substitution / Akra-Bazzi - Verify: substitution proof with explicit constants

Template 5: “Trace algorithm” - State initial state: [data structure contents, variable values] - For each iteration: [show operation, intermediate state] - After completion: [final output, final data structure] - Verification: [check invariants, test edge conditions]

Template 6: “Amortized analysis” - Operation sequence: [what operations and in what order] - Method: [aggregate/accounting/potential] - For potential: define Φ , show $\Phi \geq 0$, compute $\hat{c}_i = c_i + \Delta\Phi$ - Sum costs: $\sum \hat{c}_i =$ sum of amortized costs - Conclude: amortized cost = $O(f(n))$

Template 7: “Compare algorithms” - Dimensions: time (worst, average, best), space, stability, in-place, adaptivity, constants - Table format: compare across dimensions - When to use each: [use case A $\rightarrow$ algorithm X; use case B $\rightarrow$ algorithm Y] - Examples: [concrete problem instances]

Appendix Z — Key Equations & Formulas Quick Reference

Recurrences: - Master Theorem: $T(n) = aT(n/b) + f(n) \rightarrow$ compare f(n) to $n^{\log_b a}$ - Binary search: $T(n) = T(n/2) + O(1) \rightarrow O(\lg n)$ - Merge sort: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \lg n)$ - Quickselect: $T(n) = T(3n/4) + O(n) \rightarrow O(n)$ - SELECT (median of medians): $T(n) \leq T(n/5) + T(7n/10) + O(n) \rightarrow O(n)$

Sums: - $\sum_{i=1}^n i = n(n+1)/2$ - $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$ - $\sum_{i=0}^n c^i = (c^{n+1}-1)/(c-1)$ - $\sum_{i=1}^n 1/i = \ln n + \gamma + O(1/n)$ - $\sum_{i=1}^n \lg i = n \lg n - n \lg e + O(\lg n) = \Theta(n \lg n)$

Probability: - $E[X] = \sum \Pr[X \geq t]$ for nonnegative integer X - $\text{Var}[X] = E[X^2] - (E[X])^2$ - $\Pr[A|B] = \Pr[A \cap B] / \Pr[B]$ - Markov: $\Pr[X \geq a] \leq E[X]/a$ - Chebyshev: $\Pr[|X-\mu| \geq k\sigma] \leq 1/k^2$ - Chernoff: $\Pr[S \geq (1+\delta)\mu] \leq e^{-\mu\delta^2/3}$ - Chernoff: $\Pr[S \leq (1-\delta)\mu] \leq e^{-\mu\delta^2/2}$ - Union bound: $\Pr[\cup A_i] \leq \sum \Pr[A_i]$

Inequalities: - $\log_a n = \log_b n / \log_b a$ (change of base) - $n! \geq (n/e)^n$ (Stirling lower bound) - $n! \leq n^n - \lg(n!) = \Theta(n \lg n)$ - $H_n \leq 1 + \lg n$ (bound)

Graph Theory: - $\sum \deg(v) = 2|E|$ (Handshaking) - A connected graph has $|E| \geq |V|-1$ - A tree has exactly $|V|-1$ edges - Planar Euler: $V - E + F = 2$ - Bipartite: no odd cycles - Perfect matching in bipartite: $|A| \leq |N(A)|$ for all $A \subseteq L$

Tree Properties: - Full binary tree with n leaves: n-1 internal nodes - Binary tree of height h: max $2^{h+1} - 1$ nodes - RB tree with n nodes: height $\leq 2 \lg(n+1)$ - B-tree with n keys: height $\leq \log_t ((n+1)/2)$

Flow: - $|f| = \sum_v f(s,v)$ - $|f| \leq c(S,T)$ for any cut - $\max |f| = \min c(S,T)$ - Integer capacities $\rightarrow$ integer max flow

Complexity Classes: - $P \subseteq NP \subseteq PSPACE \subseteq EXP$ - NPC hardest in NP (NP-complete) - co-NP: complement in NP - if $P \neq NP$: NP-intermediate problems exist (graph isomorphism, factoring) - EXPTIME: $2^{\{poly(n)\}}$ time - NEXPTIME: $2^{\{poly(n)\}}$ time with nondeterminism - $L \subseteq NL \subseteq P \subseteq NP \subseteq PH \subseteq PSPACE \subseteq EXP \subseteq NEXP$ - NC: polylog depth, poly size (efficient parallel) - BPP: randomized poly-time with error $\leq 1/3$ - RP: one-sided error randomized poly-time - ZPP = $RP \cap co-RP$: expected poly-time zero-error - #P: counting class (number of accepting paths of NP machine) - PP: probabilistic poly-time (majority of paths accept) - AM, MA: Arthur-Merlin interactive proof classes - IP = PSPACE: interactive proofs characterize PSPACE - BQP: quantum poly-time (bounded error) - QMA: quantum NP (quantum Merlin-Arthur)

Data Structure Formulas: - Load factor $\alpha = n/m$ (hash tables) - Expected chain length in chaining: α - Unsuccessful open-address search probes: $\leq 1/(1-\alpha)$ - Successful open-address search probes: $(1/\alpha) \cdot \ln(1/(1-\alpha))$ - Hash table size for perfect hashing: $O(n^2)$ for first-level, $O(n)$ total - Bloom filter false positive: $(1 - e^{-kn/m})^k$ - Optimal number of hash functions: $k = (m/n) \cdot \ln 2$ - van Emde Boas: space $O(u)$ basic, $O(n)$ with dynamic tables - Disjoint-set: $\alpha(n) \leq 4$ for $n \leq 10^{\{80\}}$ - B-tree height: $h \leq \log_t ((n+1)/2)$ - BST expected height (random): $O(\lg n)$ - RB tree height: $h \leq 2 \cdot \lg(n+1)$

Sorting Formulas: - Comparison-sort lower bound: $\Omega(n \lg n)$ - Counting sort time: $\Theta(n + k)$ - Radix sort time: $\Theta(d(n + k))$ - Bucket sort expected time: $\Theta(n)$ for uniform keys - Inversions in random permutation: expected $n(n-1)/4$ - Quicksort expected comparisons: $\sim 1.39 \cdot n \lg n$ - Merge sort comparisons: $n \lg n - n + 1$ (in best case)

Graph Formulas: - Complete graph K_n : $n(n-1)/2$ edges - Complete bipartite $K_{m,n}$: $m \cdot n$ edges - Tree: exactly $|V|-1$ edges - Connected: at least $|V|-1$ edges - Cycle: at most $3 \cdot |V| - 6$ edges (planar) - Maximum bipartite matching $\min(|L|, |R|)$ - Maximum flow $\leq$ min cut capacity - Augmenting path bound for Ford-Fulkerson: $O(E \cdot |f^*|)$ - Dijkstra with binary heap: $O(E \cdot \lg V)$ - Dijkstra with Fibonacci heap: $O(V \lg V + E)$ - Bellman-Ford: $O(V \cdot E)$ - Floyd-Warshall: $\Theta(V^3)$ - Prim (array): $\Theta(V^2)$ - Prim (binary heap): $O(E \cdot \lg V)$ - Kruskal: $O(E \cdot \lg V)$

DP Formulas: - LCS DP: $O(m \cdot n)$ - Edit distance DP: $O(m \cdot n)$ - Matrix-chain DP: $O(n^3)$ - 0-1 Knapsack DP: $O(n \cdot W)$ (pseudopolynomial) - Rod cutting DP: $O(n^2)$ - LIS DP: $O(n \lg n)$ (with binary search) - Subset sum DP: $O(n \cdot T)$ (pseudopolynomial) - Optimal BST DP: $O(n^3)$, $O(n^2)$ with Knuth optimization - Floyd-Warshall DP: $\Theta(V^3)$ - Longest common substring DP: $O(m \cdot n)$

String Matching Formulas: - Naive: $O(n \cdot m)$ - Rabin-Karp: $O(n+m)$ expected - KMP: $O(n+m)$ worst-case - Finite automaton: $O(n)$ matching + $O(m \cdot |\Sigma|)$ preprocessing - Boyer-Moore-Horspool: $O(n \cdot m)$ worst, sublinear average - Z-algorithm: $O(n+m)$ - Suffix array construction: $O(n)$ (DC3 / SA-IS) - LCP array (Kasai): $O(n)$

Approximation Ratios: - Vertex Cover (greedy): 2 - Vertex Cover (LP rounding): 2 - Set Cover (greedy): $H_n \approx \ln n$ - Set Cover (LP rounding): H_n - TSP with triangle inequality (Christofides): $3/2$ - TSP with triangle

inequality (MST doubling): 2 - Max-Cut (random): $1/2$ - Max-Cut (Goemans-Williamson): ≈ 0.878 - Max-3-SAT (random): $7/8$ - Max-SAT (LP + random rounding): $3/4$ - Bin packing (First-Fit): 1.7 - Bin packing (First-Fit-Decreasing): $11/9 \approx 1.222$ - Knapsack (greedy): 2 - Knapsack (FPTAS): $(1+\epsilon)$ - Steiner tree (MST heuristic): 2 - Edge-disjoint paths (greedy): $O(\sqrt{|E|})$ - Multicommodity flow ($O(\log n)$ approximation): $O(\lg n)$

Algorithmic Number Theory Formulas: - $\gcd(a,b) = \gcd(b, a \bmod b)$ - Extended Euclid: finds x,y such that $ax + by = \gcd(a,b)$ - $a \equiv b \pmod{m}$ means $m \mid (a-b)$ - Modular inverse exists iff $\gcd(a,m) = 1$ - Fermat: $a^{p-1} \equiv 1 \pmod{p}$ for prime $p \nmid a$ - Euler: $a^{\phi(n)} \equiv 1 \pmod{n}$ for $\gcd(a,n) = 1$ - $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$ - $\phi(p) = p-1$ for prime p - $\phi(p \cdot q) = (p-1)(q-1)$ for primes p,q - CRT: solution to $x \equiv a_i \pmod{n_i}$ unique modulo $\prod n_i$ - Miller-Rabin: error $\leq 4^{-k}$ after k independent tests - Quadratic residues: $a^{(p-1)/2} \equiv 1 \pmod{p}$ iff a is QR mod p - Discrete log: find x such that $\alpha^x \equiv \beta \pmod{p}$

Miscellaneous Formulas: - Catalan numbers: $C_n = (1/(n+1)) \cdot C(2n,n)$ - Stirling numbers (2nd kind): $S(n,k) = (1/k!) \cdot \sum_{i=0}^k (-1)^i \cdot C(k,i) \cdot (k-i)^n$ - Inclusion-Exclusion: $|\cup A_i| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$ - Pigeonhole: if n items into m boxes, some box has $\geq \lceil n/m \rceil$ items - Jensen: $f(E[X]) \leq E[f(X)]$ for convex f - AM-GM: $(\sum a_i)/n \geq (\prod a_i)^{1/n}$ - Cauchy-Schwarz: $(\sum a_i b_i)^2 \leq (\sum a_i^2)(\sum b_i^2)$ - Bernoulli: $(1+x)^n \geq 1+nx$ for $x \geq -1$ - Stolz-Cesàro: analogous to L'Hôpital for sequences

Appendix AA — Quick Reference: Paradigm by Input Size

Input Size n	Feasible Algorithms	Infeasible / Impractical
$n \leq 10$	Brute force, exact TSP ($n!$), branch-and-bound	—
$n \leq 20$	Exponential DP over subsets ($2^n \cdot \text{poly}(n)$)	$n!$ for $n > 12$
$n \leq 100$	$O(n^3)$: Floyd-Warshall, matrix-chain	$O(2^n)$ for $n > 30$
$n \leq 10^3$	$O(n^2)$: insertion sort, DP with squared state	$O(n^2)$ for $n > 10^4$
$n \leq 10^5$	$O(n \lg n)$: mergesort, good sorting	$O(n^2)$
$n \leq 10^6$	$O(n)$: linear scan, counting sort (tight range)	$O(n \lg n)$ with high constant
$n \leq 10^8$	$O(\lg n)$: binary search, BST operations	$O(n)$
$n \leq 10^{12}$	$O(1)$: closed-form formulas	$O(\lg n)$ for $n > 2^{60}$
Streaming	$O(1)$ per element: reservoir sampling, bloom filter	Sorting entire stream

Appendix AB — Quick Reference: Problem Reducibility Chart

Problem	Reduces To	Significance
Bipartite matching	Maximum flow	Polynomial reduction; both solvable in P
Bipartite matching	Minimum vertex cover	König's theorem: equality in bipartite graphs
Maximum flow	Minimum cut	Max-flow min-cut theorem; dual LP
Shortest path (single-source)	Difference constraints	Bellman-Ford solves both
Topological sort	DAG shortest paths	Both solvable in $O(V+E)$
SAT	3-CNF-SAT	Clause conversion with auxiliary vars
3-CNF-SAT	CLIQUE	Literal-per-vertex, edges between non-conflicting
3-CNF-SAT	SUBSET-SUM	Base-10 encoding per clause and variable
3-CNF-SAT	3-COLOR	Palette triangle + OR gadgets
CLIQUE	VERTEX-COVER	Complement graph
VERTEX-COVER	HAM-CYCLE	Complex edge/vertex selector gadgets
HAM-CYCLE	TSP	Complete graph with 0/1 costs
SUBSET-SUM	PARTITION	Add target-sum element
3-COLOR	3-CNF-SAT	Clause and variable gadgets
INDEPENDENT-SET	CLIQUE	Complement graph
CLIQUE	INDEPENDENT-SET	Complement graph
BIN PACKING	PARTITION	Bin capacity = half total sum
HITTING SET	VERTEX-COVER	Each set size 2 $\rightarrow$ vertex cover
MAX-CUT	MAX-2-SAT	Approximation preserving
MAX-3-SAT	MAX-CUT	Gadget construction (approximation preserving)
Integer programming	0-1 Knapsack	Special case
PRIMES	COMPOSITES	Complement decision problem

End of Study Guide — Lines: ~5000

Last updated: 2026-06-04. Coverage: Chapters 1-35 of CLRS 4e including all algorithms, data structures, graph theory, NP-completeness, approximation, online/parallel algorithms, string matching, number theory, and FFT.